

ОПД ЭКЗАМЕН. ПРОСТЫМ ЯЗЫКОМ

1) Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я, двоично десятичная.

[\(51\) Аналоговые дискретные и цифровые сигналы - YouTube](#)

Представление информации в цифровой и аналоговой форме

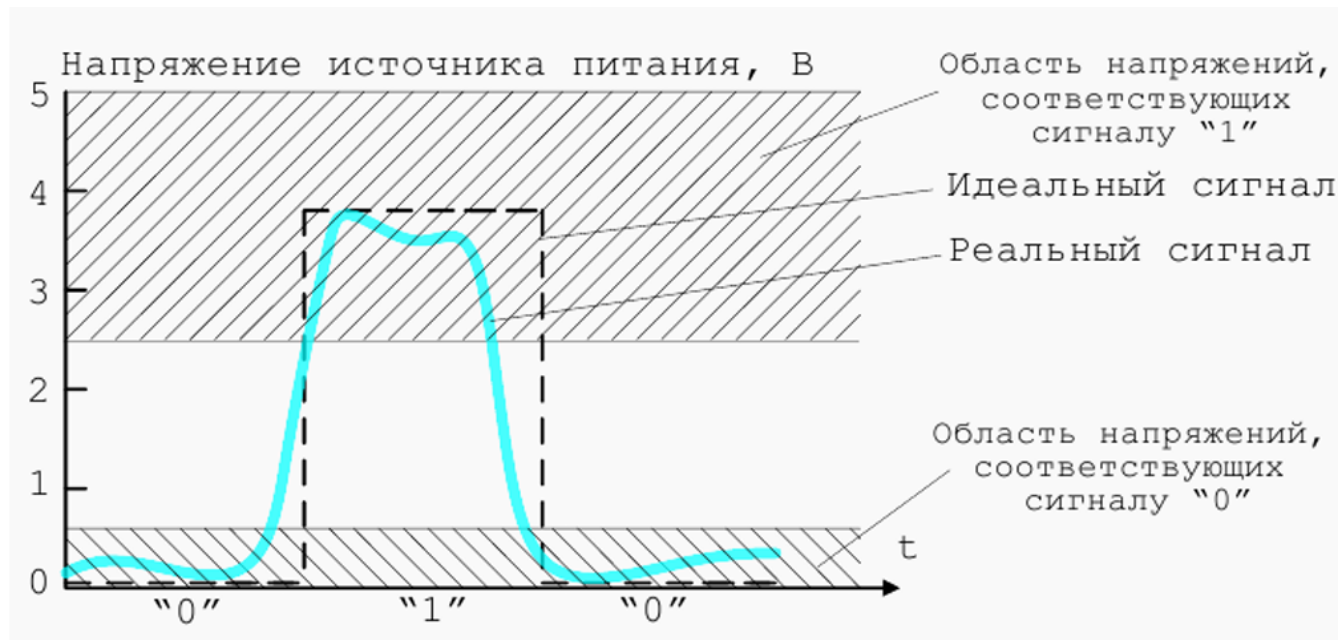
Аналоговая форма

Аналоговая (непрерывная) форма представления информации используется для описания величин, которые могут принимать бесконечное количество значений в определённом диапазоне. Например, температура может изменяться плавно от одного значения к другому без скачков и перерывов. Это свойство называется непрерывностью. В аналоговых вычислительных машинах информация представляется в виде одного сигнала, который пропорционален величине, которую он представляет.

Цифровая форма

Цифровая (дискретная) форма представления информации описывает величины, которые могут принимать только определённые значения, не все возможные. Эти величины называются дискретными. В цифровой форме информация представляется в виде набора сигналов, каждый из которых соответствует одной из цифр. Например, число 123 может быть

представлено тремя сигналами, где каждый сигнал соответствует цифре: 1, 2 и 3.



Преобразование аналоговой информации в цифровую

Электронные вычислительные машины (ЭВМ) используют цифровую форму представления информации. Это значит, что все данные в ЭВМ кодируются в виде электрических сигналов, которые могут принимать два состояния: наличие или отсутствие сигнала, что соответствует 0 и 1 в двоичной системе счисления.

Базовые понятия в цифровых системах

1. **Бит (binary digit)** — минимальная единица информации, которая может быть 0 или 1.
2. **Байт** — группа из восьми бит.
3. **Слово** — фиксированная длина данных, хранящаяся в ячейке памяти, обычно кратная байту.

Системы счисления

1. **Двоичная система счисления (2СС)** — основная система для внутренних операций в ЭВМ, где используются только цифры 0 и 1. Значение цифры зависит от её позиции в числе.
2. **Восьмеричная система счисления (8СС)** — используется для удобства представления длинных двоичных чисел, состоит из цифр от 0 до 7. Каждая цифра в восьмеричной системе соответствует группе из трёх двоичных цифр.
3. **Шестнадцатеричная система счисления (16СС)** — также применяется для сокращения записи двоичных чисел, состоит из цифр от 0 до 9 и букв от А до F. Каждая шестнадцатеричная цифра соответствует группе из четырёх двоичных цифр.
4. **Двоично-десятичная система счисления (2-10СС)** — используется в устройствах, где результаты необходимо представлять в десятичном виде. Здесь каждая десятичная цифра представляется четырьмя двоичными разрядами (от 0000 до 1001).

Важные концепции:

- **Преобразование двоичных чисел в восьмеричные и шестнадцатеричные:** Двоичные числа делятся на группы по три цифры для восьмеричной системы или по четыре цифры для шестнадцатеричной системы, и каждая группа заменяется соответствующей цифрой новой системы счисления.
- **Преобразование двоичных чисел в двоично-десятичную систему:** Каждая десятичная цифра представляется четырьмя битами, и таким образом, два десятичных числа могут быть представлены одним байтом.

Диаграмма сигнала

На изображении показана диаграмма напряжения источника питания во времени:

- **Идеальный сигнал** (пунктирная линия) — идеальное представление цифрового сигнала, где значения "0" и "1" чётко определены.

- **Реальный сигнал** (голубая линия) — реальное поведение сигнала, которое может включать в себя некоторые отклонения и шум.
- **Области напряжений, соответствующих сигналу "1" и "0"** — определённые диапазоны напряжений, которые считаются логической единицей или нулём.

Эти представления помогают понять, как цифровые системы интерпретируют и обрабатывают электрические сигналы, переводя их в цифровую информацию для дальнейшей обработки.

2) Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. При размещении таких чисел в одном 16-разрядном слове они могут

изменяться от $(0000\ 0000\ 0000\ 0000)_2 = (0000)_{16} = 0$ до $(1111\ 1111\ 1111\ 1111)_2 = (FFFF)_{16} = 2^{16} - 1 = 65535$. Такая запись называется прямым кодом числа. Подобные числа (так же как и рассмотренные ниже двоичные числа со знаком) относятся к числам с фиксированной запятой, разделяющей целую и дробную части числа. В числах, используемых в базовой ЭВМ, положение запятой строго фиксировано после младшего бита слова.

Целые двоичные числа со знаком используются тогда, когда необходимо различать положительные и отрицательные числа. В современных ЭВМ для представления целых чисел со знаком используется дополнительный код, в котором старший бит формата определяет знак числа: 0 - для положительных чисел и 1 - для отрицательных чисел. При этом дополнительный код положительного числа

совпадает с его прямым кодом. А для представления отрицательного числа в дополнительном коде производится инвертирование прямого кода модуля числа

(получение обратного кода числа) и добавление к результату единицы.

Такая же

операция используется при изменении знака числа, представленного в дополнительном коде.

Использование дополнительного кода упрощает конструкцию ЭВМ, так как при сложении двух таких чисел, имеющих разные знаки, не требуется переходить к операциям вычитания меньшего (по модулю) числа из большего и присвоения

результату знака большего числа. Кроме того, одной и той же схемой сумматора можно воспользоваться для выполнения операций над знаковым и беззнаковым представлением числа.

Признаком выхода за границы разрядной сетки для беззнакового представления числа

является перенос в старший разряд (бит C - Carry). Например, при сложении:

+1000 0000 0000 0000	█
1000 0000 0000 0000	█
1 0000 0000 0000 0000	█

В ответе должно получиться $32768+32768=65536$, но т.к. разрядность слова составляет лишь 16

бит, то в нем сохраняется только часть результата, т.е. 0. Единица, возникшая вследствие переноса оказалась в несуществующем 17 разряде.

Признаком переполнения разрядной сетки для знакового представления является бит переполнения (OVerflow). Разные знаки слагаемых, или совпадение знаков

слагаемых со знаком суммы свидетельствуют о том что результат корректен. в противном случае формируется сигнал – Переполнение

Признак отрицательного результата N при знаковом представлении выставляется в случае

когда в старшем разряде числа в доп. коде находится 1

Признак нулевого результата Z выставляется в случае когда все разряды числа равны 0

3) Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, KOI-8, ISO8859-5, Windows 1251, UTF-8, UTF-16.

Представление текстовой информации в вычислительных машинах основано на кодировании букв алфавитов для существующих на земле языков, которые традиционно используются человечеством. Символ - это графическое изображение, которое используется человеком для создания слов, текстов и другой значимой информации. Как известно, к символам относятся буквы, знаки препинания, символы валют, цифры и т.д.

В ЭВМ представление текстовой информации основывается на кодировании букв и символов при помощи кодовой таблицы. Кодовая таблица (или кодировка символов) является соглашением между разработчиками о соответствии каждому символу определенного порядкового номера или кода, чтобы его можно было сохранять в памяти ЭВМ или передавать по каналам связи. Для хранения графических начертаний символов в ЭВМ существуют шрифты. Шрифты бывают векторные и растровые. В растровых шрифтах каждому коду символа соответствует изображение, определенного (в точках) размера. В векторных хранится принцип начертания символа в виде последовательности линий.

Кодировки стандарта ASCII

ASCII — таблицы кодировок, в которых содержатся основные символы (английский алфавит, цифры, знаки препинания, символы национальных алфавитов(свои для каждого региона), служебные символы) и длина кода каждого символа $n = 8$ бит.

7 бит:

ASCII — первая кодировка, пригодная для работы с текстом. Помимо маленьких букв английского алфавита и служебных символов, содержит большие буквы английского языка, цифры, знаки препинания и другие символы (при этом старший бит использовался для контроля четности битов передаваемого по каналом связи символа).

Кодировки стандарта ASCII (8 бит):

ISO 8859 — первая кодировка, в которой стало возможно использовать символы национальных алфавитов.

КОИ8-R — первая русская кодировка. Символы кириллицы расположены не в алфавитном порядке. Их разместили в верхнюю половину таблицы так, чтобы позиции кириллических символов соответствовали их фонетическим аналогам в английском алфавите. Это значит, что даже при потере старшего бита каждого символа, например, при проходе через устаревший семибитный модем, текст остается "читаемым" (отсюда и появилось понятие транслита).

Windows-1251 — русская кодировка, использовавшаяся в русскоязычных версиях операционной системы Windows в начале 90-х годов.

Кириллические символы идут в алфавитном порядке. Содержит все символы, встречающиеся в типографике обычного текста (кроме знака ударения).

Кодировки стандарта UNICODE

Юникод — это промышленный стандарт, обеспечивающий цифровое представление символов всех письменностей мира и специальных символов.

Стандарт предложен в 1991 году некоммерческой организацией «Консорциум Юникода» (англ. Unicode Consortium). Применение этого стандарта позволяет закодировать очень большое число символов из разных письменностей. Стандарт состоит из двух основных разделов: универсальный набор символов (англ. UCS, universal character set) и семейство кодировок (англ. UTF, Unicode transformation format). Универсальный набор символов задаёт однозначное соответствие символов кодам — элементам кодового пространства, представляющим неотрицательные целые числа. Семейство кодировок определяет машинное представление последовательности кодов UCS.

Способы представления

Юникод имеет несколько форм представления (англ. Unicode Transformation Format): UTF-8, UTF 16 (UTF-16BE, UTF-16LE) и UTF-32 (UTF-32BE, UTF-32LE).

UTF-8

UTF-8 — представление Юникода, обеспечивающее наилучшую совместимость со старыми системами, использовавшими 8-битные символы. Текст, состоящий только из символов с номером меньше 128, при записи в UTF-8 превращается в обычный текст ASCII. И наоборот, в тексте UTF-8 любой байт со значением меньше 128 изображает символ ASCII с тем же кодом. Остальные символы Юникода изображаются последовательностями длиной от двух до шести байт.

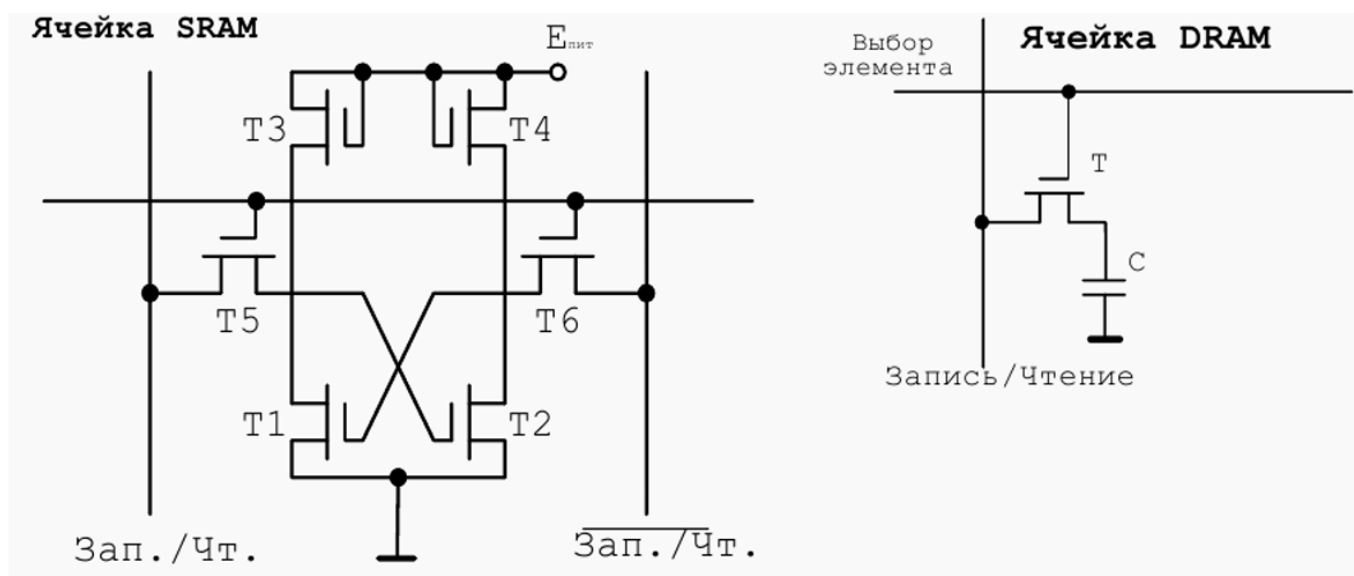
UTF-16

Первая версия Юникода (1991 г.) представляла собой 16-битную кодировку с фиксированной шириной символа; общее число разных символов было 65 536. Во второй версии Юникода (1996 г.) было решено значительно расширить кодовую область; для сохранения совместимости с теми системами, где уже был реализован 16-битный Юникод, и была создана UTF-16. Область 0xD800— 0xDFFF, отведённая для суррогатных пар, ранее принадлежала к области «символов для частного использования».

Поскольку в UTF-16 можно отобразить 1 112 064 символов, то это число и было выбрано в качестве новой величины кодового пространства Юникода.

4) Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счетчики, сумматоры.

Ячейка памяти – это минимальная единица памяти в компьютере, в которой хранятся данные. Каждая ячейка имеет уникальный адрес, по которому к ней обращаются.



SRAM и DRAM

- **SRAM (статическая память с произвольным доступом):** Она построена на транзисторах и не требует постоянного обновления данных. Данные в такой памяти хранятся, пока есть питание. Схема слева показывает, что SRAM состоит из шести транзисторов (T1-T6).
- **DRAM (динамическая память с произвольным доступом):** Здесь данные хранятся в конденсаторе, который нужно периодически обновлять, чтобы сохранить информацию. Схема справа показывает,

что DRAM состоит из одного транзистора и одного конденсатора (Т и С).

Чтение информации

Чтение информации – процесс извлечения данных из ячейки памяти без изменения её содержимого.

Шина

Шина – это набор проводов, соединяющих различные части компьютера, позволяя им обмениваться данными. Шины бывают однонаправленными (данные идут в одну сторону) и двунаправленными (данные могут идти в обе стороны).

Регистр процессора

Регистр процессора – это небольшая, но быстрая память внутри процессора, используемая для хранения данных и промежуточных результатов вычислений.

Двунаправленная шина

Двунаправленная шина – шина, по которой данные могут передаваться в обе стороны, в зависимости от управляющих сигналов.

Вентильная схема

Вентильная схема – электронная схема, управляющая потоком данных. Если управляющий сигнал равен 1, данные проходят, если 0 – блокируются.

Тактовый генератор

Тактовый генератор – устройство, которое генерирует регулярные электрические импульсы, синхронизирующие работу всех компонентов компьютера.

Функциональная логическая схема

Функциональная логическая схема – комбинация логических элементов, выполняющих определённые логические операции над входными сигналами.

Триггер

Триггер – электронное устройство, которое может находиться в одном из двух устойчивых состояний (0 или 1) и переключаться между ними под воздействием внешнего сигнала. Триггер используется для хранения одного бита информации. Отличительной особенностью триггера как функционального устройства является свойство запоминания двоичной информации. Под памятью триггера подразумевают способность оставаться в одном из двух состояний и после прекращения действия переключающего сигнала.

Счетчик числа импульсов

Счетчик числа импульсов – устройство, которое считает количество поступивших на него электрических импульсов и отображает их в виде двоичного или десятичного кода.

Сумматор

Сумматор – устройство, которое складывает два или более входных сигнала и выдает результат в виде одного выходного сигнала.

Эти компоненты и схемы являются основой работы любой вычислительной системы, помогая обрабатывать, хранить и передавать данные внутри компьютера.

5) Структура и принцип функционирования ЭВМ. Порядок

функционирования простого процессора на примере калькулятора.

Основные компоненты ЭВМ и их функции

ЭВМ (электронно-вычислительная машина) состоит из трех основных частей:

1. **Процессор** – выполняет вычисления и управление.
2. **Память** – хранит данные и программы.
3. **Устройства ввода-вывода** – взаимодействуют с пользователем.

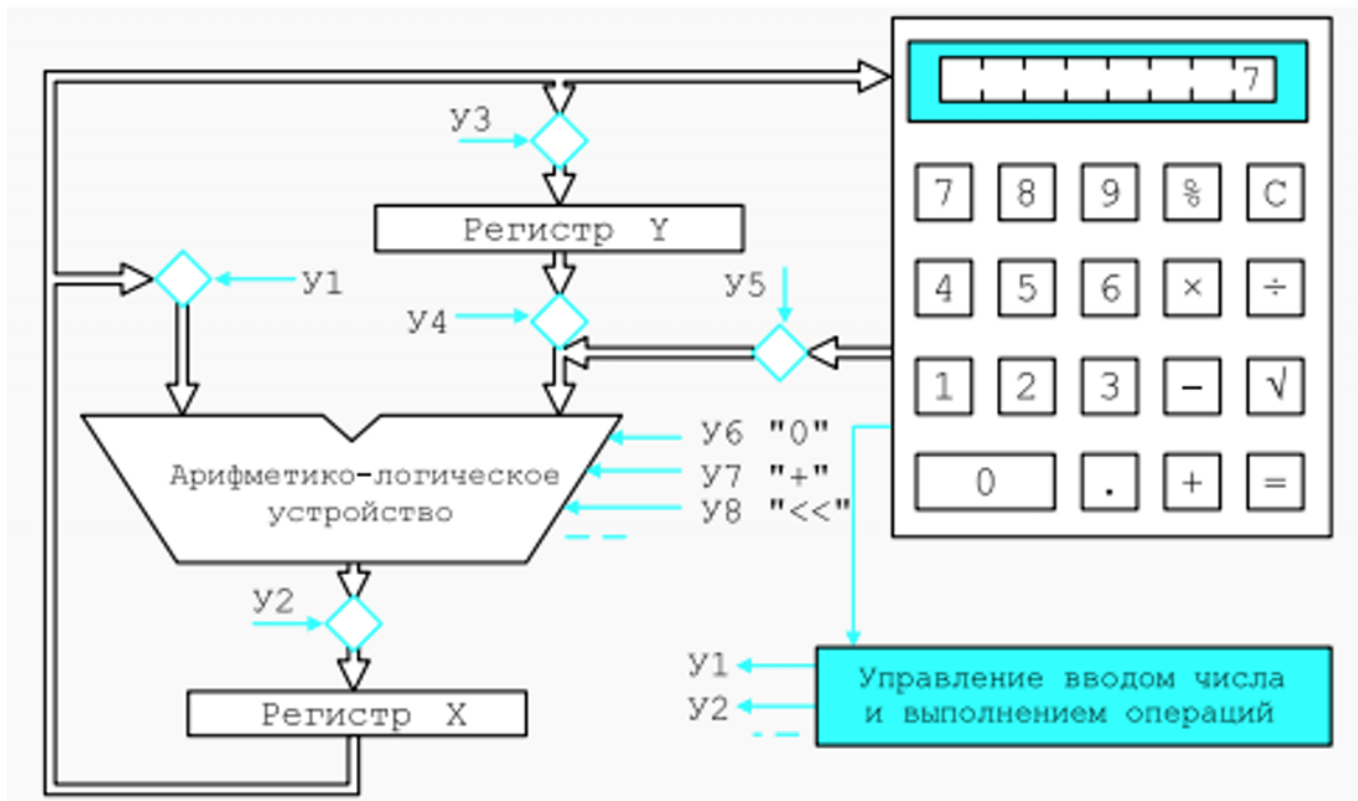
Процессор

Процессор является центральным элементом ЭВМ. Он состоит из нескольких важных частей:

- **Устройство управления** – выбирает команды из памяти и выполняет их.
- **Арифметико-логическое устройство (АЛУ)** – выполняет арифметические и логические операции.
- **Регистры** – временные хранилища данных и состояний процессора.
- **Схемы управления и связи** – обеспечивают взаимодействие процессора с памятью и устройствами ввода-вывода.

Память

Память включает устройство для хранения команд и данных. Память состоит из ячеек, каждая из которых хранит одно слово информации. Ячейки имеют уникальные адреса, что позволяет процессору обращаться к ним.



Принципы работы калькулятора

Давайте рассмотрим, как работает простой калькулятор, используя приведенную схему.

1. **Регистры X и Y:** Калькулятор имеет два регистра (X и Y), которые хранят введенные пользователем числа и промежуточные результаты.
2. **АЛУ (арифметико-логическое устройство):** Выполняет арифметические операции (сложение, вычитание и т.д.).
3. **Шины и управляющие вентили:** Эти компоненты передают данные между функциональными блоками калькулятора.
4. **Устройство управления (УУ):** Контролирует работу всех компонентов калькулятора, формируя последовательность управляющих импульсов.
5. **Клавиатура и дисплей:** Клавиатура передает команды в устройство управления, а дисплей показывает содержимое регистра X.

Пример работы калькулятора

Рассмотрим, как калькулятор обрабатывает ввод числа 7:

1. **Ввод числа 7:** Пользователь нажимает клавишу "7".

- **Передача значения:** Клавиатура передает значение на вентиль У5, и запускается УУ, которое формирует последовательность импульсов.
- **Сохранение предыдущего значения:** УУ сохраняет текущее значение регистра X в регистр Y (вентиль У3).
- **Обнуление регистра X:** УУ обнуляет регистр X (вентиль У6), подготавливая его для новой цифры.
- **Ввод новой цифры:** Введенное число 7 поступает на правый вход АЛУ (вентиль У5), а на левый вход поступает 0. Выбирается операция сложения (вентиль У7).
- **Запись результата:** Результат сложения (7) записывается в регистр X (вентиль У2) и отображается на дисплее.

2. **Ввод второй и последующих цифр:** Для ввода следующей цифры, например, 5, выполняются следующие шаги:

- **Сдвиг содержимого регистра X:** Содержимое регистра X (7) сдвигается на один разряд влево (управляющий сигнал У8), что соответствует умножению на 10.
- **Сложение с новой цифрой:** Сдвинутое значение (70) складывается с новой цифрой (5) (вентиль У7).
- **Запись результата:** Результат (75) записывается в регистр X и отображается на дисплее.

3. **Выполнение операций сложения:** При нажатии кнопки "+" или "=":

- **Сложение значений регистров:** Содержимое регистров X и Y складывается (вентиль У7).
- **Запись результата:** Результат записывается в регистр X и отображается на дисплее.

Вывод

Калькулятор – это простейшая модель ЭВМ, в которой все процессы происходят последовательно и синхронизируются тактовым генератором.

Устройство управления контролирует каждый шаг, обеспечивая правильное выполнение операций и взаимодействие всех компонентов.

Эти базовые принципы применяются и в более сложных ЭВМ, где количество регистров, типов операций и управляющих сигналов значительно больше, но суть остается той же – обработка и хранение данных с использованием регистров, АЛУ и управляющих устройств.

6) Операционная система Unix — ядро ОС и файловая система.

[\(51\) YouTube](#)

Unix — что это такое? Unix — это семейство переносимых, многозадачных и многопользовательских операционных систем. Эти системы сыграли огромную роль в развитии компьютерных операционных систем и признаны одними из самых важных в истории.

Основные характеристики Unix:

1. **Многопользовательская и многозадачная система:** В Unix одновременно могут работать множество пользователей, выполняя разные вычислительные задачи (процессы). В отличие от Microsoft Windows, где, за исключением серверных версий, в один момент за компьютером может работать только один человек, Unix позволяет одновременную работу многих пользователей.
2. **Мультиплатформенность:** Ядро Unix написано так, что его можно адаптировать под практически любой микропроцессор.

Ядро UNIX-подобных операционных систем

Ядро (или Kernel) — это центральная часть операционной системы, которая управляет всеми основными функциями компьютера. В UNIX-подобных операционных системах ядро выполняет несколько ключевых задач:

1. **Управление процессами**

- **Процесс** — это запущенная программа. Ядро отвечает за создание, планирование и завершение процессов.
- **Многозадачность**: Ядро позволяет нескольким процессам выполняться одновременно, распределяя ресурсы процессора между ними.

2. Управление памятью

- **Виртуальная память**: Ядро использует виртуальную память, чтобы программы могли работать с большими объемами данных, чем фактически доступная физическая память (оперативная память).
- **Защита памяти**: Ядро защищает память каждого процесса от доступа со стороны других процессов, что предотвращает ошибки и повышает безопасность системы.

3. Управление файлами

- **Файловая система**: Ядро управляет файловой системой, организуя хранение данных на дисках. Все файлы и каталоги находятся в единой иерархической структуре.
- **Доступ к файлам**: Ядро предоставляет механизм для чтения, записи и управления файлами и каталогами, контролируя доступ к ним.

4. Управление устройствами ввода-вывода

- **Драйверы устройств**: Ядро взаимодействует с аппаратным обеспечением через драйверы устройств, которые позволяют программам работать с различными устройствами (например, принтерами, жесткими дисками, сетевыми картами).
- **Абстракция устройств**: Ядро предоставляет единый интерфейс для работы с устройствами, скрывая от программ детали аппаратной реализации.

5. Сетевые функции

- **Сетевое взаимодействие**: Ядро поддерживает работу в сети, позволяя обмениваться данными между компьютерами. Это включает поддержку различных сетевых протоколов и интерфейсов.

Основные компоненты ядра

1. Менеджер процессов

- Управляет созданием, выполнением и завершением процессов. Планировщик процессов распределяет процессорное время между активными процессами.

2. Менеджер памяти

- Управляет распределением и использованием памяти, обеспечивая виртуальную память и защиту памяти.

3. Менеджер файлов

- Отвечает за доступ к файловой системе, организует чтение и запись данных на дисковые накопители.

4. Менеджер устройств

- Обеспечивает взаимодействие с аппаратными устройствами через драйверы.

5. Менеджер сетей

- Обрабатывает сетевые запросы и обеспечивает сетевую коммуникацию между устройствами.

Пример работы ядра

Когда пользователь запускает программу, например текстовый редактор, ядро выполняет следующие действия:

1. **Создание процесса:** Ядро создает новый процесс для текстового редактора.
2. **Выделение памяти:** Ядро выделяет необходимую память для работы программы.
3. **Открытие файлов:** Если текстовый редактор открывает файл, ядро предоставляет доступ к файлу, контролируя права доступа.
4. **Обработка ввода-вывода:** Когда пользователь вводит текст или сохраняет файл, ядро управляет взаимодействием с клавиатурой и диском.

5. **Управление ресурсами:** Ядро распределяет процессорное время между текстовым редактором и другими активными процессами.

Заключение

Ядро UNIX-подобных операционных систем — это "мозг" системы, который управляет всеми основными функциями компьютера, обеспечивая эффективное и безопасное выполнение программ и взаимодействие с аппаратным обеспечением.

Отличительные особенности Unix:

1. **Простые текстовые файлы для настройки и управления системой:** В Unix настройка системы осуществляется через текстовые файлы, что делает процесс гибким и понятным.
2. **Широкое применение утилит командной строки:** Unix активно использует команды, которые можно запускать из командной строки, что предоставляет мощные инструменты для работы.
3. **Взаимодействие с пользователем через виртуальные устройства — терминалы:** Пользователи работают с системой через терминалы, которые представляют собой виртуальные устройства.
4. **Представление устройств в виде файлов:** Все физические и виртуальные устройства, а также некоторые механизмы межпроцессового взаимодействия представлены в виде файлов.
5. **Использование конвейеров из программ:** В Unix программы могут быть объединены в конвейеры, где каждая программа выполняет свою задачу и передает результат следующей программе.

Файловая система Unix: В Unix файл — это хранилище двоичных и символьных данных, организованных как поток байтов. Файлы в Unix играют ключевую роль и расположены в файловой системе, которая представляет собой дерево. Вершины дерева — это каталоги, а листья — файлы и пустые каталоги.

Структура файловой системы:

- **Корневой каталог (/)** — корень всей файловой системы.
- **Уникальные пути:** Каждый файл и каталог имеют уникальный полный путь.
- **Относительные пути:** Пути к файлам могут быть указаны относительно текущего рабочего каталога.
- **Специальные ссылки:** В каждом каталоге есть два специальных файла-ссылки — «.» (текущий каталог) и «..» (родительский каталог).

Иноды (inode): Индексный узел (inode) — это уникальный идентификатор файла в Unix. Он содержит всю информацию о файле, кроме имени и данных самого файла. В inode хранится:

- Тип файла (например, обычный файл, каталог).
- Права доступа к файлу.
- Время создания и модификации файла.
- Другие метаданные, такие как количество ссылок на файл, размер файла и указатели на блоки данных.

Дополнительные сведения об inode: Индексные узлы являются ключевыми элементами в файловой системе Unix, так как они позволяют системе быстро находить и управлять файлами. Каждый файл имеет свой уникальный inode, который можно рассматривать как паспорт файла, содержащий всю необходимую информацию для его идентификации и управления.

Таким образом, Unix является мощной и гибкой операционной системой, которая благодаря своим уникальным характеристикам и структуре файловой системы остаётся востребованной и популярной во многих областях.

7) Операционная система Unix — интерпретаторы, стандартные потоки ввода вывода, фильтры.

Операционная система Unix предоставляет мощные средства для работы с командной строкой и управления потоками данных. Давайте разберем эту тему более детально и простыми словами.

Интерпретаторы команд

Командный интерпретатор (или оболочка, shell) — это программа, которая позволяет пользователю вводить команды в командной строке. Оболочка переводит введенные команды в формат, понятный операционной системе. Наиболее популярные оболочки в Unix — это sh, bash (стандарт в Unix), и c shell.

Пользователи могут вводить команды по одной или использовать скрипты — файлы, содержащие последовательности команд. Эти команды могут быть:

- Системные утилиты (например, команды для управления файлами).
- Прикладные утилиты (программы, написанные пользователем).
- Управляющие конструкции (условия, циклы и т.д.).

Оболочка также управляет перенаправлением потоков ввода-вывода, что позволяет более гибко взаимодействовать с данными.

Стандартные потоки ввода-вывода

Unix использует три основных потока для взаимодействия с данными:

1. **Стандартный ввод (stdin)** — поток, из которого программы читают данные. Обычно это клавиатура.
2. **Стандартный вывод (stdout)** — поток, в который программы записывают результаты своей работы. Обычно это экран.
3. **Стандартный поток ошибок (stderr)** — поток, в который записываются сообщения об ошибках и другая служебная информация.

Эти потоки привязаны к файловым дескрипторам:

- `0` — `stdin`
- `1` — `stdout`
- `2` — `stderr`

Перенаправление потоков

Unix позволяет перенаправлять потоки, что дает возможность гибко управлять данными. Рассмотрим основные команды перенаправления:

- `> файл` — перенаправить `stdout` в файл (перезаписать файл).
- `>> файл` — добавить `stdout` в конец файла.
- `2> файл` — перенаправить `stderr` в файл.
- `2>> файл` — добавить `stderr` в конец файла.
- `< файл` — использовать файл в качестве источника `stdin`.
- `<< EOF` — записать в `stdin` из терминала до символов «EOF».
- `команда1 | команда2` — использовать вывод команды₁ в качестве ввода для команды₂ (конвейер).

Примеры перенаправления

1. Перенаправление вывода в файл:

```
ls > output.txt
```

Эта команда запишет результат выполнения команды `ls` в файл `output.txt`.

2. Добавление вывода в конец файла:

```
echo "Hello" >> output.txt
```

Эта команда добавит строку "Hello" в конец файла `output.txt`.

3. Перенаправление ошибок в файл:

```
ls non_existing_file 2> error.log
```

Эта команда запишет сообщение об ошибке, вызванное отсутствием файла, в файл `error.log`.

4. Использование файла как источник ввода:

```
sort < unsorted.txt > sorted.txt
```

Эта команда отсортирует строки из файла `unsorted.txt` и запишет результат в `sorted.txt`.

5. Конвейер (pipe):

```
ls | sort
```

Вывод команды `ls` будет использован как ввод для команды `sort`, что приведет к сортировке списка файлов.

Фильтры

Фильтры — это утилиты, которые обрабатывают данные на входе и выводят результат. Примеры фильтров:

- **wc** — подсчет слов, строк и символов в файле.
- **grep** — поиск строк, соответствующих шаблону.
- **sort** — сортировка строк.

Пример использования фильтров

1. Подсчет строк в файле:

```
wc -l file.txt
```

Эта команда выведет количество строк в файле `file.txt`.

2. Поиск строки, содержащей слово "error":

```
grep "error" file.txt
```

Эта команда выведет все строки из файла `file.txt`, содержащие слово "error".

3. Сортировка строк в файле:

```
sort file.txt
```

Эта команда отсортирует строки в файле `file.txt` и выведет результат.

Таким образом, Unix предоставляет мощные средства для работы с командной строкой и потоками данных, позволяя пользователям эффективно выполнять различные задачи и автоматизировать процессы.

8) Операционная система Unix — основные команды, права файлов и способы их задания.

Давайте рассмотрим основные команды Unix, которые помогают управлять файлами и каталогами, и разберём их простыми словами.

Команды для работы с файлами и каталогами

1. **touch** файл

- Создаёт пустой файл. Если файл уже существует, обновляет время его последней модификации.

2. **mkdir** каталог

- Создаёт новый пустой каталог (папку).

3. **rm** файл

- Удаляет файл. Опция `-r` позволяет рекурсивно удалять каталоги и их содержимое. Без этой опции можно удалить только файлы, не каталоги.

4. **rmdir** каталог

- Удаляет только пустые каталоги.

5. **echo**

- Выводит строку текста на экран или в файл.

6. **cat** файл

- Показывает содержимое файла.

7. **pwd**

- Показывает полный путь текущего рабочего каталога.

8. ls файл

- Показывает список файлов в каталоге или информацию о файле.

Опции:

- `-l` — выводит подробную информацию о каждом файле.
- `-a` — показывает также скрытые файлы.
- `-F` — добавляет к имени файла его тип.
- `-R` — рекурсивно показывает содержимое всех подкаталогов.

9. cd каталог

- Переходит в указанный каталог.

10. cp файл1 файл2

- Копирует файл в другой файл.

11. mv файл каталог

- Перемещает файл в указанный каталог.

12. ln файл1 файл2

- Создаёт новую жёсткую ссылку на файл. Опция `-s` создаёт символическую ссылку, которая может указывать на любой файл, независимо от его расположения.

Команды для вывода и поиска

1. head/tail файл

- Показывает первые (`head`) или последние (`tail`) строки файла.

Опции:

- `-n` — показывает первые/последние n строк.
- `-c` — показывает первые/последние n байт.

2. wc файл

- Показывает количество строк, слов и байт в файле. Опции:

- `-l` — только количество строк.
- `-w` — только количество слов.
- `-c` — только количество байт.
- `-m` — количество символов.

3. **find** выражение

- Ищет файлы в иерархии каталогов по заданным параметрам.

4. **man** команда

- Показывает справочную информацию по указанной команде.

Права доступа к файлам

Каждый файл в Unix имеет права доступа, которые определяют, кто и что может с ним делать. Существует три основных типа прав:

- **r (read)** — право на чтение файла или просмотр содержимого каталога.
- **w (write)** — право на запись в файл или изменение содержимого каталога.
- **x (execute)** — право на выполнение файла или поиск и переход в каталог.

Также есть три категории пользователей:

- **u (user)** — владелец файла.
- **g (group)** — члены группы, владеющей файлом.
- **o (others)** — все остальные пользователи.

Права записываются в виде последовательности из 9 символов, по три для каждой категории:

- Первый набор символов (rwx) — для владельца.
- Второй набор символов (rwx) — для группы.
- Третий набор символов (rwx) — для всех остальных.

Например, права `rw xr - -` означают, что владелец может читать, записывать и выполнять файл, группа может только читать и выполнять, а остальные могут только читать.

Установка прав с помощью команды **chmod**

Для изменения прав доступа используется команда `chmod` с различными форматами записи:

1. Символьный формат

```
chmod u+x файл
```

Разрешает выполнение файла владельцем.

```
chmod ugo+r файл
```

Разрешает чтение файла всем.

2. Восьмеричный формат

```
sh
```

[Копировать код](#)

```
chmod 755 файл
```

- Устанавливает права `rwxr-xr-x`. Владелец может читать, записывать и выполнять, группа и остальные могут только читать и выполнять.

3. Копирование прав

```
sh
```

[Копировать код](#)

```
chmod g=u файл
```

- Копирует права владельца на группу.

Примеры использования прав доступа

1. Добавить право на выполнение для владельца:

sh

 Копировать код

```
chmod u+x файл
```

2. Разрешить выполнение для всех:

sh

 Копировать код

```
chmod ugo+x файл
```

3. Запретить выполнение для остальных:

sh

 Копировать код

```
chmod o-x файл
```

4. Установить все права для всех категорий:

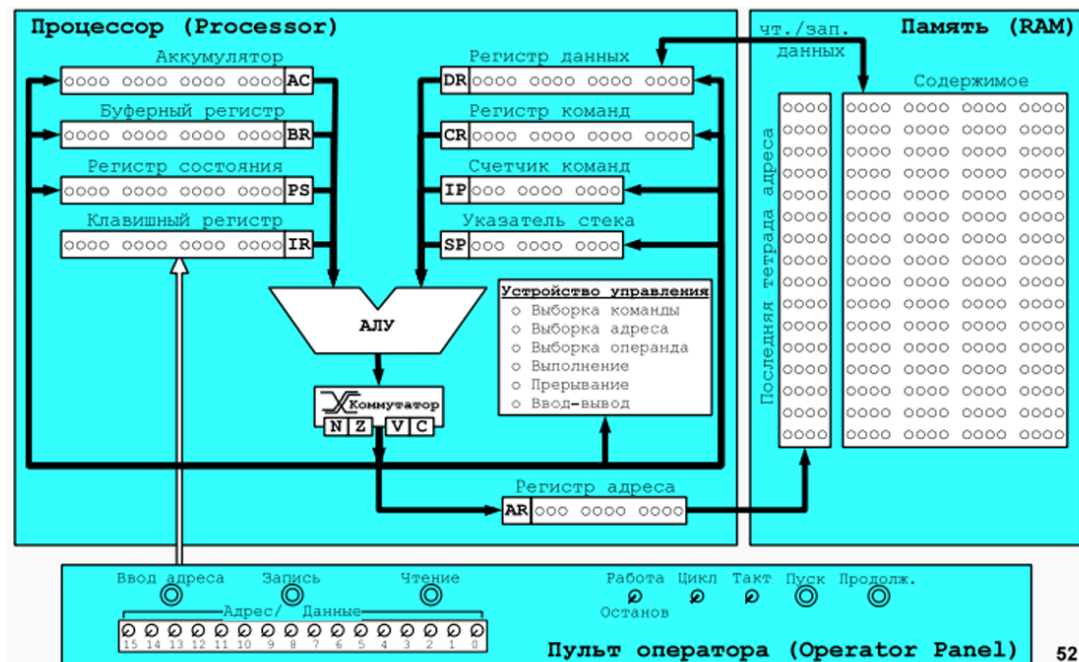
sh

 Копировать код

```
chmod ugo+rwX файл
```

Таким образом, Unix предоставляет мощные инструменты для управления файлами и каталогами, а также для настройки прав доступа к ним.

9) Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.



52

БЭВМ включает в себя нескольких функциональных блоков и регистров:

БЭВМ (базовая электронная вычислительная машина) состоит из нескольких функциональных блоков и регистров, обеспечивающих её работу.

Основные компоненты БЭВМ:

1. Память

- Состоит из 2048 ячеек, каждая из которых имеет 16 разрядов.
- Два регистра для работы с памятью:
 - **Регистр адреса (AR):** 11-разрядный регистр, в который помещается адрес памяти.
 - **Регистр данных (DR):** 16-разрядный регистр для чтения или записи данных в/из ячеек памяти.

2. Счётчик команд (IP)

- 11-разрядный регистр, хранящий адрес следующей команды, которая будет выполнена.

3. Арифметико-логическое устройство (ALU)

- Выполняет арифметические и логические операции: сложение, логическое умножение, инверсия и добавление единицы.

- Обработывает биты переполнения и переноса при сложении.
4. **Буферный регистр (BR)**
 - 16-разрядный регистр для промежуточного хранения данных.
 5. **Регистр команд (CR)**
 - Хранит код команды и декодирует операции во время работы.
 6. **Аккумулятор (AC)**
 - Основной регистр для выполнения всех вычислений с данными.
 7. **Указатель стека (SP)**
 - 11-разрядный регистр, указывающий на вершину стека, используемого для хранения адресов возвратов и параметров подпрограмм.
 8. **Клавишный регистр (IR)**
 - 16-разрядный регистр, предназначенный для ввода данных с пульта оператора, включая адреса программ, коды программ и данные.
 9. **Регистр состояния (PS)**
 - 16-разрядный регистр, хранящий биты, управляющие работой БЭВМ, и признаки результата выполнения программ.

Устройство управления (MCU)

Устройство управления в БЭВМ реализовано как микропрограммное устройство управления (MCU). Это своего рода простой компьютер, который выполняет программы, состоящие из микроопераций.

Микрооперации управляют изменениями значений вентилях, что задаёт атомарные операции: вычисления в ALU, пересылка данных между регистрами и проверки.

Исполнение команд в MCU

Исполнение машинной команды в MCU делится на пять циклов:

1. Цикл выборки команды

- Загрузка команды в регистр команды и её частичное декодирование.

2. Цикл выборки адреса

- Обработка адресных команд и выборка адреса операнда с учётом режимов адресации.

3. Цикл выборки операнда

- Размещение второго операнда команды в буфере. Первый операнд — аккумулятор.

4. Цикл исполнения

- Непосредственное выполнение команды.

5. Цикл прерывания

- Выполняется, если разрешены прерывания и устройство ввода-вывода готово к обмену данными.

Пультовые операции

Для обеспечения работы оператора предусмотрена микропрограммная реализация циклов пультовых операций:

- **Ввод адреса:** Адрес из клавишного регистра вводится в счётчик команд.
- **Чтение:** Чтение информации по адресу из памяти в буферный регистр, увеличение счётчика команд.
- **Запись:** Запись информации из буферного регистра в память, увеличение счётчика команд.
- **Пуск:** Сброс состояния БЭВМ и переход к выполнению программы.

Органы управления на панели оператора

На панели оператора расположены:

- **Переключатель «Работа/Останов»:** Останавливает программу после каждой команды.
- **Переключатель «Такт»:** Выполняет микрокод по одному такту.

- **Кнопка «Продолжение»:** Возобновляет работу остановленной БЭВМ.

... с прямой абсолютной адресацией															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				0	Адрес										

... с относительной адресацией															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	Режим		Смещение								

... с непосредственной загрузкой операнда															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	1	1	1	Число							

Безадресная команда															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0				0	0	0	Расширение КОП								

Команда ввода-вывода															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0				0	0	1	Приказ			Устройство					

Команда ветвления															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1				1	1	1	Расш. КОП			Смещение					

10) Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды.

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. В 16-разрядном слове они могут изменяться от 0 до 65535. Это числа с фиксированной запятой. Целые двоичные числа со знаком используются, когда необходимо различать положительные и отрицательные числа. Отрицательные числа представляются в дополнительном коде. Это упрощает конструкцию ЭВМ. Сложение целых двоичных чисел со знаком и без знака выполняется в базовой ЭВМ с помощью команды ADD. По команде INC к содержимому аккумулятора прибавляется единица, а по команде DEC – единица вычитается. Если при этом возникает перенос из старшего разряда A, то в регистр переноса заносится 1, в противном случае в него заносится 0. Вычитание может выполняться путем сложения уменьшаемого и дополнительного кода вычитаемого. В базовой ЭВМ нет команд для выполнения умножения и деления (АЛУ не выполняет таких операций), поэтому произведение и частное необходимо получать программным путем. Для изменения знака числа необходимо его инвертировать, а затем прибавить единицу к младшему разряду (NEG). Побитовая обработка данных обеспечивается командами логического умножения, циклических сдвигов, а также командами инвертирования и очистки регистра переноса.

Команда AND выполняет над каждым разрядом аккумулятора и содержимым ячейки булеву операцию «И». Результат выполнения команды для каждой пары битов операндов равен 1 только тогда, когда оба бита равны 1, а в остальных случаях бит результата равен 0, т.е. команда позволяет выделять или очищать определенные биты слова.

Команды ROL и ROR замыкают аккумулятор и регистр переноса в кольцо и сдвигают все биты кольца влево или вправо. Арифметическими сдвигами числа (ASL и ASR) можно реализовать операции умножения или деления на 2 (один сдвиг), 4 (два сдвига), 8 (три сдвига) и т.д.

![[Pasted image 20240606211211.png]]

11) Организация массивов данных. Режимы адресации. Цикл выборки адреса и операнда БЭВМ.

![[Pasted image 20240606211302.png]]

![[Pasted image 20240606211329.png]]

12) Управление вычислительным процессом в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP

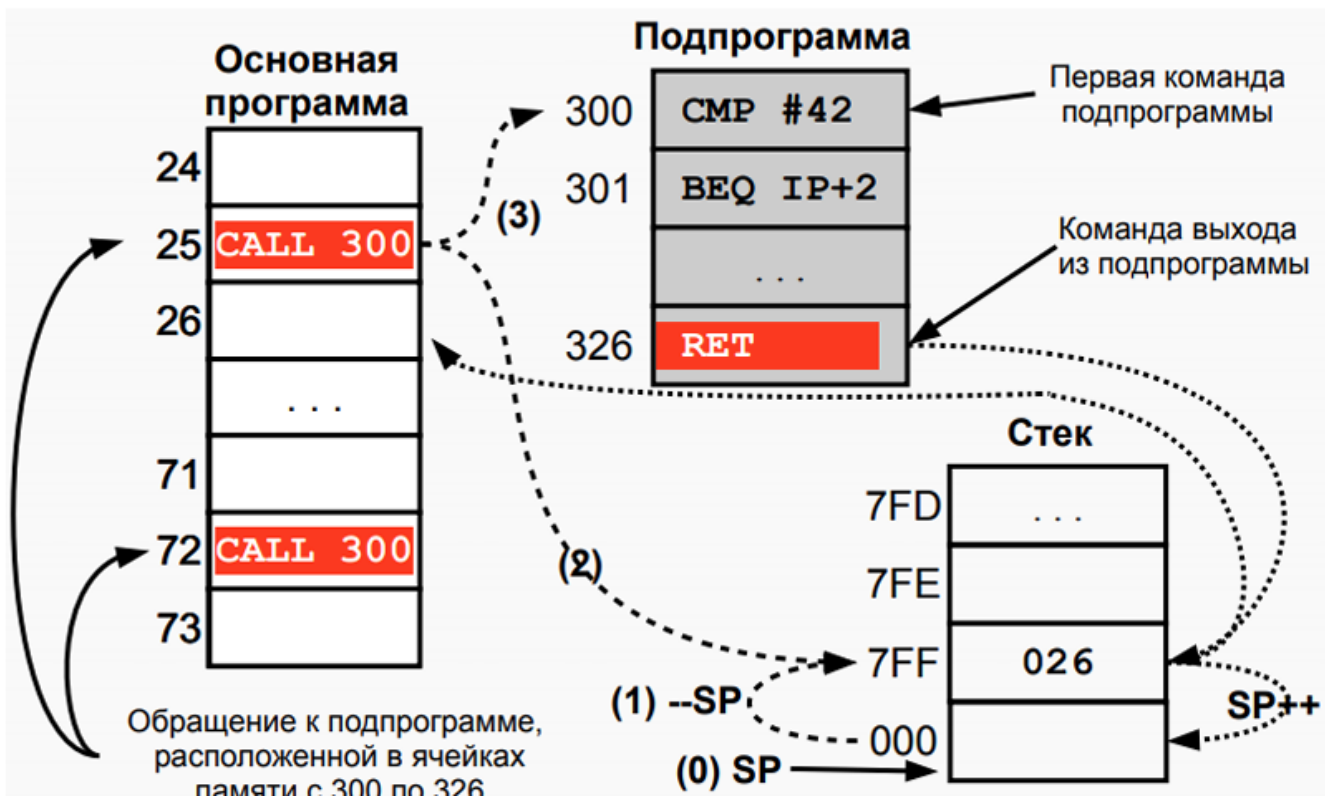
Задача управления вычислительным процессом, т.е. требуемой последовательностью выполнения команд, решается в базовой ЭВМ при помощи команд перехода, команды «Приращение и пропуск» (LOOP) и «Останов» (HLT).

```
Цикл исполнения LOOP: ~0 + DR ? DR
~0 + DR ? BR;
DR ? MEM(AR) if BR(15) = 0 then GOTO INT @ C4
IP + 1 ? IP
GOTO INT @ C4
```

![[Pasted image 20240606211459.png]]

13) Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.

Достаточно часто встречаются ситуации, когда отдельные части программы должны выполнить одни и те же действия по обработке данных. В подобных случаях повторяющиеся части программы выделяют в подпрограмму. В базовой ЭВМ для этой цели используются команды CALL и RET. <структура подпрограммы>



Варианты передачи данных в подпрограмму:

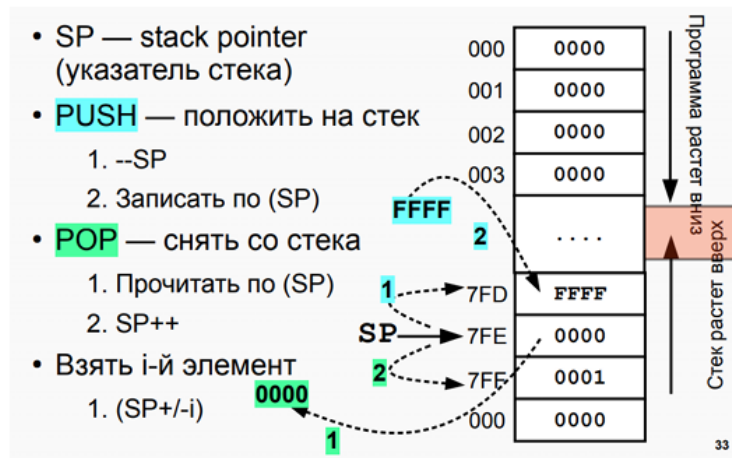
- Аккумулятор (Регистры Общего Назначения)
- Адресуемые ячейки памяти
- Стек
- Регистровые окна

Цикл исполнения CALL

- $DR \rightarrow BR$; Адрес перехода записать в BR
- $IP \rightarrow DR$; подготовить адрес возврата для записи в стек
- $BR \rightarrow IP$; Переход на подпрограмму
- $\sim 0 + SP \rightarrow SP, AR$; уменьшить стек на 1
- $DR \rightarrow MEM(AR)$; записать адрес возврата
- GOTO INT; Завершение цикла

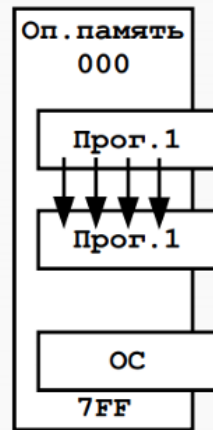
Цикл исполнения RET

- $SP \rightarrow AR$; Вершину стека поместить в AR
- $MEM(AR) \rightarrow DR$; прочесть адрес возврата
- $DR \rightarrow IP$; вернуться из подпрограммы
- $SP + 1 \rightarrow SP$; увеличить стек на 1
- GOTO INT; Завершение цикла



PIC - Position Independent Code

- Код, который работает относительно того адреса на который загружен



- Необходим для, например, модулей ядра (даже при наличии виртуальной памяти!)
- Внутри программы только относительные адреса (смещения)
- Внешние ссылки только абсолютные
- В БЭВМ

Длина перемещаемой программы в БЭВМ? ации!

Загрузчик и динамический линковщик программ:

Любая ОС имеет соответствующую программу или часть ядра

- Загрузка по выбранному ОС адресу (даже в виртуальной памяти)
- Изменение константных частей адресов в программе
- Загрузка базовых значений регистров
- Динамическая загрузка разделяемых библиотек
- Связывание адресов основной программы с вызываемыми библиотеками

Библиотеки:

- Набор стандартных библиотечных функций
- Разделяемые (динамически линкуемые) и архивные (статически линкуемые)
- # find /lib /usr/lib -name "*.so" | wc -l 3510
- Статические связывают вызовы функций с телом функции в процессе компиляции
- Динамические — в момент загрузки
- Если вам нужна функция — см. в библиотеки

потактовое выполнение

call

=====

dr -> br команду в буфер

ip -> dr адрес возврата в др

br -> ip адрес пп в ип

sp -1 -> sp, ar указатель стека прибавился, указатель в адрес

dr -> mem(ar) адрес возврата на стек

ret

=====

sp -> ar вершину стека в адрес

mem(ar) -> dr адрес возврата в др

dr -> ip адрес возврата в ип

sp + 1 -> sp вершина - 1

push

```

=====
ac -> dr    данные которые кладем на стек в dr
sp-1 -> sp   вершина + 1
sp -> ar    вершина в ar
dr -> mem(ar) данные на вершину

pop

=====
sp -> ar    адрес из вершины в ar
mem(ar) -> dr взяли с вершины
dr -> ac    закинули в ac
sp + 1 -> sp   вершина - 1

```

Ну это пиво. Давай разберем материал на фото по порядку.

PIC - Position Independent Code

Это код, который может выполняться независимо от того, по какому адресу он загружен в память. Такой подход позволяет коду быть более гибким и легко переносимым. Вот основные моменты:

- **Необходим для модулей ядра:** Даже при наличии виртуальной памяти модули ядра могут использовать позиционно-независимый код.
- **Относительные адреса:** Внутри программы используются только относительные адреса (смещения), что делает её независимой от конкретного места в памяти, куда она загружена.
- **Абсолютные внешние ссылки:** Ссылки на внешние элементы остаются абсолютными.
- **В БЭВМ (большая электронная вычислительная машина):** В таких системах важно учитывать длину перемещаемой программы, так как это влияет на эффективность и возможность её загрузки в различные области памяти.

Загрузчик и динамический линковщик программ

Любая операционная система имеет специальную программу или часть ядра, которая отвечает за загрузку и связывание программ. Это делается для того, чтобы программы могли выполняться правильно независимо от их местоположения в памяти. Основные функции:

- **Загрузка по адресу:** Программа загружается по адресу, который выбирает ОС, даже в случае виртуальной памяти.
- **Изменение адресов:** При загрузке изменяются константные части адресов в программе.
- **Инициализация регистров:** Загружаются базовые значения регистров.
- **Динамическая загрузка библиотек:** Программы могут загружать разделяемые библиотеки по мере необходимости.
- **Связывание адресов:** Адреса основной программы связываются с вызываемыми библиотеками.

Библиотеки

Библиотеки содержат набор стандартных функций, которые можно использовать в разных программах. Они бывают двух типов:

- **Разделяемые (динамические) библиотеки:** Эти библиотеки загружаются в момент выполнения программы. Они позволяют уменьшить размер программ и использовать один экземпляр библиотеки для нескольких программ.
- **Статические библиотеки:** Эти библиотеки связываются с программой на этапе компиляции. Вызовы функций из таких библиотек включаются прямо в исполняемый файл программы, что делает её автономной, но увеличивает размер.

Пример команды для поиска библиотек

- Команда `find /lib /usr/lib -name "*.so" | wc -l` используется для поиска всех динамических библиотек (файлы с расширением .so) и подсчёта их количества.

Дополнительные моменты:

1. Что имеется в виду под "частью ядра", которое отвечает за загрузку и связывание программ?

Это специальная часть операционной системы, которая занимается подготовкой программ к выполнению. Она загружает программы в память и связывает их с необходимыми библиотеками и ресурсами, чтобы они могли правильно работать.

2. Как происходит эта загрузка программ?

Загрузка программ происходит следующим образом:

- Программа загружается в память по адресу, который выбирает операционная система.
- Адреса в программе, которые были указаны как фиксированные, изменяются на те, что соответствуют текущему положению программы в памяти.
- Базовые значения регистров процессора устанавливаются для корректного начала работы программы.
- Если программа использует динамические библиотеки, они загружаются в память по мере необходимости.

3. Что значит "изменяются константные части адресов в программе"?

Когда программа загружается в память, фиксированные адреса (константы) в ней могут не совпадать с фактическим местоположением. Эти адреса изменяются на правильные значения, чтобы программа могла корректно обращаться к своим данным и кодам.

4. Про какие регистры идет речь в пункте про инициализацию регистров?

Речь идет о регистрах процессора, таких как регистр команд (указывающий на следующую команду для выполнения), регистр данных (содержащий данные для обработки) и другие. Инициализация

регистров означает установку начальных значений, необходимых для начала выполнения программы.

5. Что значит "Абсолютные внешние ссылки: Ссылки на внешние элементы остаются абсолютными"?

В контексте PIC (позиционно-независимого кода), ссылки на внешние элементы (например, функции или данные, находящиеся в других модулях или библиотеках) остаются фиксированными (абсолютными) и не зависят от того, где загружен сам код. То есть, такие ссылки указывают на конкретное место в памяти, независимо от местоположения кода программы.

14) Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы.

К ЭВМ можно подключать большое число разнообразных устройств ввода-вывода или внешних устройств (ВУ). Эти устройства передают в ЭВМ и получают из нее большой объем информации, который не может быть размещен только в регистрах процессора. Поэтому информация передается из ВУ в память ЭВМ и поступает на ВУ из ее памяти. При этом обмен может идти под управлением программы ЭВМ через регистры процессора (программно-управляемая передача данных) или под управлением специального внешнего устройства (контроллера прямого доступа в память), минуя процессор (передача данных при прямом доступе к памяти).

Программно-управляемый обмен осуществляется малыми порциями, при прямом доступе к памяти информация передается большими блоками. При использовании программно управляемого обмена должна быть составлена программа, обеспечивающая пересылку данных из памяти ЭВМ в аккумулятор и далее в регистр памяти контроллера ВУ (вывод данных) или из регистра данных контроллера ВУ в аккумулятор и затем в память ЭВМ

(ввод данных). В этой программе можно реализовать один из трех типов обмена: синхронный, асинхронный и по прерыванию.

Передача данных: Синхронная/Асинхронная

Завершение обмена и получение драйвером (программой) результата:
Синхронное/асинхронное

Драйверы:

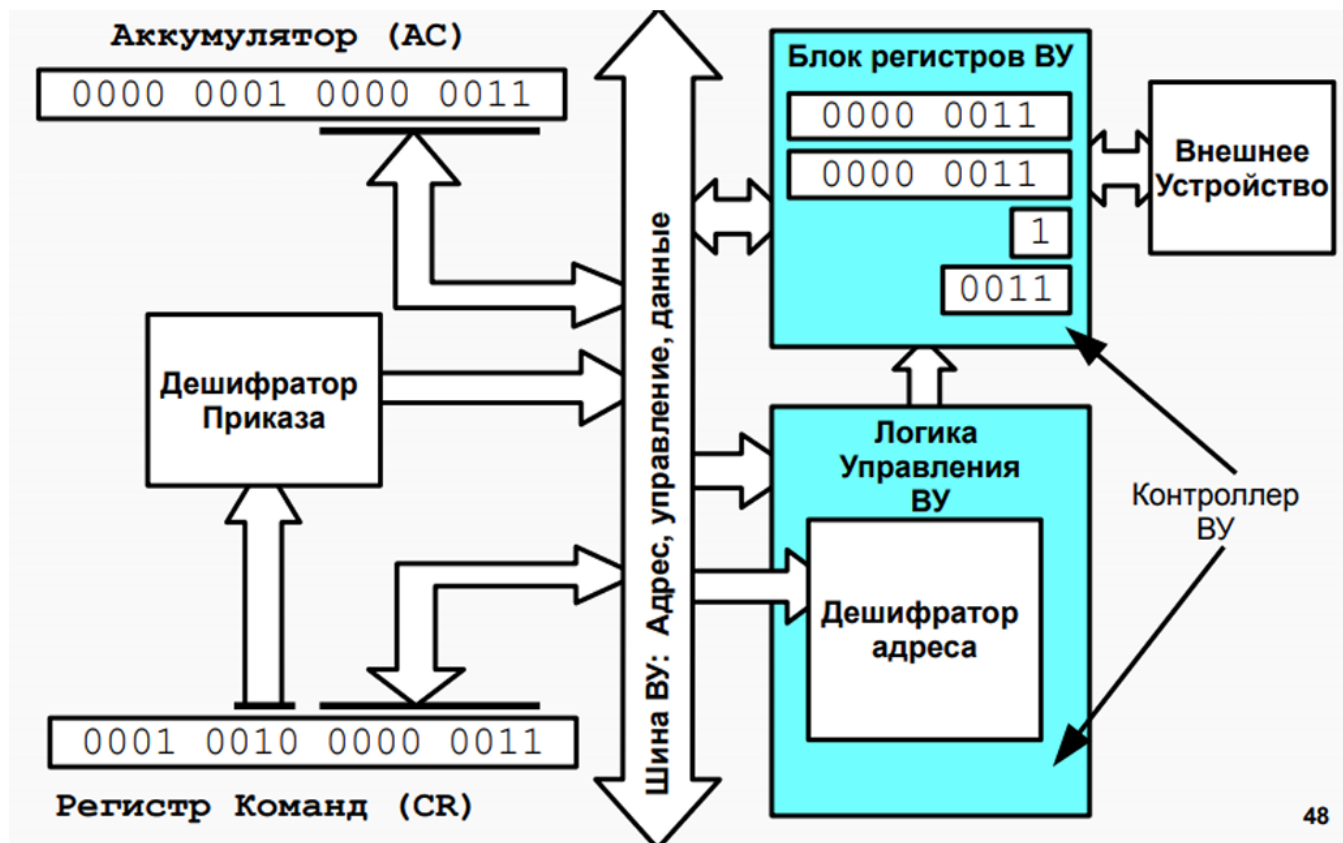
- Организуют совместную работу с устройством
- «Знают» о принципах работы устройства, адресах регистров, поддерживаемых режимах работы
- Управляются единообразным программным интерфейсом

15) Организация ввода-вывода в БЭВМ. Устройства ввода-вывода, команды.

Команда ввода-вывода

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	Приказ				Регистр устройства							

Наименование	Мнемон.	Код	Описание
Запрет прерываний	DI	1000	
Разрешение прерываний	EI	1100	
Ввод	IN REG	12XX	REG → AC
Вывод	OUT REG	13XX	AC → REG
Прерывание	INT NUM	18XX	Програмное прерывание с вектором NUM
Возврат из прерывания	IRET	0B00	(SP)+ → PS, (SP)+ → IP



48

Между ВУ и процессором установлены простейшие контроллеры, каждый из которых содержит: регистр данных (для обмена данными между ВУ и процессором), дешифратор адреса (позволяющий выделить обращение к данному ВУ среди всех обращений у устройствам ввода вывода, подключенным к процессору), логику управления (декодирующий приказ от процессора на выполнение тех или иных операций) и регистр состояния (в котором хранится информация о готовности ВУ к обмену данными с

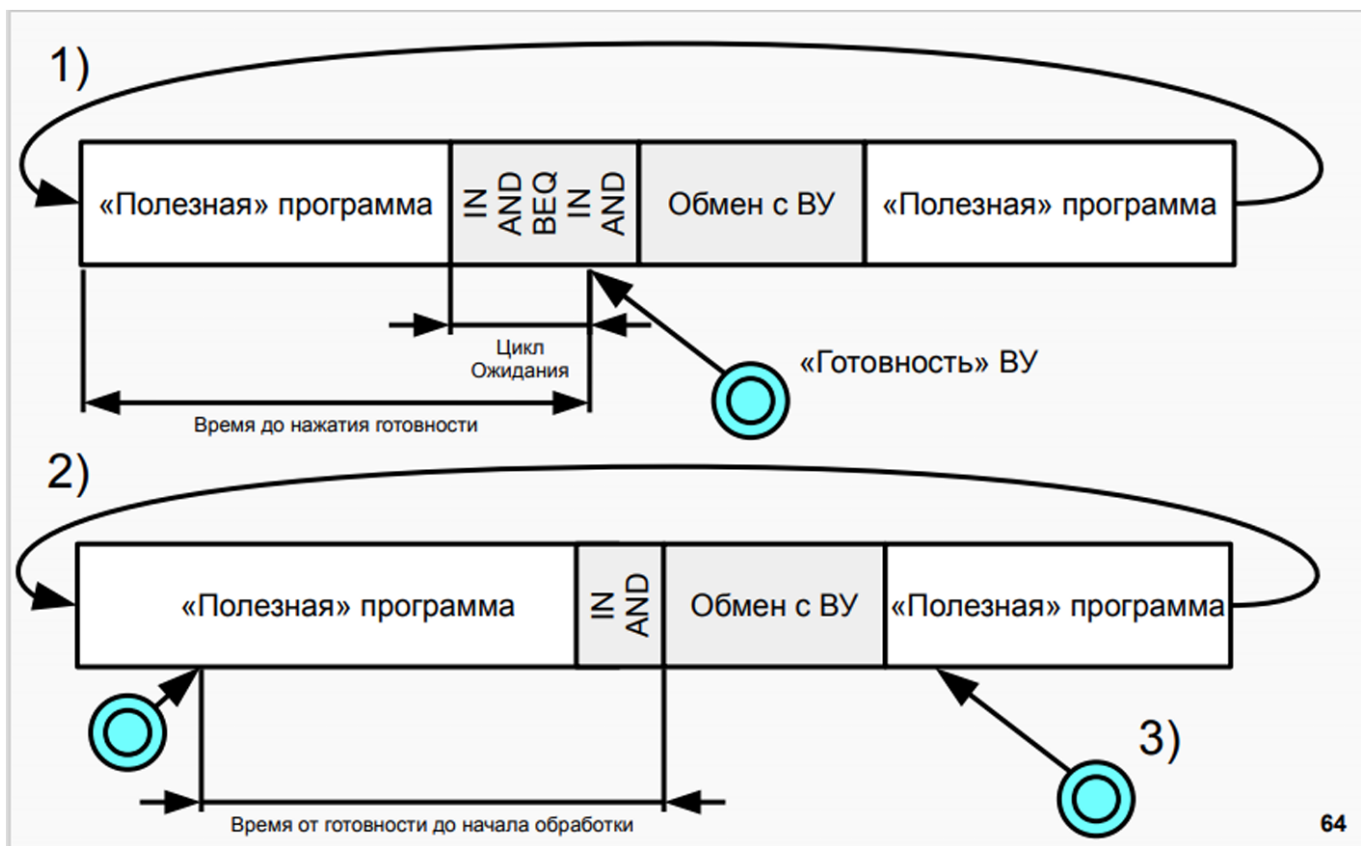
процессором). В контроллерах простейших ВУ обычно используется однокбитовые регистры состояния, которые часто называют флагом. Контроллеры ВУ связаны с процессором шинами ввода и вывода информации, шиной адреса и шиной управления, по которым передаются приказы от процессора и сведения о состоянии ВУ.

Устройства ВУ: вывода, ввода, ввода-вывода, ввода-вывода PRO, таймер, принтер, бегущая строка, 7 разр. индикатор, клавиатура, numrad

16) Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена.

Алгоритм программы асинхронного обмена: сначала проверяется готовность ВУ к обмену и, если оно готово, дается команда на обмен (ввод или вывод). ВУ сообщает о готовности установкой в единицу флага в контроллере ВУ. При асинхронном обмене ЭВМ должна тратить время на ожидание момента готовности, а так как готовность проверяется командным путем, то в это время ЭВМ не может выполнять никакой другой работы по преобразованию данных.

	ORG	0x10	
START:	CLA		
S1:	IN	5	; Ожидание ввода первого символа
	AND	#0x40	; Бит 6 SR == 0 («Готов» нажата?)
	BEQ	S1	; Нет - "Спин-луп"
	IN	4	; Ввод первого символа
	SWAB		; Сдвиг первого символа
	ST	RES	; Сохранение его в ячейке RES



17) Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания.

Прерывание — сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленного внимания. В контексте БЭВМ это сигнал о готовности обмена данными с некоторым ВУ.

Также стоит отметить преимущества над асинхронным режимом передачи данных (если Вы помните, то реализовывать его можно двумя способами: через spin-loop или одну проверку с последующим окном ввода-вывода в циклической полезной программе):

- процессор не простаивает в ожидании готовности ВУ
- организованная работа сразу со всеми нужными нам ВУ
- если все ВУ не готовы к обмену, то процессор занят полезной работой

Ярким отличием прерываний является обязательное сохранение состояния процессора в момент прерывания, чтобы когда прерывание было обработано, мы смогли вернуться в основную программу и смогли продолжить ее выполнение без разнообразных коллизий.

Вектор прерывания:

Адр .	Сод .
0x0	Адр 0
0x1	PS 0
0x2	Адр 1
0x3	PS 1
0x4	Адр 2
0x5	PS 2
0x6	Адр 3
0x7	PS 3
0x8	Адр 4
0x9	PS 4
0xA	Адр 5
0xB	PS 5
0xC	Адр 6
0xD	PS 6
0xE	Адр 7
0xF	PS 7

- Совокупность адреса программы обработки прерывания и регистра состояния (PS)
- Необходимо инициализировать перед началом обработки прерывания – Хотя бы установить на подпрограмму, которая ничего не делает
- Ответственность OS и БИОС
- В БЭВМ-NG ячейки с 0x000 по 0x10 – Всего 8 векторов, по два слова на вектор
- На одном векторе может быть несколько прерываний

Ответственность OS и BIOS

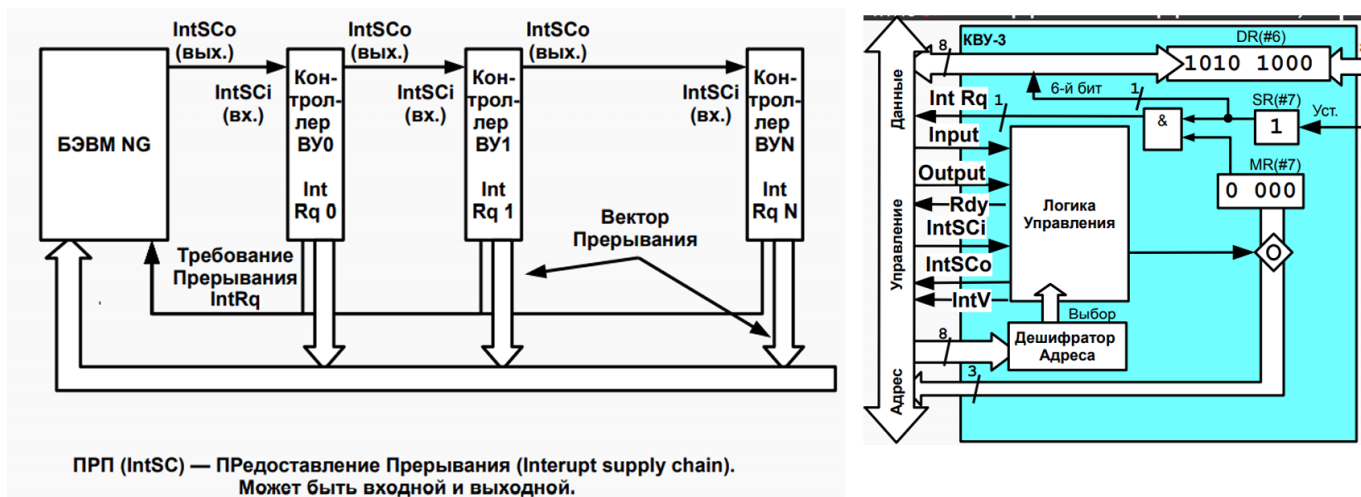
OS (операционная система) и BIOS (базовая система ввода-вывода) отвечают за настройку и управление векторами прерываний. Это значит, что они должны инициализировать векторы, чтобы система знала, что делать при возникновении прерывания. ОС и BIOS назначают адреса программ обработки прерываний и записывают их в соответствующие ячейки памяти.

BIOS

BIOS (Basic Input/Output System) — это программный код, который выполняется процессором при включении компьютера. Он отвечает за начальную инициализацию оборудования, проверку его работоспособности и загрузку операционной системы. BIOS находится в постоянной памяти (ПЗУ) на материнской плате компьютера.

Регистр управления или Management Register — регистр, хранящий в себе информации о разрешении/запрете прерывания от данного ВУ и номер вектора прерывания, к которому привязано ВУ.

В 3х младших битах содержится номер вектора прерывания, в 3 бите будет 1, если прерывания от данного ВУ разрешены и 0 — если запрещены.



Сигнал последовательно проходит через контроллеры 0, 1, 2 и останавливается в логике управления КВУ-3

Внутри ЛУ происходит магия и открывается вентиль у MR (переключаемся на 2 картинку)

MR подключается к CR через шину адреса и младшие три бита MR записываются в младшие 3 бита CR, при этом оставшиеся 5 бит младшего байта CR заполняются нулями.

Параллельно с этим ЛУ маскирует сигнал IntSCo и сигнал предоставления прерывания не передается в следующие контроллеры

Также на шину управления поступает сигнал из ЛУ IntV (Interrupt Vector), который информирует о предоставлении данным контроллером номера вектора прерывания.

18) Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания.

В БЭВМ доступно 8 векторов прерываний, и расположены они в ячейках памяти 0x0 — 0xF включительно. На один вектор прерывания может приходиться несколько прерываний. До начала необходимо загрузить

вектора прерываний в ВУ и разрешить прерывания.

Цикл прерывания:

- 1) IF PS(W) == 0: GOTO STOP; Если тумблер РАБОТА\ОСТАНОВ установлен на ОСТАНОВ, то работа БЭВМ прекращается
- 2) IF PS(IRQ) == 0: GOTO INFETCH; Если запроса на прерывания нет (0 в 6 бите PS), то переходим к циклу выборки СЛЕДУЮЩЕЙ команды
- 3) IRQSC; Формируется сигнал предоставления прерывания
- 4) SP + ~0 -> SP, AR;
- 5) IP -> DR; ///IP -> -(SP)
- 6) DR -> MEM(AR);
- 7) SP + ~0 -> SP, AR;
- 8) PS -> DR; ///PS -> -(SP)
- 9) DR -> MEM(AR); LTOL(CR) -> BR; Выделили номер вектора прерывания
- 10) SHL(BR) -> BR, AR; Выделили адрес адреса обработчика прерывания (тавтология, привет)
- 11) MEM(AR) -> DR; в DR адрес обработчика прерывания
- 12) DR -> IP
- 13) LTOL(BR + 1) -> AR ; в AR адрес, в котором лежит PS для обработчика прерывания
- 14) MEM(AR) -> DR
- 15) DR -> PS

Пример проги с прерываниями:

Готовность ВУ1: 2*А→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F	
	ORG 0x0 ; Инициализация векторов прерывания
V0:	WORD \$DEFAULT,0x180 ; Вектор прерывания #0
V1:	WORD \$INT1,0x180 ; Вектор прерывания #1
V2:	WORD \$DEFAULT,0x180 ; Вектор прерывания #2
V3:	WORD \$INT3,0x180 ; Вектор прерывания #3
	...
V7:	WORD \$DEFAULT,0x180 ; Вектор прерывания #7
DEFAULT:	IRET ; Просто возврат
START:	ORG 0x020 ; Загрузка начальных векторов прерывания
	DI
	CLA
	OUT 1 ; MR КВУ-0 на вектор 0
	OUT 5 ; MR КВУ-2 на вектор 0
	LD #9 ; разрешить прерывания и вектор №1
	OUT 3 ; (1000 0001=1001) в MR КВУ-1
	LD #0xB ; разрешить прерывания и вектор №3
	OUT 7 ; (1000 0011=1011) в MR КВУ-3
	...
	BR \$PROG

Готовность ВУ1: 2*А→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F		
PROG:	EI	; Установка состояния разр. прерывания
	CLA	; Первоначальная очистка аккумулятора
INCLP:	INC	; Цикл для наращивания
	BR INCLP	; содержимого аккумулятора
	ORG 0x03F	
		; Ячейка для хранения кодов, поступающих с ВУ-3
IO3:	WORD ?	
	ORG 0x040	
INT1:		; Прерывание сохранило содержимое PS
	NOP	; отладочная точка останова (NOP/HLT)
	PUSH	; Сохранили AC
	ASL	; Умножили AC на 2
	OUT 2	; Записали в РДВУ-1 (DR#2)
	POP	; Вернули AC назад
	NOP	; отладочная точка останова (NOP/HLT)
	IRET	; Возврат из обработки прерывания

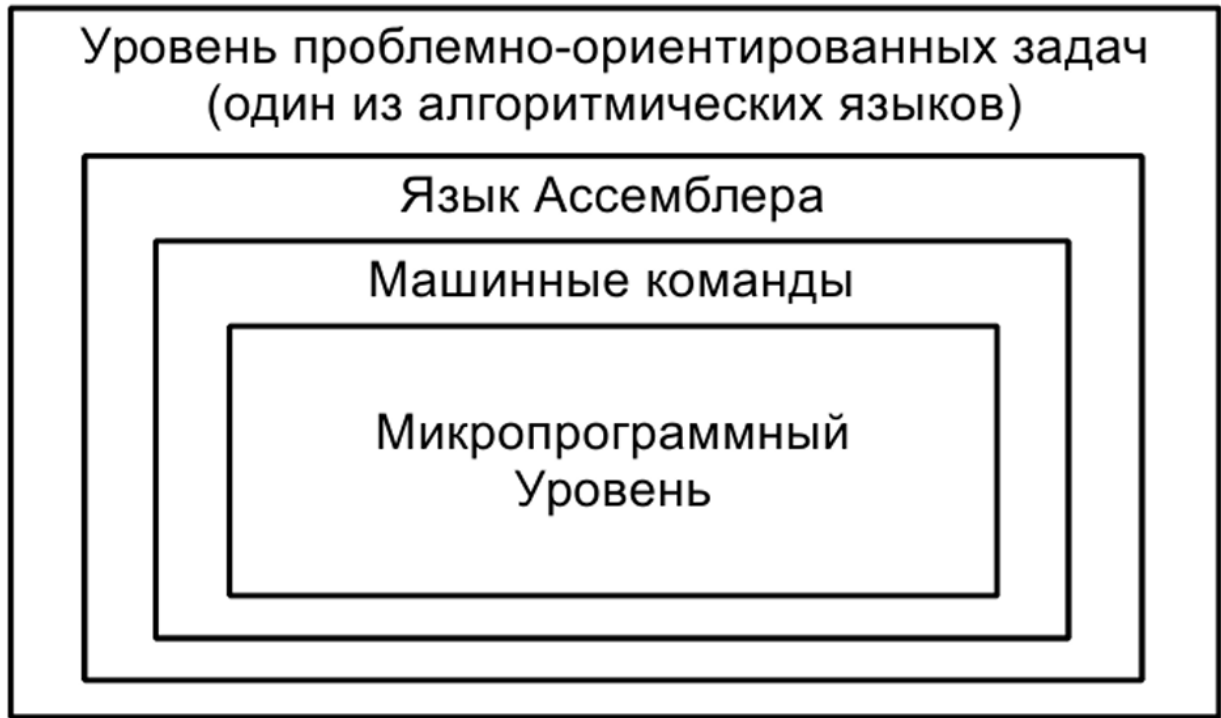
Готовность ВУ1: 2*А→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F		
	ORG 0x040	
INT3:		; Прерывание сохранило содержимое PS
	NOP	; отладочная точка останова (NOP/HLT)
	PUSH	; AC кладем в стек
	CLA	
	IN 6	; РДВУ-3
	ST \$IO3	; сохранение
	NOP	; отладочная точка останова (NOP/HLT)
	POP	; AC вынимаем из стека
	IRET	; Возврат из обработки прерывания

19) Понятие многоуровневой ЭВМ. Понятие и пример программы на разных уровнях.

- **Микропрограммный уровень** - самый низкий уровень, на котором работают непосредственно с микропрограммами. Микропрограммы управляют аппаратными ресурсами компьютера.

- **Машинные команды** - следующий уровень, где используются машинные инструкции, понятные процессору. Эти команды представляют собой двоичные коды, которые процессор выполняет непосредственно.
- **Язык ассемблера** - еще один уровень выше. Это символический язык программирования, который используется для написания программ, которые затем преобразуются в машинные команды с помощью ассемблеров. Язык ассемблера удобнее для человека, так как позволяет использовать символические имена и метки вместо двоичных кодов.
- **Уровень проблемно-ориентированных задач** - это уровень, где используются алгоритмические языки программирования, такие как Fortran, Pascal, C и другие. Программы на этих языках более понятны для людей и ближе к человеческому языку. Такие программы нужно переводить на язык машинных команд с помощью компиляторов или интерпретаторов.
- **Уровень программных систем** - самый верхний уровень, который включает в себя специализированные языки для написания сложных программных систем и приложений.

Уровень программных систем (специальный язык)



Пример программы на разных уровнях

Представьте, что нужно написать программу для сложения двух чисел:

- **Микропрограммный уровень:** Здесь нужно было бы управлять аппаратными ресурсами на самом низком уровне, что очень сложно и трудоемко.
- **Машинные команды:** Программа состояла бы из последовательности двоичных инструкций для процессора, например, загрузить первое число, загрузить второе число, сложить их, сохранить результат.
- **Язык ассемблера:** Программа на ассемблере для сложения двух чисел выглядела бы примерно так:

```
MOV AX, [число1]
ADD AX, [число2]
MOV [результат], AX
```

Алгоритмический язык: На более высоком уровне, например, на языке C, программа была бы написана так:

```
int число1 = 5;  
int число2 = 3;  
int результат = число1 + число2;
```

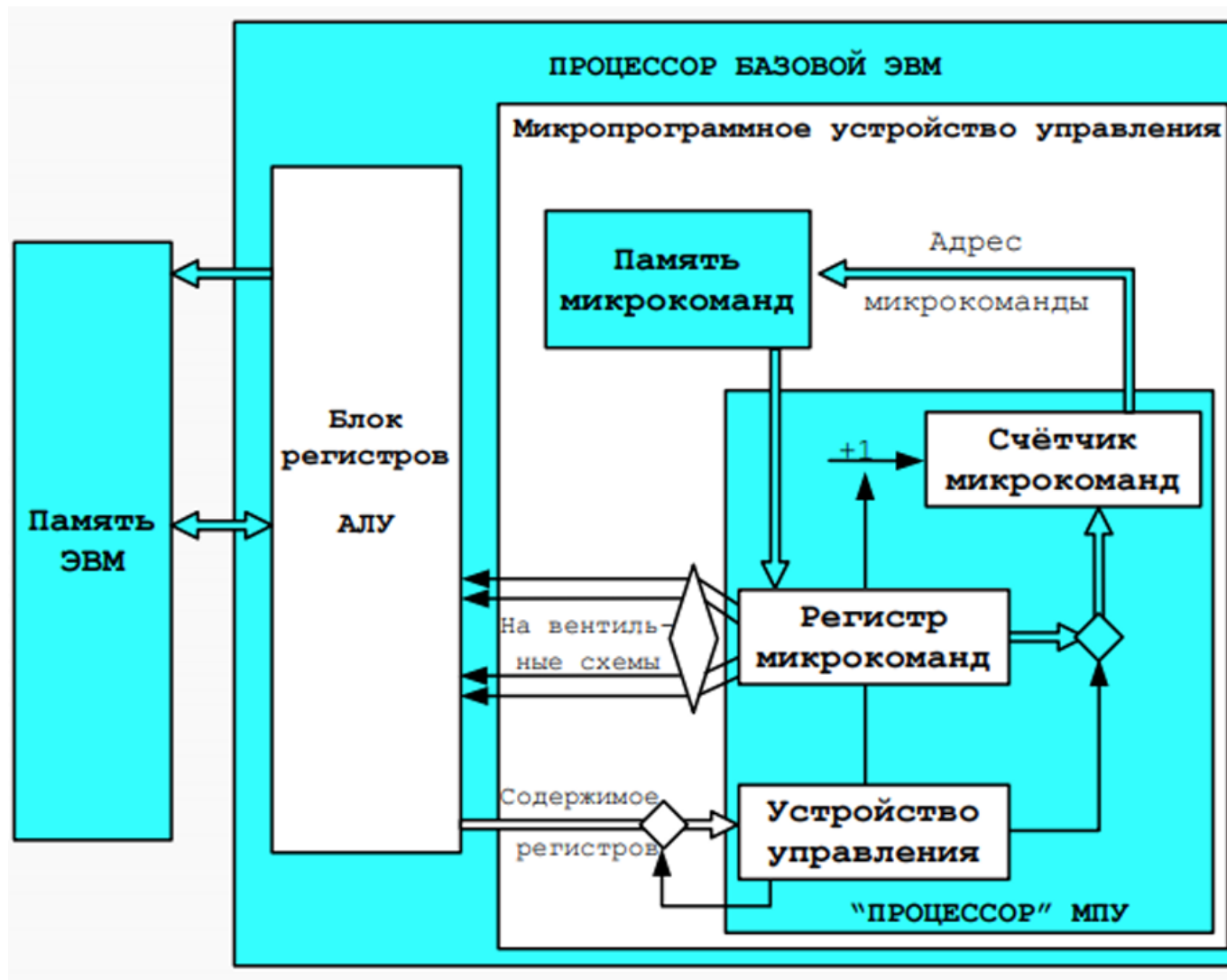
Компиляторы и интерпретаторы

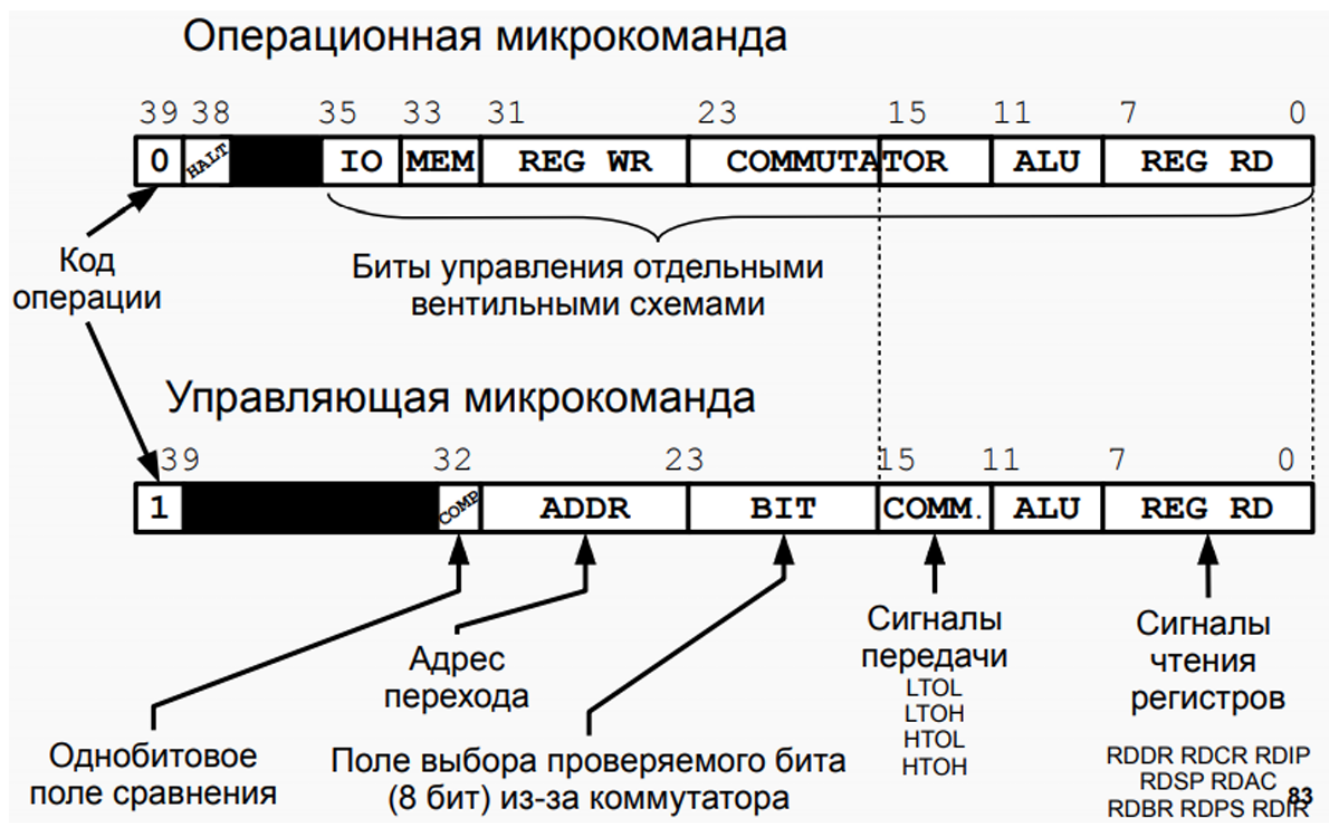
- **Компиляторы:** Они переводят программу на алгоритмическом языке в машинный код целиком до выполнения. Это ускоряет выполнение, так как весь код уже переведен.
- **Интерпретаторы:** Они переводят и выполняют программу построчно. Это проще для разработки и отладки, так как изменения можно сразу проверить, но медленнее по сравнению с компиляцией.

Многоуровневая ЭВМ

Многоуровневая ЭВМ позволяет работать с программами на разных уровнях абстракции, каждый из которых имеет свой язык программирования. Каждый уровень виртуализирует машину, предоставляя пользователям видимость работы на более высоком уровне, что упрощает программирование и взаимодействие с вычислительными машинами.

20) Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микрокоманд.





21) Структура и принципы работы арифметико-логического устройства и коммутатора. Регистр состояния БЭВМ

Арифметико-логическое устройство (АЛУ) — это основная часть центрального процессора, выполняющая арифметические и логические операции.

1. Структура АЛУ:

- **Сумматор:** выполняет операции сложения и вычитания.
- **Логический блок:** осуществляет логические операции (AND, OR, XOR, NOT и др.).
- **Сдвиговый блок:** выполняет операции сдвига (влево, вправо, циклический сдвиг).
- **Мультиплексоры:** используются для выбора операций и входных данных.

2. Принципы работы АЛУ:

- **Входные данные** поступают в АЛУ из регистров или памяти.
- **Команда управления** определяет, какую операцию нужно выполнить.
- **Результат** операции записывается в выходной регистр или обратно в память.
- **Флаги состояния** (перенос, нуль, отрицательное число и др.) обновляются на основе результата операции.

Структура и принципы работы коммутатора

Коммутатор — это устройство, которое управляет потоками данных между различными компонентами вычислительной системы.

1. Структура коммутатора:

- **Порты ввода-вывода:** обеспечивают соединение с другими устройствами и компонентами.
- **Кроссбарная матрица:** обеспечивает прямое соединение между любыми двумя портами.
- **Контроллер управления:** обрабатывает команды по перенаправлению данных.

2. Принципы работы коммутатора:

- **Получение данных** через один из входных портов.
- **Определение направления:** контроллер решает, куда перенаправить данные на основе управляющих сигналов.
- **Перенаправление данных** через кроссбарную матрицу к нужному выходному порту.

Регистр состояния БЭВМ

Регистр состояния (или флаговый регистр) — это регистр, который хранит различные флаги, отражающие текущее состояние процессора после выполнения операции.

1. Основные флаги:

- **Флаг переноса (Carry):** устанавливается при возникновении переноса из старшего разряда.
- **Флаг нуля (Zero):** устанавливается, если результат операции равен нулю.
- **Флаг знака (N):** показывает знак результата (положительный или отрицательный).
- **Флаг переполнения (Overflow):** указывает на переполнение при выполнении арифметических операций.

2. Принципы работы регистра состояния:

- **Обновление флагов:** после выполнения каждой команды АЛУ обновляет соответствующие флаги в регистре состояния.
- **Использование флагов:** эти флаги используются для принятия решений при выполнении условных переходов и других операций, зависящих от состояния процессора.

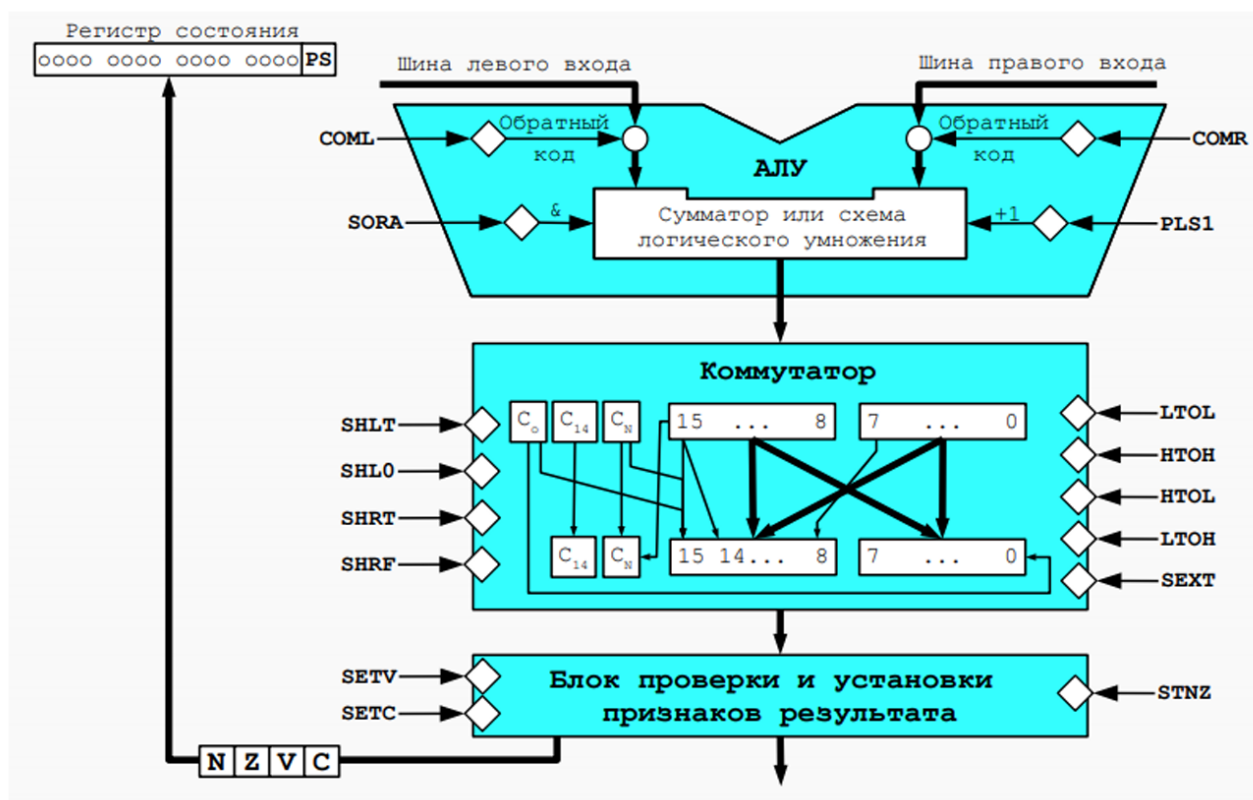
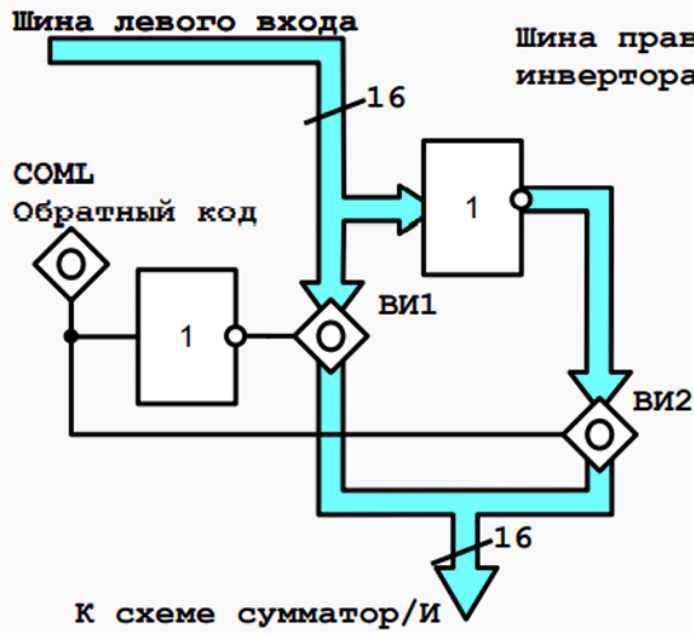
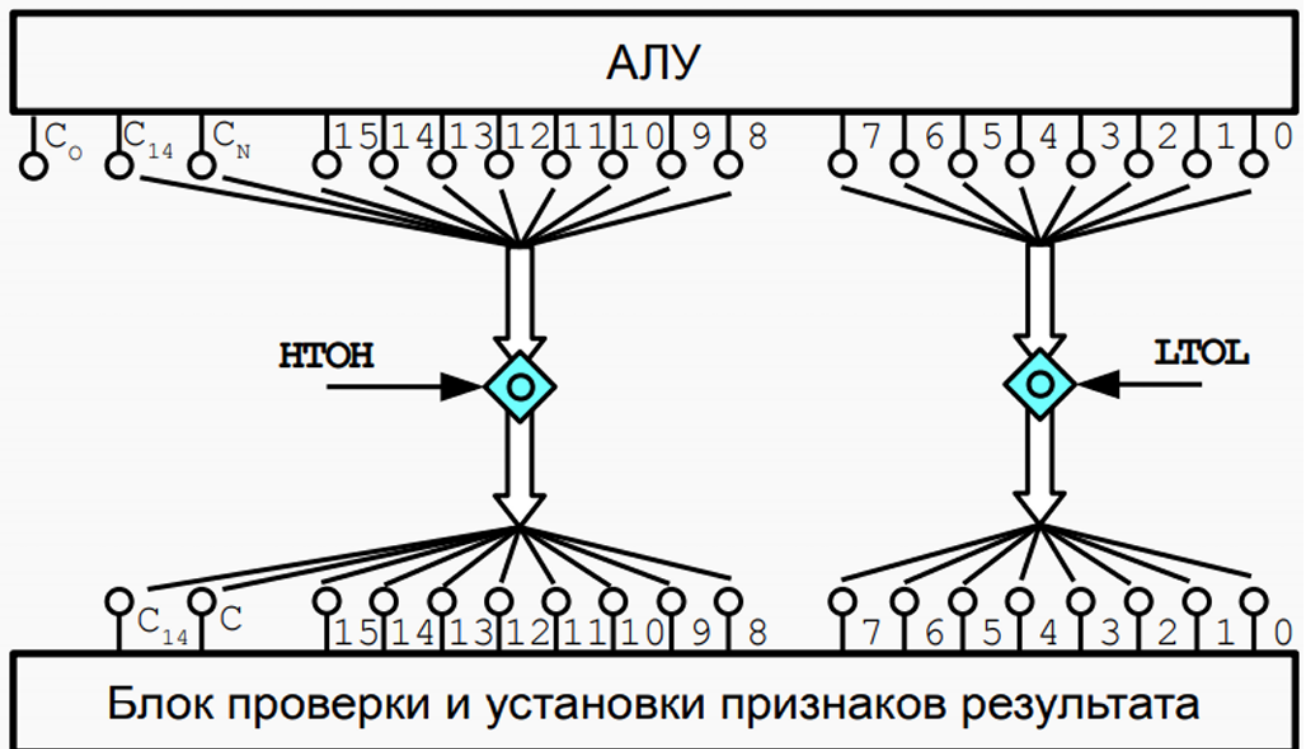
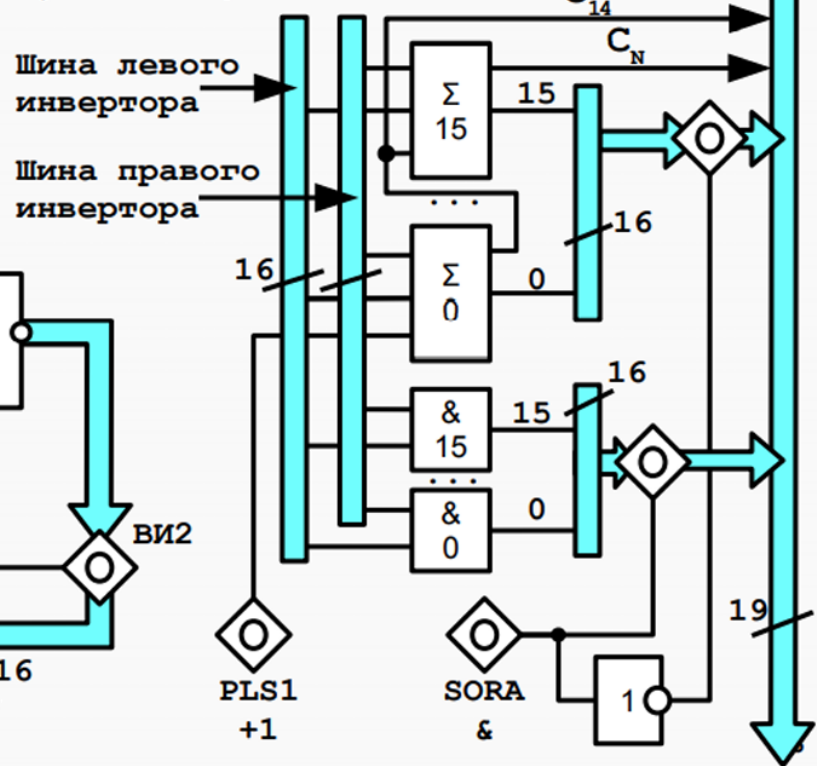
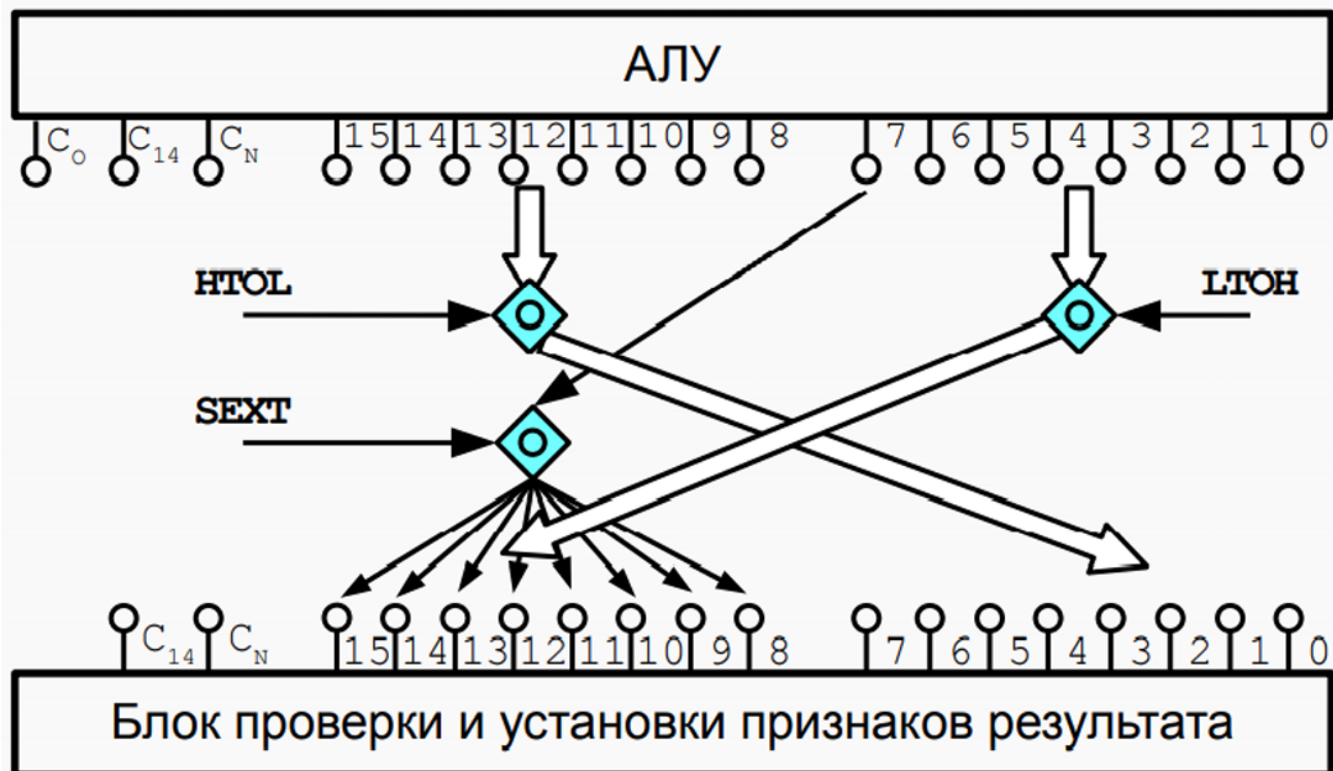


Схема обратного кода



Сумматор и лог. «И»





1. Сумматор или логическое умножение:

- **Сигналы:**

- **COML и COMR (Обратный код):** Эти сигналы позволяют инвертировать входные данные, что необходимо для выполнения операции вычитания.
- **PLS1 (+1):** Сигнал, который используется для инкрементации (увеличения) значения на единицу.
- **SORA (&):** Логическое И (AND) между двумя входами.

2. Результат работы АЛУ передается в коммутатор для дальнейшей обработки.

Коммутатор

Коммутатор управляет потоком данных между различными частями процессора.

1. Структура коммутатора:

- Коммутатор включает в себя множество мультиплексоров, которые позволяют направлять данные между различными

частями процессора.

2. Сигналы управления коммутатором:

- **SHLT, SHLO, SHRT, SHRF**: Сигналы сдвига данных влево или вправо.
- **LTOL, LTON, HTON, HTOL, SEXT**: Сигналы для перенаправления данных между разными частями регистра (например, из старшего байта в младший).

Регистр состояния

Регистр состояния хранит различные флаги, отражающие текущее состояние процессора.

1. Основные флаги:

- **N (Negative)**: Указывает на отрицательный результат.
- **Z (Zero)**: Указывает на нулевой результат.
- **V (Overflow)**: Указывает на переполнение.
- **C (Carry)**: Указывает на перенос.

Блок проверки и установки признаков результата

Этот блок обновляет флаги в регистре состояния на основе результатов выполнения операций.

Управляющие сигналы

Список управляющих сигналов и их назначение:

- **COML и COMR**: Включение режима обратного кода (инвертирование входных данных).
- **PLS1**: Прибавление единицы к результату.
- **SORA**: Логическое И.
- **SHLT, SHLO, SHRT, SHRF**: Сдвиг влево и вправо.
- **LTOL, LTON, HTON, HTOL, SEXT**: Перенос данных между частями регистра.

- **SETV, SETC:** Установка флагов переполнения и переноса.
- **STNZ:** Установка флага нуля.

Операции, выполняемые АЛУ

1. **Сложение:** Включает использование сумматора.
2. **Логическое умножение (AND):** Включает использование логического блока.
3. **Инкрементация:** Прибавление единицы к значению.
4. **Обратный код:** Инвертирование входных данных для выполнения вычитания.

Схема обратного кода

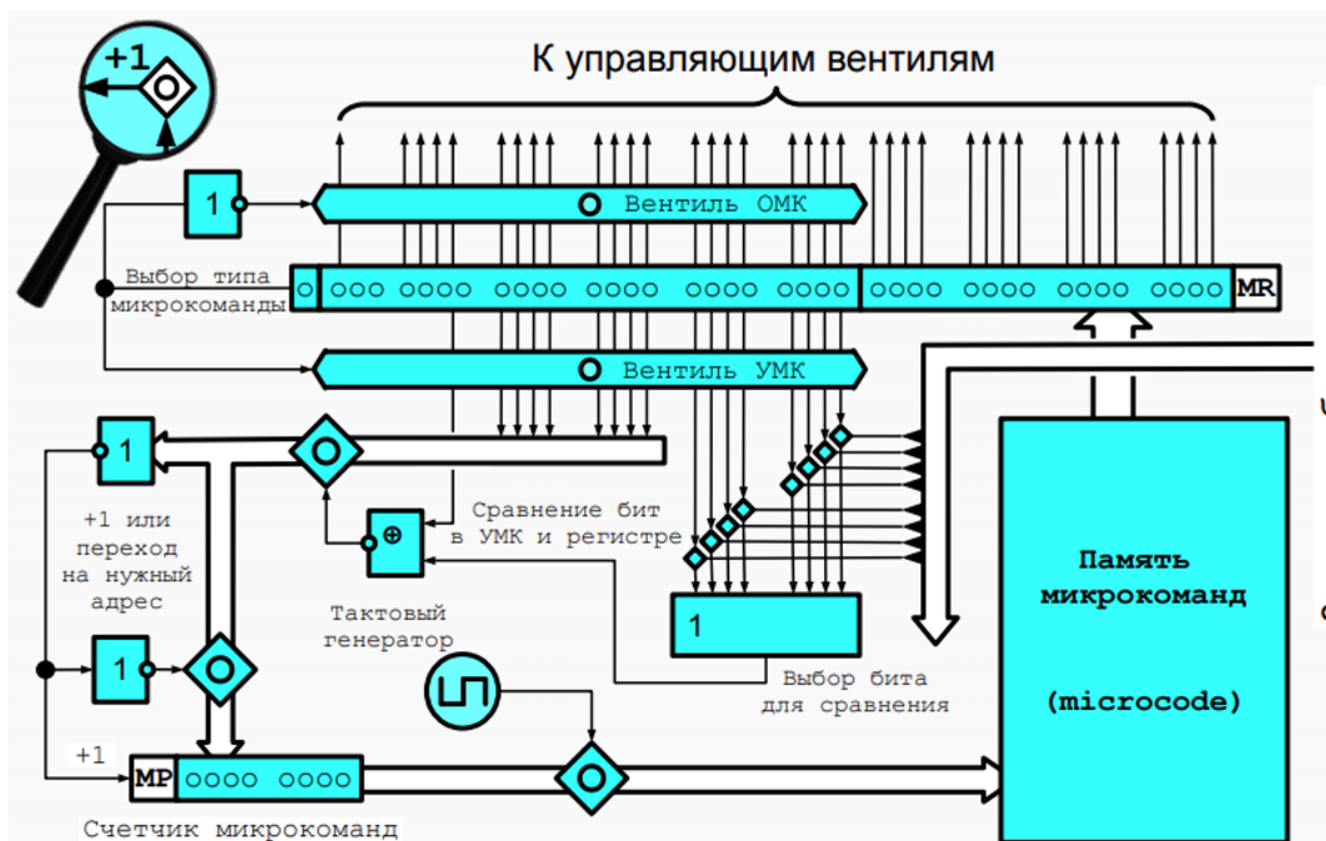
На второй схеме представлена работа с обратным кодом и сумматором:

- **Схема обратного кода:** Позволяет инвертировать входные данные.
- **Сумматор и логическое И:** Схема, которая объединяет операции сложения и логического умножения.

Пример работы с сигналами

1. **Сложение:**
 - Включаются сумматор и сигнал PLS1 для прибавления единицы.
2. **Вычитание:**
 - Включаются сигналы COML и COMR для инвертирования входных данных, затем используется сумматор.

22) Микропрограммное управление вентильными схемами. Схема управления. Интерпретатор БЭВМ.



Микропрограммное управление вентильными схемами

1. Схема управления

На изображении показана структура микропрограммного управления вентильными схемами. Рассмотрим основные элементы и их функции:

1. Счетчик микрокоманд (MP):

- Увеличивается на 1 (сигнал +1) или переходит на нужный адрес.
- Используется для последовательного выполнения микрокоманд из памяти микрокоманд.

2. Память микрокоманд (Microcode):

- Хранит микрокоманды, которые управляют процессором.
- Объем памяти составляет 256 ячеек, что позволяет хранить множество микрокоманд.

3. Вентиль УМК (Управляющий Микрокомандами) и Вентиль ОМК (Основной Микрокомандами)**:

- Определяют, какой тип микрокоманды будет выполняться.
- Эти вентили выбирают нужную микрокоманду для выполнения.

4. Тактовый генератор:

- Обеспечивает синхронизацию выполнения микрокоманд.
- Генерирует тактовые импульсы, которые используются для управления процессом выполнения микрокоманд.

5. Сравнение бит в УМК и регистре:

- Сравниваются биты микрокоманды и регистра для принятия решений о дальнейшем выполнении.
- Выбор бита для сравнения осуществляется с помощью мультиплексора.

2. Интерпретатор БЭВМ

Интерпретатор БЭВМ управляет выполнением команд процессора с помощью микрокоманд. Рассмотрим основные этапы его работы:

1. 256 ячеек для хранения микрокоманд, включая резерв:

- Объем памяти микрокоманд позволяет хранить до 256 различных микрокоманд, включая резервные.

2. Содержит горизонтальные микрокоманды:

- Горизонтальные микрокоманды включают множество управляющих сигналов, которые могут выполняться параллельно.

Циклы выполнения команд:

1. Цикл выборки команд:

- Извлечение следующей команды из памяти.

2. Цикл выборки адреса операнда и обработки режимов адресации:

- Определение адреса операнда с учетом режимов адресации.

3. Цикл выборки операнда:

- Извлечение операнда из памяти или регистра.

4. Цикл исполнения:

- Декодирование и исполнение различных типов команд:

- **Адресные команды:** Команды, требующие доступа к памяти.
- **Команды ветвлений:** Команды, изменяющие поток выполнения программы.
- **Безадресные команды:** Команды, не требующие указания адреса (например, команды работы с аккумулятором).

5. Декодирование и исполнение команд ввода-вывода:

- Управление операциями ввода-вывода с использованием специальных микрокоманд.

6. Цикл прерывания:

- Обработка прерываний, поступающих от внешних устройств или программ.

7. Пультовые операции:

- Управление работой процессора с помощью панели оператора (ввод, вывод, диагностика).

8. Свободные ячейки для:

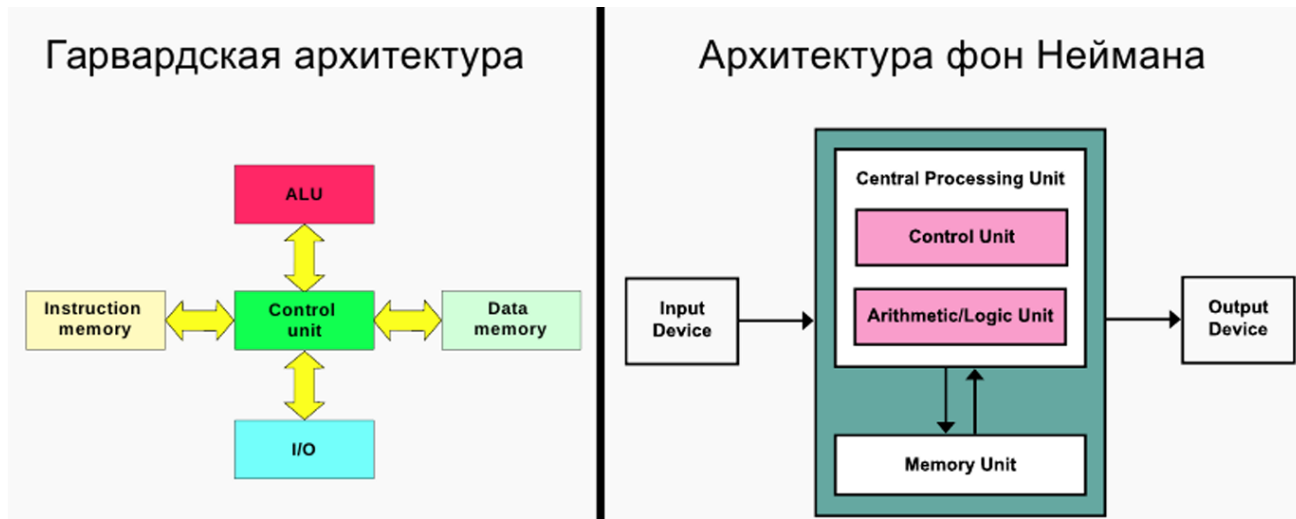
- Хранение микрокоманд, необходимых для выполнения специфических операций:
 - **Арифметическая команда**
 - **Команда перехода**
 - **Безадресная команда**

Итог

Микропрограммное управление вентиляемыми схемами является основой работы процессора. Интерпретатор БЭВМ, используя память микрокоманд и счетчик микрокоманд, обеспечивает выполнение команд процессора в несколько этапов, начиная с выборки команды и заканчивая обработкой прерываний и пультовых операций. Это позволяет реализовать сложные вычислительные процессы с высокой эффективностью и точностью.

23) Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура.

Организация обмена архитектуры ЭВМ с использованием шин.



Гарвардская архитектура

Гарвардская архитектура предполагает разделение памяти данных и памяти инструкций. Это означает, что для доступа к инструкциям и данным используются отдельные шины и устройства ввода-вывода.

Плюсы:

1. **Высокая производительность:** Параллельный доступ к памяти данных и памяти инструкций позволяет выполнять операции более эффективно.
2. **Отсутствие конфликтов:** Разделение памяти исключает конфликт доступа к памяти между инструкциями и данными.
3. **Увеличение скорости:** Возможность одновременного чтения инструкций и данных увеличивает общую скорость выполнения программы.

Минусы:

1. **Сложность реализации:** Наличие двух отдельных шин и памяти усложняет конструкцию и увеличивает стоимость.

2. **Гибкость:** Менее гибкая в плане изменения программного кода и данных, так как каждая часть имеет свое фиксированное место.

Архитектура фон Неймана

Архитектура фон Неймана использует одну общую память для инструкций и данных. Все компоненты компьютера подключены к одной шине.

Плюсы:

1. **Простота реализации:** Общая шина и память упрощают конструкцию и снижают стоимость производства.
2. **Гибкость:** Легкость в изменении программного кода и данных, так как они находятся в одной памяти.

Минусы:

1. **Конфликт доступа:** Процессор не может одновременно выполнять чтение инструкций и данных, что снижает производительность.
2. **Узкое место:** Чем больше шина, тем больше и ответственность. Шина является узким местом, так как все данные и инструкции проходят через нее, что ограничивает скорость.

Организация обмена архитектуры ЭВМ с использованием шин

Организация обмена данными в архитектурах ЭВМ осуществляется через шины, которые могут быть общими для всех компонентов (фон Нейман) или отдельными для инструкций и данных (Гарвард).

1. **Шина данных:** Используется для передачи данных между памятью, процессором и внешними устройствами.
2. **Шина адреса:** Передает адреса ячеек памяти или устройств ввода-вывода.
3. **Шина управления:** Передает управляющие сигналы, координируя работу всех компонентов.

Первое поколение ЭВМ — Электронные лампы

- **Примеры:** МЭСМ (1950), БЭСМ (1953).
- **Характеристики:** Низкая производительность (10000 операций в секунду), высокая потребляемая мощность (53 кВт).

Второе поколение ЭВМ — Транзисторы

- **Примеры:** 5Э926 (1964), БЭСМ-6 (1965).
- **Характеристики:** Увеличение производительности (до 500000 операций в секунду), самодиагностика, горячая замена компонентов, удаленное управление.

Третье поколение ЭВМ — Интегральные схемы

- **Примеры:** Директива «Ряд» (1968), ЕС ЭВМ (1971).
- **Характеристики:** Клонирование S/360, разработка клонов PDP-11.

Четвертое поколение ЭВМ — Сверхбольшие интегральные схемы

- **Примеры:** Эльбрус (разработка по настоящее время).
- **Характеристики:** Высокая производительность, масштабируемость, улучшенная архитектура.

Сравнение архитектур с точки зрения разработчика

Гарвардская архитектура:

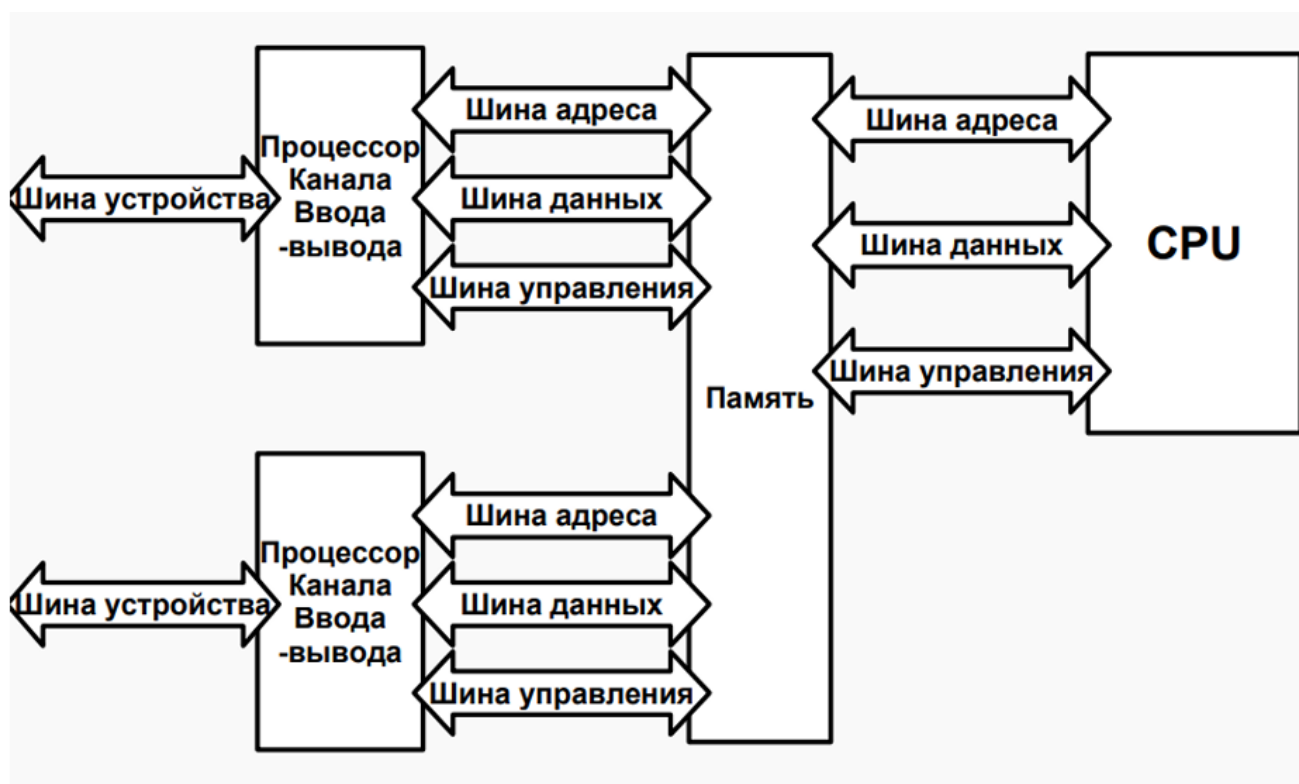
- **Преимущества:** Подходит для систем, где важна высокая производительность и минимальные задержки. Идеальна для применения в специализированных системах, таких как цифровые сигнальные процессоры (DSP) и микроконтроллеры.
- **Недостатки:** Сложность реализации и стоимость могут быть барьером для универсальных вычислительных систем.

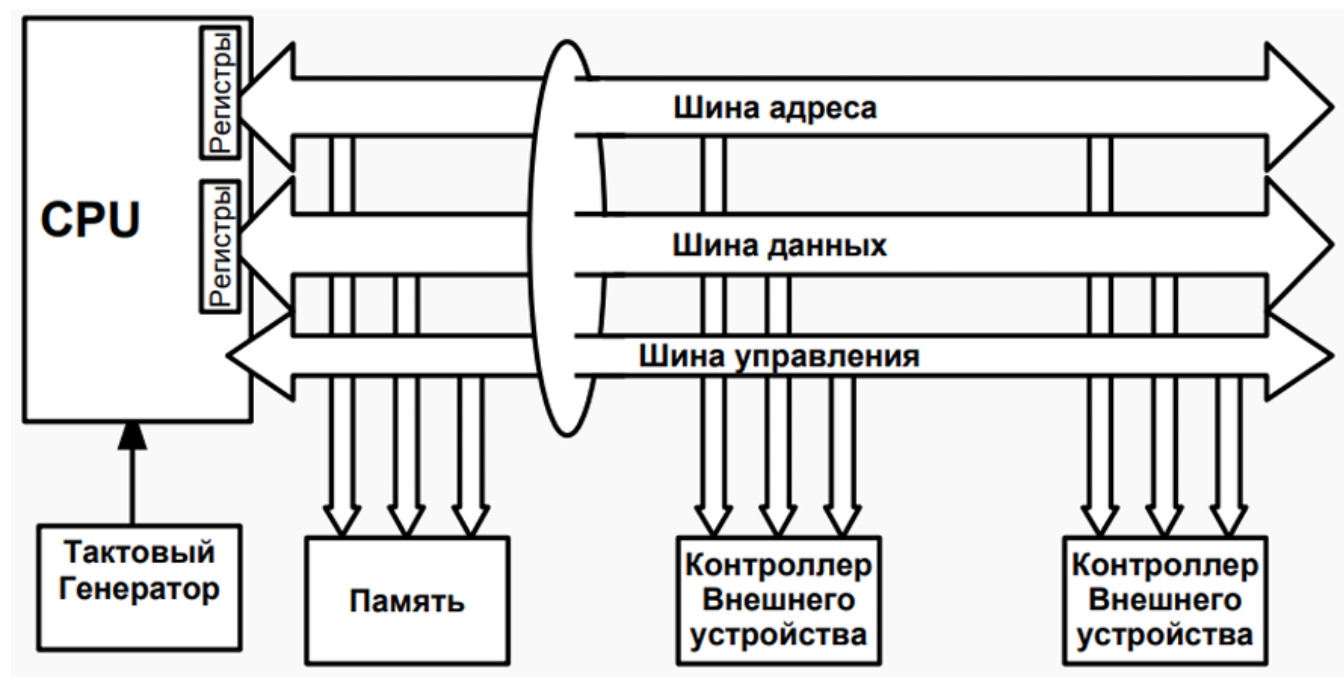
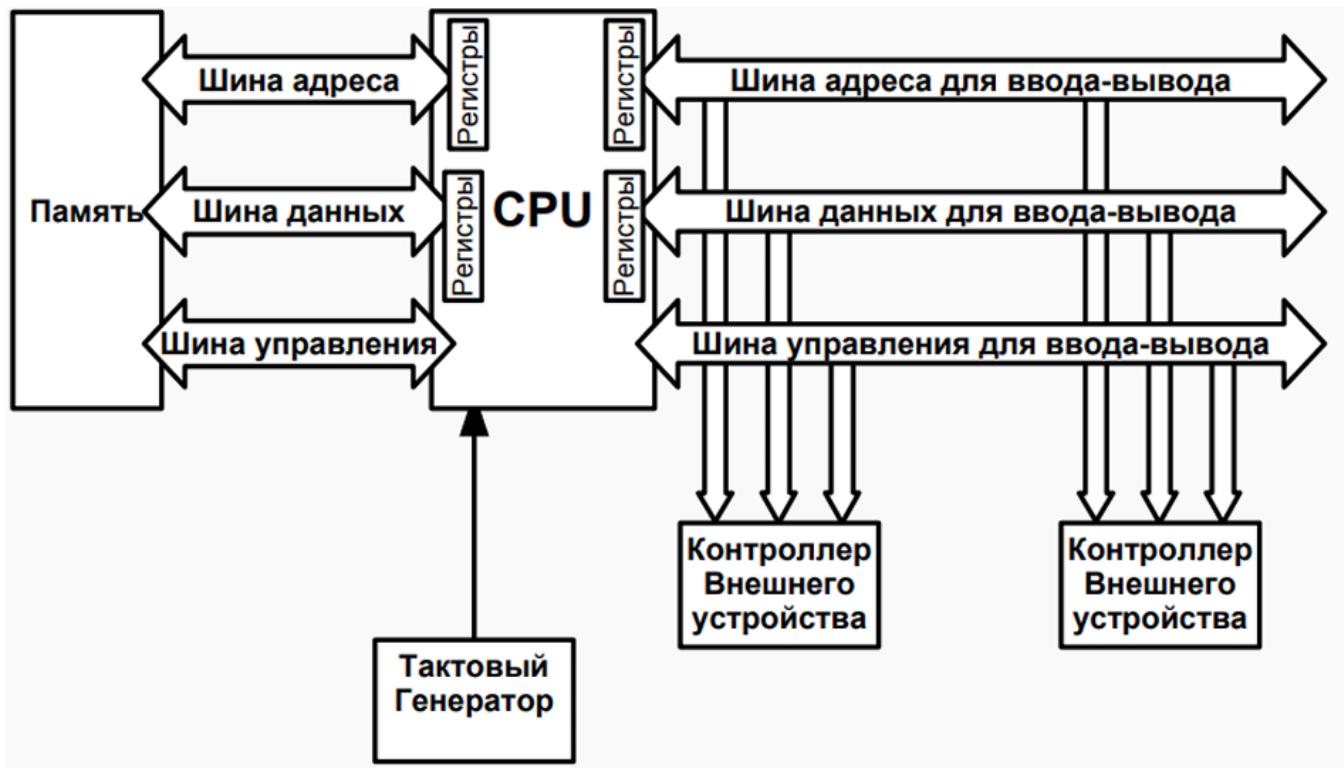
Архитектура фон Неймана:

- **Преимущества:** Простота и гибкость делают ее идеальной для общего применения, включая персональные компьютеры и серверы.
- **Недостатки:** Возможные конфликты доступа к памяти могут ограничивать производительность.

Выбор архитектуры зависит от конкретных требований проекта:

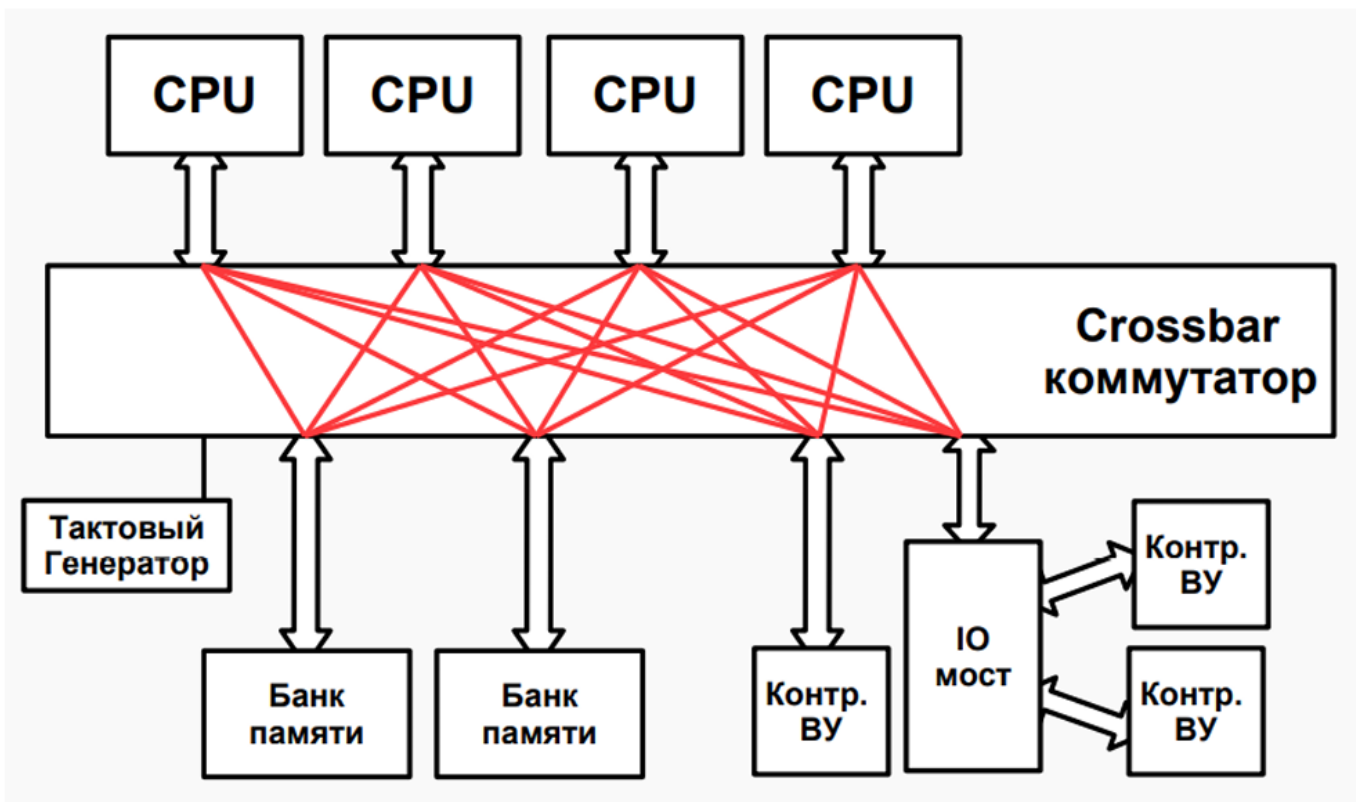
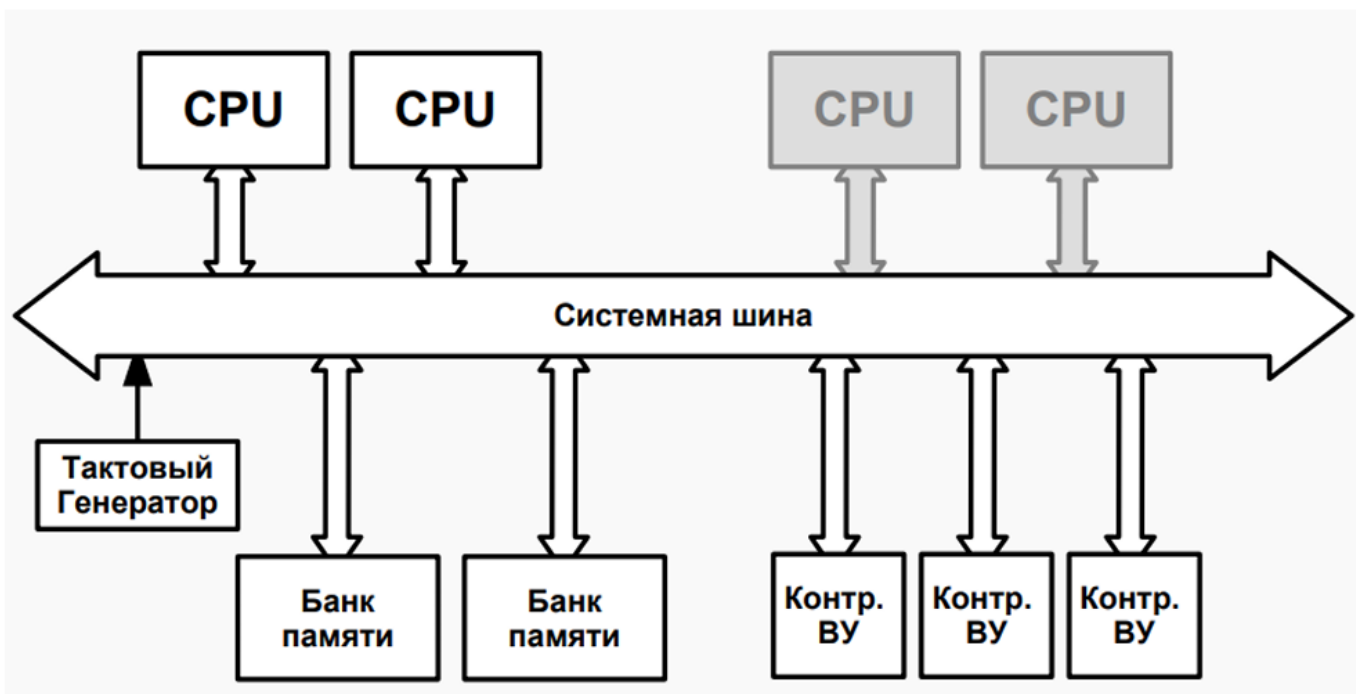
- Для высокопроизводительных систем реального времени предпочтительнее Гарвардская архитектура.
- Для универсальных и гибких систем, где важна простота и низкая стоимость, лучше подходит архитектура фон Неймана.



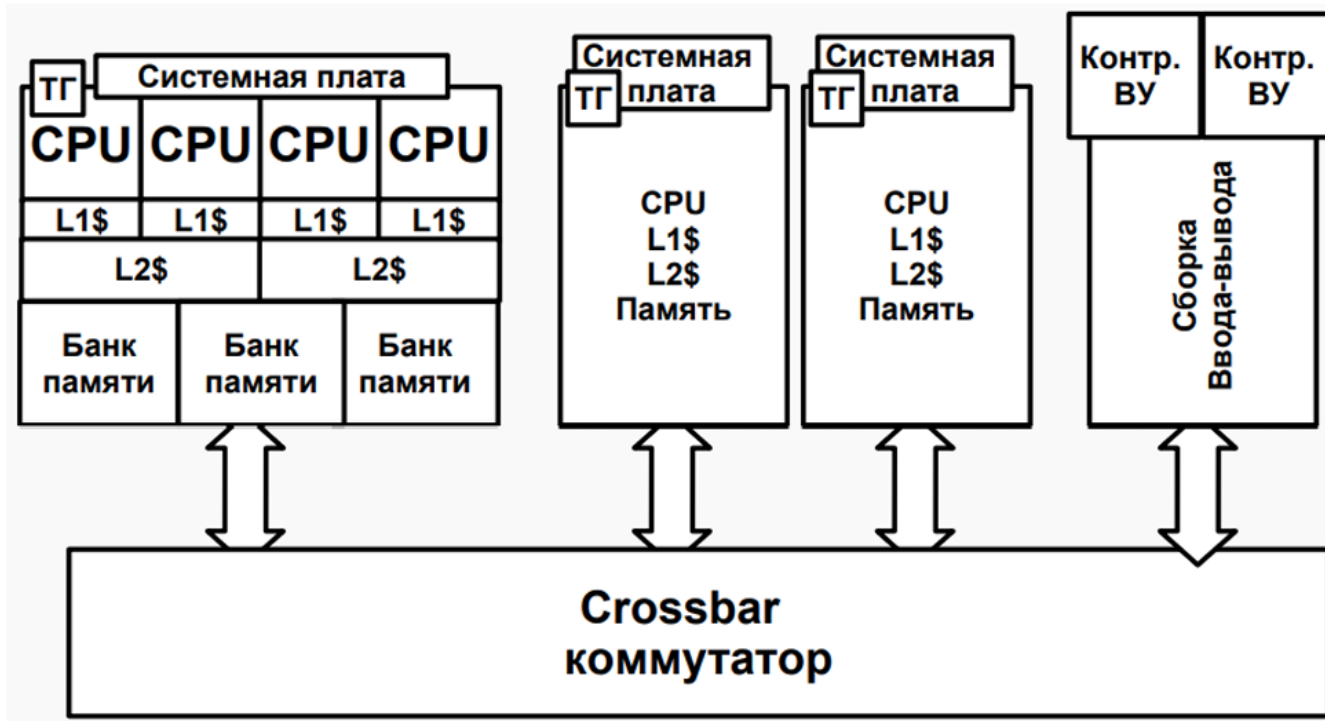


24) Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA.

UMA:



NUMA:



Система UMA (Uniform Memory Access)

Система UMA представляет собой архитектуру многопроцессорных систем с общей памятью. Все процессоры обращаются к единой памяти через межсоединительную сеть. Это позволяет каждому процессору иметь одинаковое время доступа к памяти и скорость доступа.

1. Структура:

- **Единая память:** Все процессоры обращаются к одной и той же памяти.
- **Межсоединительная сеть:** Может использовать одну шину, несколько шин или коммутатор для соединения процессоров с памятью.

2. Преимущества:

- **Равный доступ:** Одинаковое время доступа к памяти для всех процессоров.
- **Простота программирования:** Легче разрабатывать приложения, так как не нужно учитывать различия в доступе к памяти.

3. Недостатки:

- **Ограниченная пропускная способность:** Ограничена пропускной способностью шины или сети.
- **Медленная работа:** Меньшая скорость по сравнению с NUMA, особенно в параллельных системах.

NUMA (Non-Uniform Memory Access)

NUMA также является архитектурой многопроцессорных систем, но каждый процессор имеет свою выделенную память. Эти небольшие части памяти объединяются в единое адресное пространство, что позволяет получить доступ к любой ячейке памяти, используя физический адрес.

1. Структура:

- **Выделенная память:** Каждый процессор связан с определенной частью памяти.
- **Единое адресное пространство:** Все части памяти объединяются в одно адресное пространство.

2. Преимущества:

- **Высокая пропускная способность:** Более высокая пропускная способность по сравнению с UMA.
- **Быстрый доступ:** Уменьшенное время доступа к памяти, близкой к процессору.

3. Недостатки:

- **Сложность программирования:** Необходимость учитывать различия в доступе к памяти.
- **Изменяющееся время доступа:** Время доступа к памяти зависит от расстояния до памяти.

Сравнение UMA и NUMA

1. Контроллеры памяти:

- **UMA:** Использует один или два контроллера памяти.
- **NUMA:** Может иметь несколько контроллеров памяти.

2. Типы шин и сетей:

- **UMA:** Одиночные, множественные и перекрестные шины.
 - **NUMA:** Иерархические и древовидные шины и сетевые подключения.
3. **Время доступа к памяти:**
- **UMA:** Время доступа одинаково для всех процессоров.
 - **NUMA:** Время доступа изменяется в зависимости от расстояния до памяти.
4. **Приложения:**
- **UMA:** Подходит для приложений общего назначения и разделения времени.
 - **NUMA:** Ориентировано на приложения реального времени и критичные ко времени.
5. **Производительность:**
- **UMA:** Медленнее в параллельных системах.
 - **NUMA:** Быстрее, чем UMA, в параллельных системах.
6. **Пропускная способность:**
- **UMA:** Ограниченная пропускная способность.
 - **NUMA:** Более высокая пропускная способность.

Разбор схем

Первая схема: Архитектура UMA

1. **Системная шина:** Основной элемент, соединяющий все компоненты.
2. **Процессоры (CPU):** Подключены к общей шине.
3. **Банки памяти:** Подключены к общей шине и доступны всем процессорам.
4. **Контроллеры ввода-вывода:** Подключены к общей шине и обслуживают внешние устройства.
5. **Тактовый генератор:** Синхронизирует работу всех компонентов.

Вторая схема: Архитектура NUMA

1. **Crossbar коммутатор:** Соединяет все процессоры и банки памяти, позволяя прямой доступ между ними.
2. **Процессоры (CPU):** Подключены к коммутатору и имеют выделенную память.
3. **Банки памяти:** Каждый банк памяти связан с определенным процессором, но доступен всем через коммутатор.
4. **Контроллеры ввода-вывода:** Подключены к коммутатору через мост I/O.

Третья схема: Детализированная архитектура NUMA

1. **Системная плата:** Включает процессоры, кэш памяти первого (L1) и второго (L2) уровней.
2. **Банки памяти:** Связаны с определенной системной платой, но доступны через коммутатор.
3. **Crossbar коммутатор:** Центральный элемент, обеспечивающий соединение между всеми компонентами.
4. **Контроллеры ввода-вывода:** Подключены к коммутатору, обслуживают внешние устройства.

Выводы

UMA:

- Простой дизайн и равный доступ к памяти, но ограниченная пропускная способность и медленная работа в параллельных системах.

NUMA:

- Высокая производительность и пропускная способность, но сложное программирование и изменяющееся время доступа к памяти.

Для разработчика выбор архитектуры зависит от специфики задач. Для приложений общего назначения подходит UMA, для

высокопроизводительных и критичных ко времени приложений — NUMA.

25) Структура современных процессоров. Окружение процессора. CISC, RISC, VLIW.

<https://habr.com/en/companies/selectel/articles/542074/>

Современные процессоры имеют сложную и многоуровневую структуру. Рассмотрим основные характеристики и виды процессоров:

Разрядность адреса и данных

- **16/32/64 бита:** Процессоры могут работать с данными и адресами различной длины (16, 32 или 64 бита), что влияет на их производительность и возможности.

Тактовые частоты

- **500 МГц - 5 ГГц:** Современные процессоры работают на тактовых частотах от 500 мегагерц до 5 гигагерц, что определяет скорость выполнения инструкций.

Многопроцессорные и многоядерные системы

- **Многопроцессорные:** Системы могут иметь от 1 до 100 и более процессоров.
- **Многоядерные:** Один процессор может содержать от 1 до 16 ядер, каждое из которых способно выполнять задачи независимо.

Оперативная память

- **От 1 ГБ до терабайтов ОЗУ:** Современные компьютеры могут иметь от одного гигабайта до нескольких терабайтов оперативной памяти, что позволяет обрабатывать большие объемы данных.

Кэш-память разных уровней

- Процессоры используют кэш-память различных уровней (L1, L2, L3) для ускорения доступа к данным.

Суперскалярные архитектуры

- Современные процессоры могут выполнять несколько инструкций за один такт, что увеличивает их производительность.

Типы архитектур: CISC, RISC, VLIW

CISC (Complex Instruction Set Computer)

- **Описание:** Традиционные процессоры с большим набором сложных команд, таких как Intel.
- **Особенности:**
 - Небольшое количество регистров общего назначения.
 - Большое количество машинных команд, некоторые из которых реализуют сложные операции.
 - Разнообразие способов адресации операндов.
 - Множество форматов команд различной разрядности, некоторые команды обращаются к памяти.

RISC (Reduced Instruction Set Computer)

- **Описание:** Процессоры с упрощенным набором инструкций, выполнение каждой инструкции занимает один такт.
- **Идея:**
 - Ограничение списка команд наиболее часто используемыми простыми командами, работающими с данными в регистрах.
 - Обращение к памяти осуществляется только с помощью специальных команд чтения и записи.
 - Сокращение количества форматов команд и способов указания адресов операндов.

- Увеличение быстродействия.

VLIW (Very Long Instruction Word)

- **Описание:** Процессоры, в которых несколько инструкций упаковываются в одну команду и выполняются параллельно.
- **Идея:**
 - Основана на RISC-архитектуре, где несколько простых RISC-команд объединяются в одну длинную команду.
 - Компилятор анализирует программу, чтобы найти команды, которые могут выполняться одновременно, без конфликтов.
 - Компилятор объединяет такие команды в пакеты, каждый из которых рассматривается как одна сверхдлинная команда.

VLIW – появилась в России. Она не попадает под принципы фон Неймана (нарушает принцип программного управления, т.е. последовательного выполнения команд).

Архитектура ЭВМ с длинным командным словом (VLIW – Very Long Instruction Word) позволяет сократить объем оборудования, требуемого для реализации параллельной выдачи несколько команд, базируется на множестве независимых командных устройств. Вместо того чтобы выдавать на эти устройства последовательные команды, операции упаковываются в одну очень длинную команду. Ответственность за вывод параллельно выдаваемых для выполнения команд полностью ложится на компилятор.

В процессорах VLIW задача решается распределения работы между вычислительными модулями во время компиляции. Аппаратные средства, необходимые для реализации параллельной обработки, отсутствуют.

Для машин с VLIW-архитектурой был разработан новый метод планирования выдачи команд – трассировочное планирование. Во время планирования генерируется длинное командное слово. Процесс упаковки команд последовательной программы в длинные командные слова продолжается до тех пор, пока не будет оптимизирована вся программа.

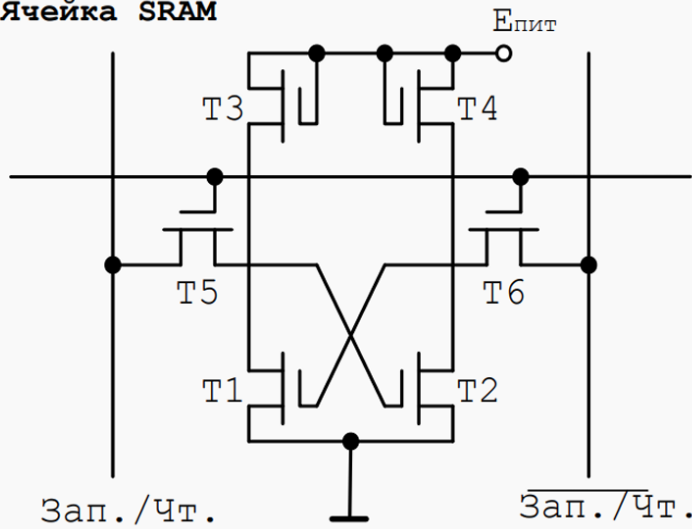
Примеры процессоров на этой архитектуре – Intel Itanium (архитектура IA-64, Merce

Итог

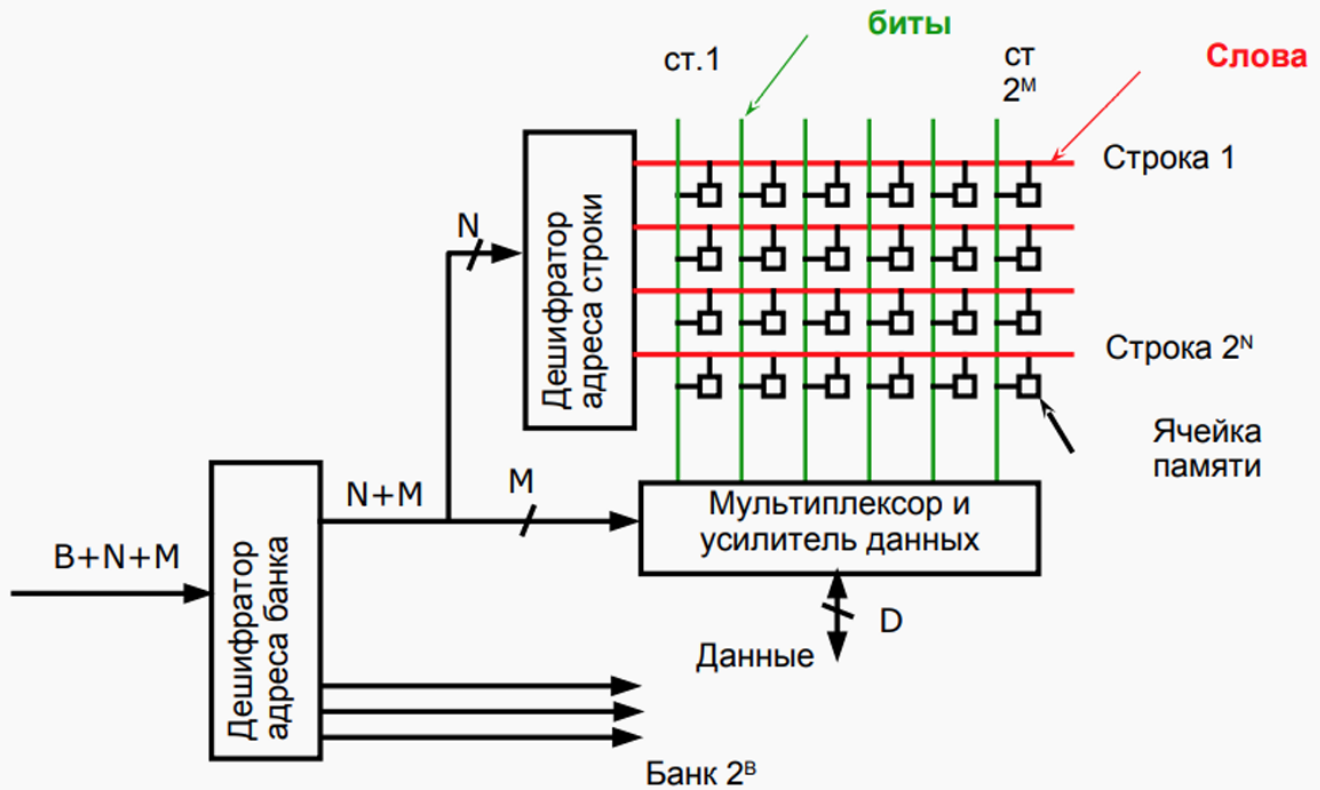
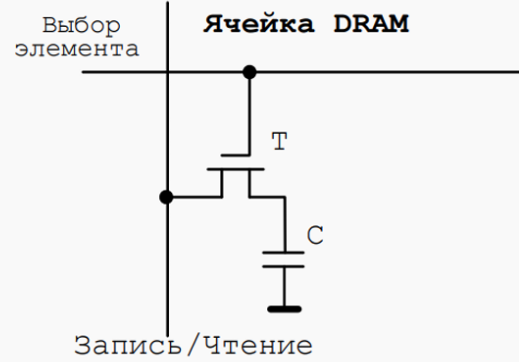
Современные процессоры могут иметь разную разрядность адреса и данных, высокие тактовые частоты, поддержку многопроцессорных и многоядерных конфигураций, а также использовать кэш-память разных уровней для увеличения производительности. Различные архитектуры процессоров, такие как CISC, RISC и VLIW, имеют свои особенности и преимущества, которые позволяют оптимизировать работу в различных сценариях.

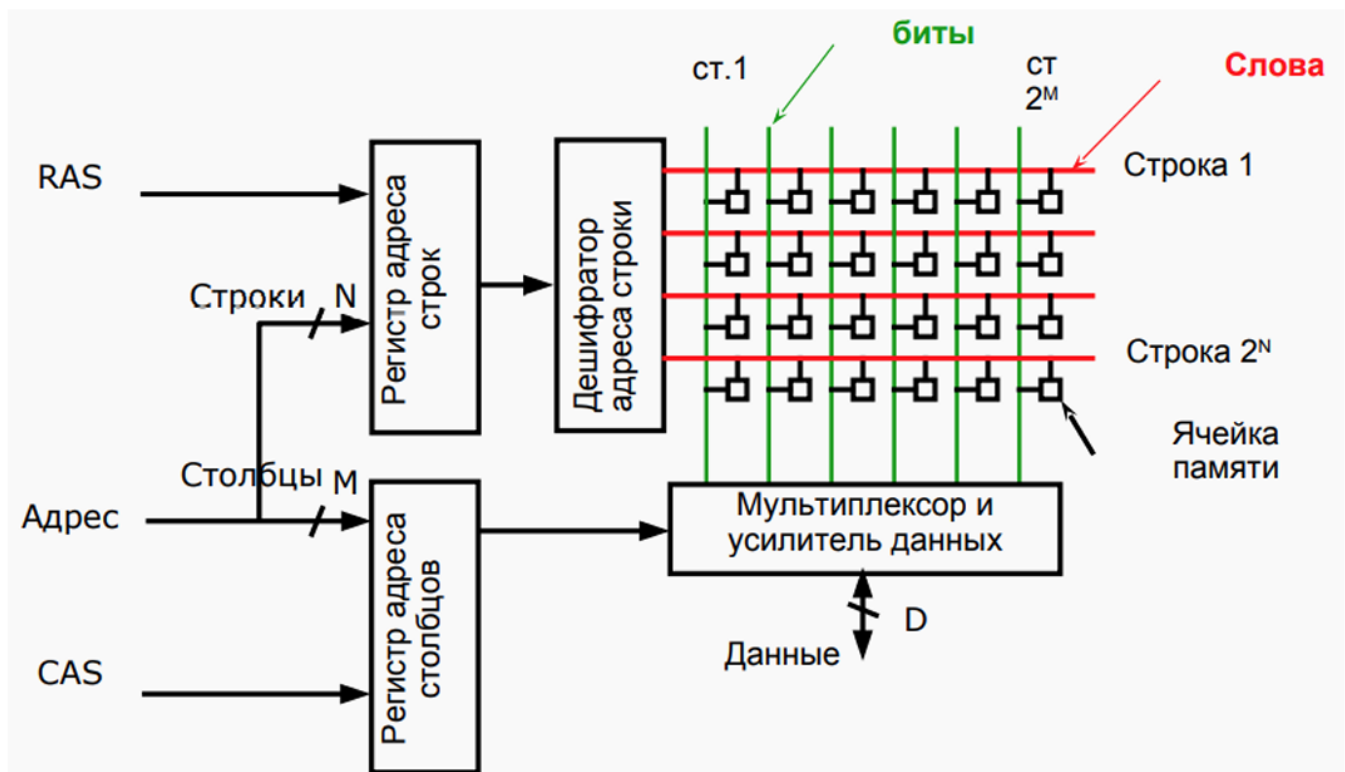
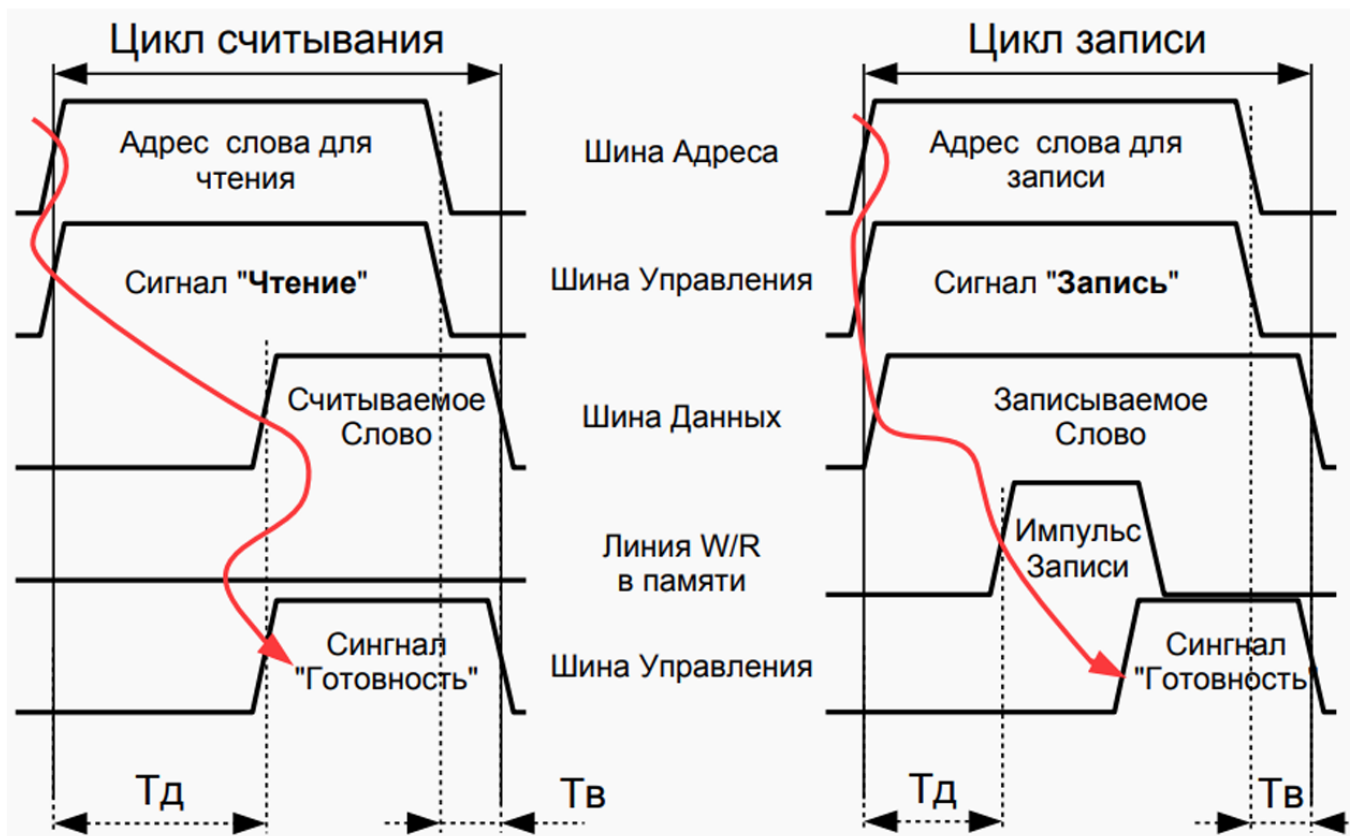
26) Адресуемая память, организация и временные диаграммы. Конструктивные особенности современной памяти.

Ячейка SRAM



Ячейка DRAM





Конструктивные особенности современной памяти

Современная память имеет ряд особенностей, которые улучшают её производительность и функциональность:

1. **Burst mode — пакетный режим**

- Процессор запрашивает данные не по одному байту, а сразу целыми пакетами, состоящими из 32 или 64 бит.
- Это позволяет считывать 2 или 4 машинных слова за один такт, что значительно ускоряет работу.

2. **Double Data Rate (DDR) — передача данных и по фронту, и по спаду**

- Передача данных осуществляется как по восходящему, так и по нисходящему фронтам тактового сигнала.
- Это удваивает скорость передачи данных без увеличения частоты тактового сигнала.

3. **SPD (Serial Presence Detect) — чип с идентификационной информацией**

- Чип содержит информацию о параметрах модуля памяти, что позволяет автоматически настраивать систему для оптимальной работы с данной памятью.

4. **Interleaving — расслоение памяти**

- Логически связанные байты располагаются в разных микросхемах памяти.
- Пока одна микросхема восстанавливается, процессор может работать с другой, что повышает производительность.

5. **DDR4-2133 8192MB PC4-17000 — индекс производительности**

- Обозначение DDR4-2133 указывает на тип памяти и её тактовую частоту (2133 МГц).
- 8192MB — объём памяти (8 ГБ).
- PC4-17000 — показатель производительности (17 000 МБ/с).

Другие особенности

1. **Включение динамической памяти в статическую память**

- Некоторое количество статической памяти (SRAM) включается в микросхемы динамической памяти (DRAM), что позволяет ускорить доступ к часто используемым данным.

2. Синхронная работа памяти и ЦП

- Использование внутренней конвейерной архитектуры и чередование адресов позволяет памяти работать синхронно с процессором, повышая общую производительность системы.

3. Чередование памяти (Interleaving Mode)

- Данные размещаются в разных микросхемах памяти, чтобы во время регенерации одной микросхемы процессор мог работать с другой.

4. Разбиение памяти на страницы (Paging Mode)

- Ячейки с одинаковым адресом строки группируются в страницы. Это экономит время на поиск и считывание данных, так как адрес строки остаётся неизменным.
- Обычно страницы имеют размер 512 байт и более.

5. Кэширование памяти

- Между процессором и оперативной памятью (RAM) размещается кэш-память объёмом до 2 МБ.
- Кэш-память работает на частоте процессора, что позволяет избежать циклов ожидания при обращении к часто используемым данным.

Разбор схем

Первая схема: Адресация и организация памяти

1. **Дешифратор адреса банка:** Преобразует адрес банка памяти, позволяя процессору выбрать нужный банк для чтения или записи данных.
2. **Дешифратор адреса строки:** Определяет конкретную строку в выбранном банке памяти.
3. **Мультиплексор и усилитель данных:** Считывает или записывает данные в ячейки памяти.

Временные диаграммы

1. Цикл считывания:

- **Адрес слова для чтения:** Процессор указывает адрес слова, которое нужно считать.
- **Сигнал "Чтение":** Активируется сигнал для чтения данных.
- **Считываемое слово:** Данные считываются и передаются процессору.
- **Сигнал "Готовность":** Указывает, что данные готовы для использования.

2. Цикл записи:

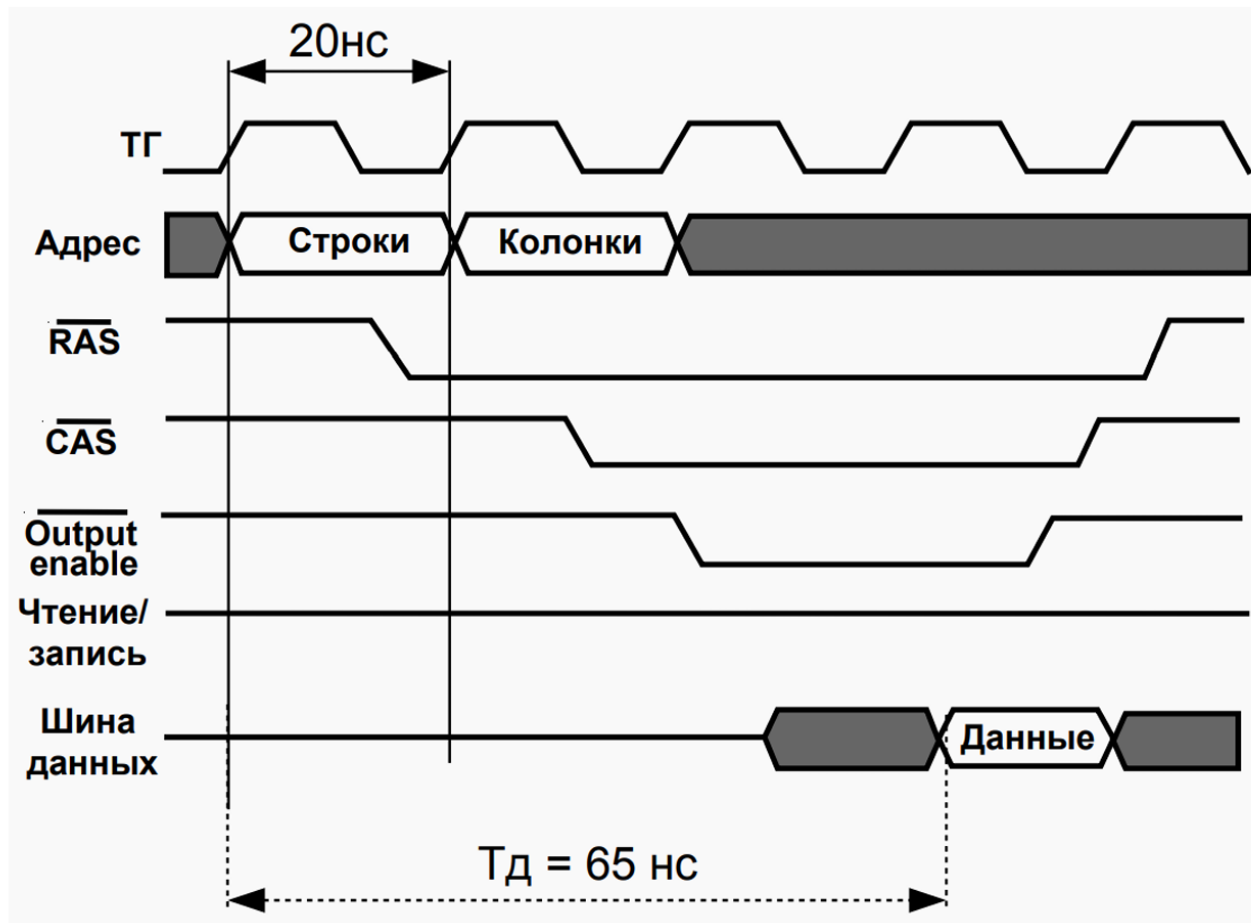
- **Адрес слова для записи:** Процессор указывает адрес слова, которое нужно записать.
- **Сигнал "Запись":** Активируется сигнал для записи данных.
- **Записываемое слово:** Данные передаются в память для записи.
- **Импульс записи:** Записывает данные в ячейку памяти.
- **Сигнал "Готовность":** Указывает, что запись завершена.

Вторая схема: Дешифрация и усиление данных

1. **Регистры адресов строк и столбцов:** Хранят адреса строк и столбцов, которые определяют ячейку памяти.
2. **Дешифратор адреса строки:** Преобразует адрес строки в сигнал для выборки конкретной строки.
3. **Мультиплексор и усилитель данных:** Управляет считыванием или записью данных в ячейки памяти.

Третья схема: Временные диаграммы DRAM

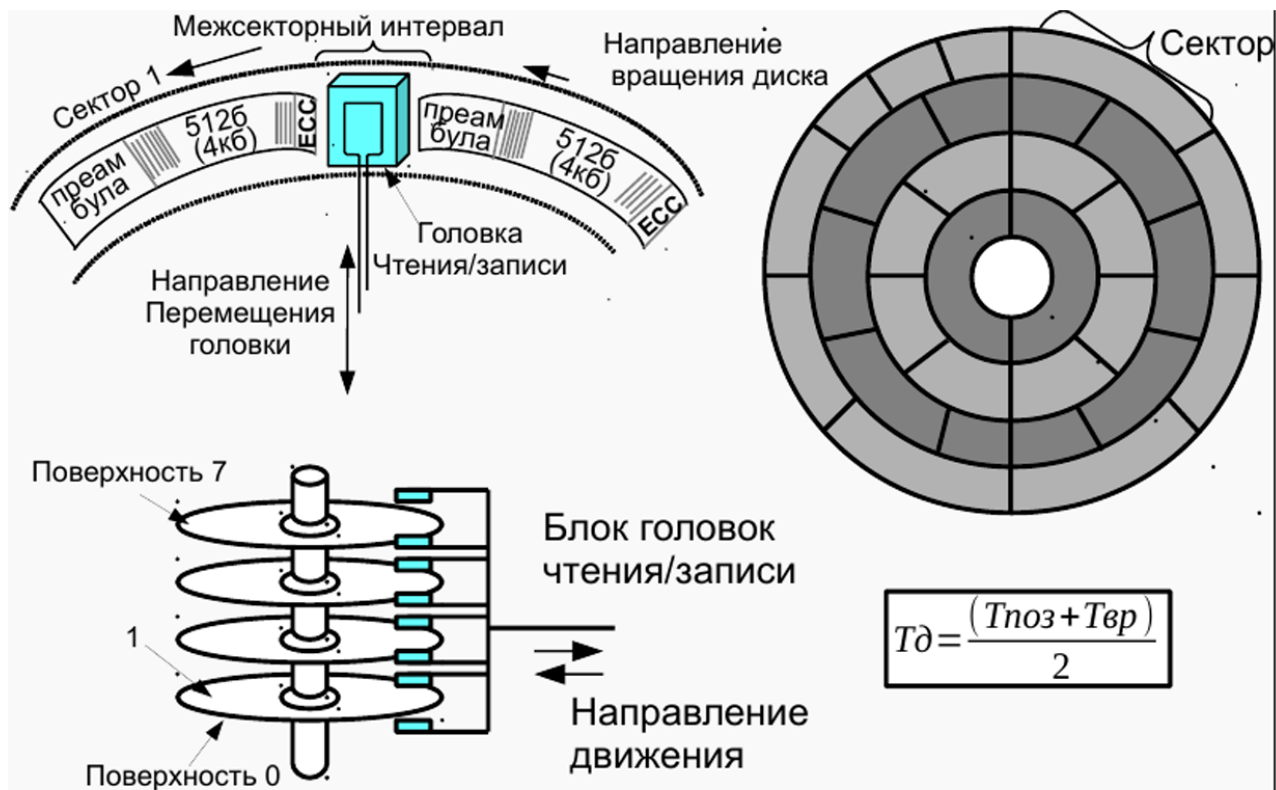
1. **Сигналы RAS и CAS:** Определяют строки и столбцы для доступа к данным.
2. **Сигнал Output Enable:** Активирует выходные данные.
3. **Сигнал Чтение/Запись:** Определяет режим работы (чтение или запись).
4. **Тайминг:** Общее время доступа к данным (Тд) составляет 65 нс, что показывает задержку при доступе к данным.



Итог

Современная память имеет сложную организацию и использует различные технологии для повышения производительности. Burst mode, Double Data Rate, Interleaving, Paging и кэширование — это основные методы, которые делают память более быстрой и эффективной. Временные диаграммы показывают, как процессор взаимодействует с памятью во время чтения и записи данных, что помогает оптимизировать работу системы.

27) Память, ориентированная на записи (блочная память). Организация дисковой памяти и памяти на магнитных лентах.



Блочная память, такая как дисковая память и память на магнитных лентах, организована в виде блоков или секторов, что позволяет эффективно управлять данными и обеспечивать их долговременное хранение.

Организация дисковой памяти

Дисковая память использует магнитные диски для хранения данных. Она состоит из нескольких ключевых элементов и организована особым образом:

1. Диск и его структура:

- **Секторы:** Диск разделён на сектора, каждый из которых содержит определённое количество данных (обычно 512 байт или 4 КБ).
- **Треки:** Сектора располагаются вдоль concentric окружностей, называемых треками.
- **Цилиндры:** Совокупность треков одинакового радиуса на всех поверхностях диска образует цилиндр.

2. Головка чтения/записи:

- **Блок головок чтения/записи:** Каждый диск имеет отдельные головки для чтения и записи данных, которые перемещаются над поверхностью диска.
- **Направление движения:** Головка перемещается радиально для доступа к различным трекам и секторам.
- **Межсекторный интервал:** Пространство между секторами, позволяющее головке перемещаться между ними.

3. Процесс чтения/записи:

- **Адресация:** Адрес каждого блока данных включает номер цилиндра, номер трека и номер сектора.
- **Чтение:** Процесс чтения данных начинается с позиционирования головки над нужным цилиндром и треком, после чего считывается сектор данных.
- **Запись:** Процесс записи данных аналогичен чтению, но включает изменение магнитного состояния диска для записи новых данных.

4. Среднее время доступа (Тд):

- Среднее время доступа к данным на диске определяется как среднее арифметическое между временем позиционирования головки (Тпоз) и временем вращения диска (Твр): $T_d = \frac{(T_{поз} + T_{вр})}{2}$

Организация памяти на магнитных лентах

Магнитные ленты также используются для долговременного хранения данных, но их организация отличается от дисковой памяти:

1. Лента и её структура:

- **Ленточный носитель:** Данные записываются на длинную магнитную ленту.
- **Блоки данных:** Лента делится на блоки, которые содержат данные и метаданные (например, контрольные суммы для проверки целостности данных).

2. Процесс чтения/записи:

- **Последовательный доступ:** Данные читаются и записываются последовательно, что делает доступ к данным медленным по сравнению с дисковыми накопителями.
- **Головка чтения/записи:** Перемещается вдоль ленты для чтения или записи данных.

3. Преимущества и недостатки:

- **Преимущества:** Высокая плотность записи, низкая стоимость хранения больших объёмов данных.
- **Недостатки:** Медленный доступ к данным, необходимость перемотки ленты для поиска нужного блока данных.

Разбор схем

Первая схема: Организация дисковой памяти

1. Сектора и межсекторный интервал:

- Данные хранятся в секторах, между которыми есть интервалы для перемещения головки.
- **Пример буфера:** Предварительно записанные данные, предназначенные для ускорения процесса чтения/записи.

2. Головка чтения/записи:

- Перемещается по радиусу для доступа к различным трекам.
- Читает или записывает данные на диске в зависимости от команды.

3. Треки и цилиндры:

- Треки располагаются вдоль концентрических окружностей.
- Совокупность треков одинакового радиуса на всех дисках образует цилиндр.

Вторая схема: Организация памяти на магнитных лентах

1. Ленты и блоки данных:

- Лента состоит из блоков данных, которые хранятся последовательно.
- Используются для долговременного хранения больших объёмов данных.

2. Головка чтения/записи:

- Перемещается вдоль ленты для доступа к нужным блокам данных.
- Последовательный доступ обеспечивает надёжность хранения, но замедляет процесс поиска данных.

Итог

Блочная память, как дисковая память и память на магнитных лентах, организована для эффективного хранения и управления данными. Дисковые накопители предлагают быстрый доступ благодаря случайному доступу, в то время как магнитные ленты обеспечивают надёжное и экономичное хранение больших объёмов данных, но с медленным доступом.

28) Характеристики запоминающих устройств. Пирамида памяти.

Запоминающие устройства в компьютерных системах имеют различные характеристики, которые определяют их эффективность и область применения. Рассмотрим основные характеристики памяти и её пирамиду.

Месторасположение

- **Процессорные:** Встроенные в процессор (например, регистры).
- **Внутренние:** Память, находящаяся внутри системного блока (например, ОЗУ).
- **Внешние:** Внешние устройства хранения (например, жесткие диски, SSD, магнитные ленты).

Ёмкость

- Емкость памяти измеряется в метрических множителях (килобайты, мегабайты) или в двоичных множителях (кибибайты, мебибайты).

Единица пересылки

- **Слово:** Минимальная единица данных, обрабатываемая процессором.
- **Строка кэша:** Блок данных, используемый в кэш-памяти.
- **Блок на диске:** Минимальная единица данных, хранящаяся на диске.

Метод доступа

- **Произвольный (адресный):** Доступ к любой ячейке памяти возможен напрямую по её адресу.
- **Ориентированный на записи (прямой):** Доступ осуществляется к блокам данных (например, SSD).
- **Последовательный:** Доступ к данным происходит последовательно (например, магнитные ленты).
- **Ассоциативный:** Доступ к данным по их содержимому (например, кэш-память).

Характеристики памяти

1. Быстродействие и временные соотношения

- **Время доступа (Тд):** Время от момента подачи адреса до момента получения данных.
- **Длительность цикла памяти (Тц):** Время между двумя последовательными операциями доступа к памяти.
- **Время чтения и время записи:** Интервалы времени, необходимые для чтения и записи данных.
- **Время восстановления (Тв):** Время, необходимое для восстановления содержимого ячеек памяти.
- **Скорость передачи информации:** Количество данных, передаваемых за единицу времени.

2. Физический тип и особенности

- Различные типы памяти имеют свои физические особенности, такие как энергозависимость, плотность записи и т.д.

3. **Стоимость**

- Стоимость памяти зависит от её типа, ёмкости и быстродействия.

Пирамида памяти

Пирамида памяти показывает различные уровни памяти в компьютере, их объём и скорость доступа:

1. **CPU:**

- Объём: 100-1000 байт
- Время доступа (Тд): <1 нс
- Тип: Регистры
- Управление: Компилятор

2. **L1 Cache:**

- Объём: 32-128 КБ
- Время доступа (Тд): 1-4 нс
- Тип: Ассоциативная память
- Управление: Аппаратура

3. **L2-L3 Cache:**

- Объём: 0.5-32 МБ
- Время доступа (Тд): 8-20 нс
- Тип: Ассоциативная память
- Управление: Аппаратура

4. **Основная память (ОЗУ):**

- Объём: 0.5 ГБ - 4 ТБ
- Время доступа (Тд): 60-200 нс
- Тип: Адресная память
- Управление: Программное

5. **SSD:**

- Объём: 128 ГБ - 1 ТБ на диск

- Время доступа (Тд): 25-250 мкс
- Тип: Блочная память
- Управление: Программное

6. Жесткие диски:

- Объём: 0.5 ТБ - 4 ТБ на диск
- Время доступа (Тд): 5-20 мс
- Тип: Блочная память
- Управление: Программное

7. Магнитные ленты:

- Объём: 1-6 ТБ на кассету
- Время доступа (Тд): 1-240 секунд
- Тип: Последовательная память
- Управление: Программное

Детали

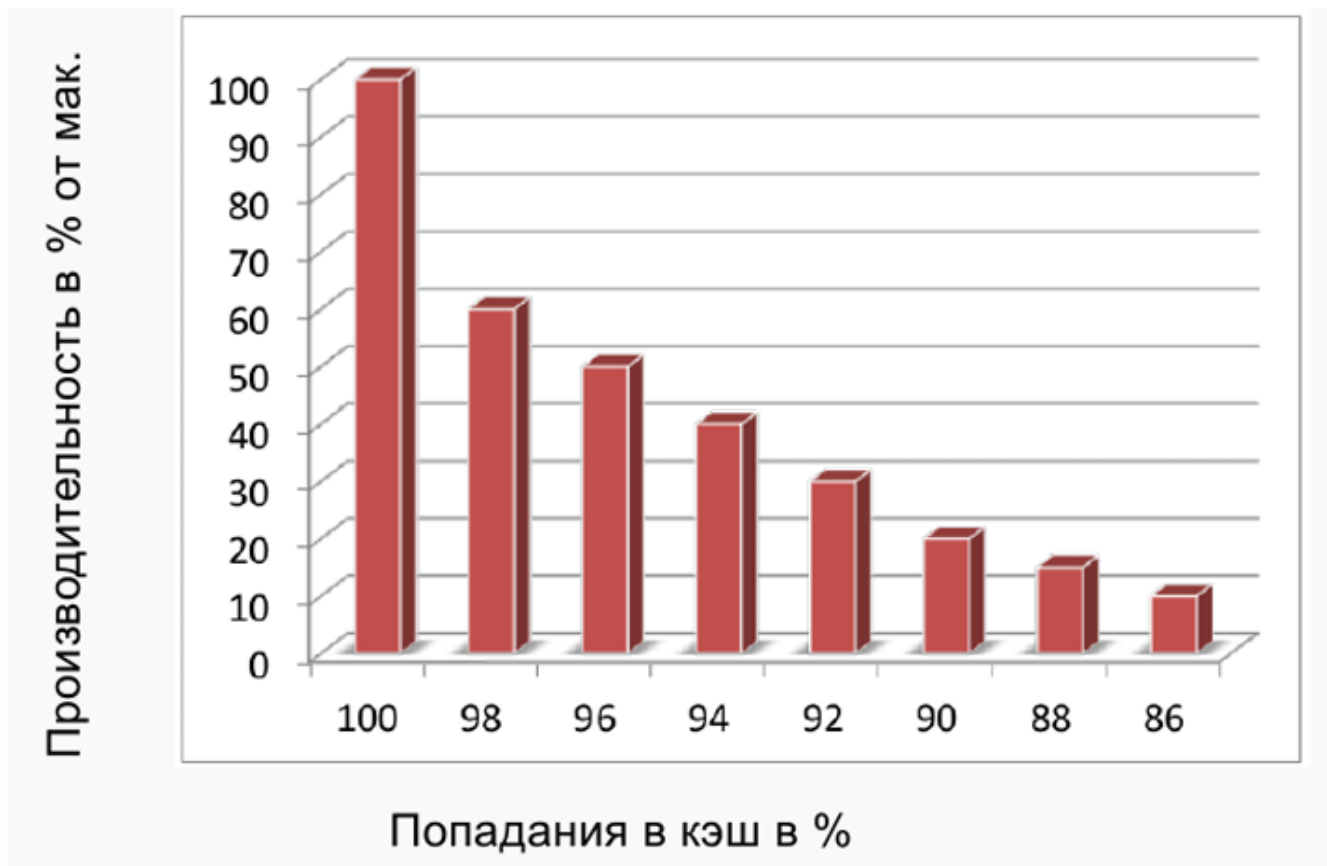
- **Время доступа:** Временной интервал от подачи адреса до получения данных.
- **Время записи:** Интервал времени для записи данных в память.

	Объем	Тд	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Кб	1-4нс	2с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Мб	8-20нс	19с	Ассоц.	аппаратура
Основная память	0.5Гб-4ТБ	60-200нс	50-300с	Адресная	программно
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	программно
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4м	Блочн.	программно
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

Итог

Запоминающие устройства имеют различные характеристики, влияющие на их производительность и стоимость. Пирамида памяти показывает, как различные уровни памяти взаимодействуют и обеспечивают баланс между скоростью доступа и ёмкостью хранения данных.

29) Ассоциативная память, Кэш-память. Влияние промахов кэш памяти на производительность.



Введение

Процессоры всегда работают быстрее, чем память, поэтому разница в скорости приводит к задержкам в работе. Инженеры знают, как построить быструю память, но она дорого стоит и ограничена в размерах. Для решения этой проблемы используется кэш-память, которая находится ближе к процессору и хранит наиболее часто используемые данные.

Ассоциативная память (АМ)

Ассоциативная память, или память, адресуемая по содержимому, используется для быстрого поиска данных. В ней данные выбираются не по адресу, а по содержимому. Пример ассоциативной памяти — это кэш-память.

Принципы работы ассоциативной памяти:

1. Минимальная адресуемая единица (МАЕ):

- В ассоциативной памяти МАЕ называется строкой ассоциативной памяти (СтрАП).
- Каждая СтрАП содержит два поля: поле тега (метки) и поле данных.

2. Запрос на чтение:

- Запрос выглядит как: "выбрать строку, у которой тег равен заданному значению".
- Возможны три результата: одна строка с заданным тегом, несколько строк с заданным тегом, ни одной строки с заданным тегом.

Схема ассоциативной памяти:

- **Регистр ассоциативного признака:** Хранит тег.
- **Регистр маски:** Указывает, какие биты тега важны для поиска.
- **Схемы совпадения:** Проверяют соответствие тегов строк в памяти с запрашиваемым тегом.
- **Регистр совпадений:** Содержит результаты проверки.

Кэш-память

Кэш-память используется для уменьшения времени доступа к данным. Она хранит копии часто используемых данных из основной памяти.

Структура кэш-памяти:

- **Кэш-контроллер:** Управляет работой кэш-памяти, загружает данные из оперативной памяти и возвращает модифицированные данные обратно.
- **Процессор:** Запрашивает данные у кэш-контроллера.
- **Оперативная память:** Хранит все данные, к которым обращается процессор.

Принципы работы кэш-памяти:

1. Кэш-попадание (cache hit):

- Процессор запрашивает данные, которые находятся в кэш-памяти.
- Данные извлекаются быстро.

2. Кэш-промах (cache miss):

- Запрашиваемые данные отсутствуют в кэш-памяти.
- Кэш-контроллер запрашивает данные из оперативной памяти.

Влияние кэш-промахов на производительность:

- Чем чаще кэш-промахи, тем ниже производительность.
- **График зависимости производительности от попаданий в кэш:**
 - При 100% попаданий в кэш производительность максимальна.
 - Снижение попаданий в кэш приводит к значительному падению производительности.

Принципы локальности

Принцип локальности заключается в том, что программы часто обращаются к небольшому участку памяти в течение некоторого времени.

Типы локальности:

1. **Временная локальность:** Если данные были использованы недавно, то они, вероятно, будут использованы снова в ближайшем будущем.

2. **Пространственная локальность:** Если данные из определенного адреса памяти были использованы, то данные из соседних адресов, вероятно, будут использованы вскоре.

Уровни кэш-памяти

1. **L1 Cache:** Находится непосредственно в процессоре, очень быстрая, но малая по объему.
2. **L2 Cache:** Расположена рядом с процессором, больше по объему, чем L1, но медленнее.
3. **L3 Cache:** Расположена на материнской плате, ещё больше по объему и медленнее, чем L2.

Итог

Ассоциативная память и кэш-память играют важную роль в повышении производительности современных компьютеров. Кэш-память уменьшает задержки при доступе к данным, а ассоциативная память обеспечивает быстрый поиск данных по содержимому. Основной принцип работы кэш-памяти — использование принципа локальности для хранения наиболее часто используемых данных, что позволяет значительно сократить время доступа к ним.

30) Предназначение и организация виртуальной памяти. Сегментно страничная организация. Устройство управления памятью (MMU), буфер трансляции (TLB).

Предназначение виртуальной памяти

Виртуальная память позволяет операционной системе (ОС) использовать больше памяти, чем установлено физически в компьютере. Это достигается за счёт использования части жёсткого диска как

дополнительной памяти. Виртуальная память позволяет размещать программы на диске так, будто они находятся в оперативной памяти, что облегчает работу пользователю, так как он работает с общим адресным пространством и не думает о том, где именно находятся данные.

Организация виртуальной памяти

Страничная организация

В большинстве современных ОС виртуальная память организуется с помощью страничной адресации. Основные моменты:

1. Разделение памяти на страницы:

- Память делится на фиксированные области, называемые страницами.
- Виртуальный адрес содержит номер страницы и смещение внутри страницы.

2. Процесс обращения:

- Если страница находится в оперативной памяти, данные читаются напрямую.
- Если страницы нет в оперативной памяти, ОС подкачивает её с жёсткого диска.

3. Управление памятью:

- ОС может выгружать неиспользуемые страницы на диск.
- Критические данные всегда находятся в оперативной памяти.

Сегментная организация

Сегментная организация разделяет виртуальную память на сегменты произвольного размера:

1. Сегменты:

- Виртуальное пространство делится на части разного размера — сегменты.
- Сегменты могут иметь права доступа.

2. Загрузка сегментов:

- Часть сегментов загружается в оперативную память, часть остаётся на диске.
- Сегменты могут занимать несмежные участки памяти.

3. Недостатки:

- Фрагментация памяти.
- Более медленное преобразование адреса по сравнению со страничной организацией.

Устройство управления памятью (MMU) и буфер трансляции (TLB)

MMU (Memory Management Unit)

MMU отвечает за преобразование виртуальных адресов в физические:

1. Процесс преобразования:

- Процессор подаёт виртуальный адрес на вход MMU.
- MMU трансформирует виртуальный адрес в физический, заменяя номер виртуальной страницы на номер физической.

2. Порядок работы:

- Если MMU выключено или адрес попал в нетранслируемую область, физический адрес равен виртуальному.
- Если MMU включено и адрес в транслируемой области, происходит трансляция адреса:
 - Если нужный номер страницы есть в TLB, используется он.
 - Если нет, идёт поиск в таблицах страниц ОС, после чего запись добавляется в TLB.

TLB (Translation Lookaside Buffer)

TLB ускоряет процесс преобразования виртуальных адресов:

1. Функции:

- Кэширует часто используемые преобразования адресов.

- Разделён для адресов и данных.

2. Организация:

- Организован как ассоциативная память.

Итог

Виртуальная память позволяет эффективно использовать ресурсы компьютера, предоставляя возможность запускать программы, требующие больше памяти, чем установлено физически. Сегментно-страничная организация и использование MMU и TLB обеспечивают быстрый доступ к данным и минимизируют задержки.

31) Сетевые технологии, Понятие сети ЭВМ, классификация компьютерных сетей. Сообщение и пакет. Модель взаимодействия открытых систем.

Понятие сети ЭВМ и классификация компьютерных сетей

Компьютерная сеть — это система взаимосвязанных компьютеров, которые могут обмениваться информацией без использования промежуточных носителей. Компьютерные сети включают в себя аппаратные и программные компоненты, такие как сетевые операционные системы и сетевые приложения (например, почтовые программы, сетевые базы данных).

Классификация сетей:

1. По размеру:

- PAN (Personal Area Network) — персональная сеть.
- LAN (Local Area Network) — локальная сеть.
- MAN (Metropolitan Area Network) — городская сеть.
- WAN (Wide Area Network) — глобальная сеть.

2. По принадлежности:

- Офисные.
- Корпоративные.
- Частные VPN (Virtual Private Network).

3. По назначению:

- Вычислительные.
- Информационные.
- Информационно-вычислительные.
- Информационно-управляющие.

4. По области применения:

- SAN (Storage Area Network) — сети хранения данных.
- Серверные фермы.

5. Прочие типы:

- Иерархические.
- Беспроводные.
- Виртуальные VLAN.

Virtual Local Area Network (виртуальная локальная сеть).

Подробнее о VLAN:

VLAN позволяет объединить группы устройств в разные логические сети, даже если они находятся в разных физических сегментах сети. Это делается для улучшения управления сетью, повышения безопасности и оптимизации использования ресурсов. Основные преимущества VLAN:

1. **Сегментация сети:** Устройства, принадлежащие разным VLAN, находятся в разных логических сетях и могут обмениваться данными только через маршрутизатор, что снижает вероятность столкновений данных и повышает производительность сети.
2. **Безопасность:** Ограничение трафика между VLAN повышает безопасность, так как устройства в одной VLAN не могут напрямую взаимодействовать с устройствами в другой VLAN без соответствующей настройки маршрутизаторов и правил безопасности.

3. **Управляемость:** VLAN позволяет легко управлять и настраивать сеть, изменять принадлежность устройств к определенным VLAN без физического перемещения оборудования.

VLAN обычно настраиваются на управляемых коммутаторах и могут быть идентифицированы с помощью меток (тегов), которые добавляются к Ethernet-кадрам для обозначения их принадлежности к определенной VLAN.

Сообщение и пакет

Сообщение — это целостная единица данных, состоящая из заголовка и информации. При передаче сообщений создается логический канал, по которому информация передается от одного узла к другому.

Пакет — это часть сообщения, передаваемая по сети. Сообщения разбиваются на пакеты, каждый из которых содержит заголовок с адресной информацией и номер пакета, необходимый для сборки сообщения на узле назначения.

Модель взаимодействия открытых систем (OSI)

Модель OSI состоит из 7 уровней, каждый из которых выполняет определенные функции:

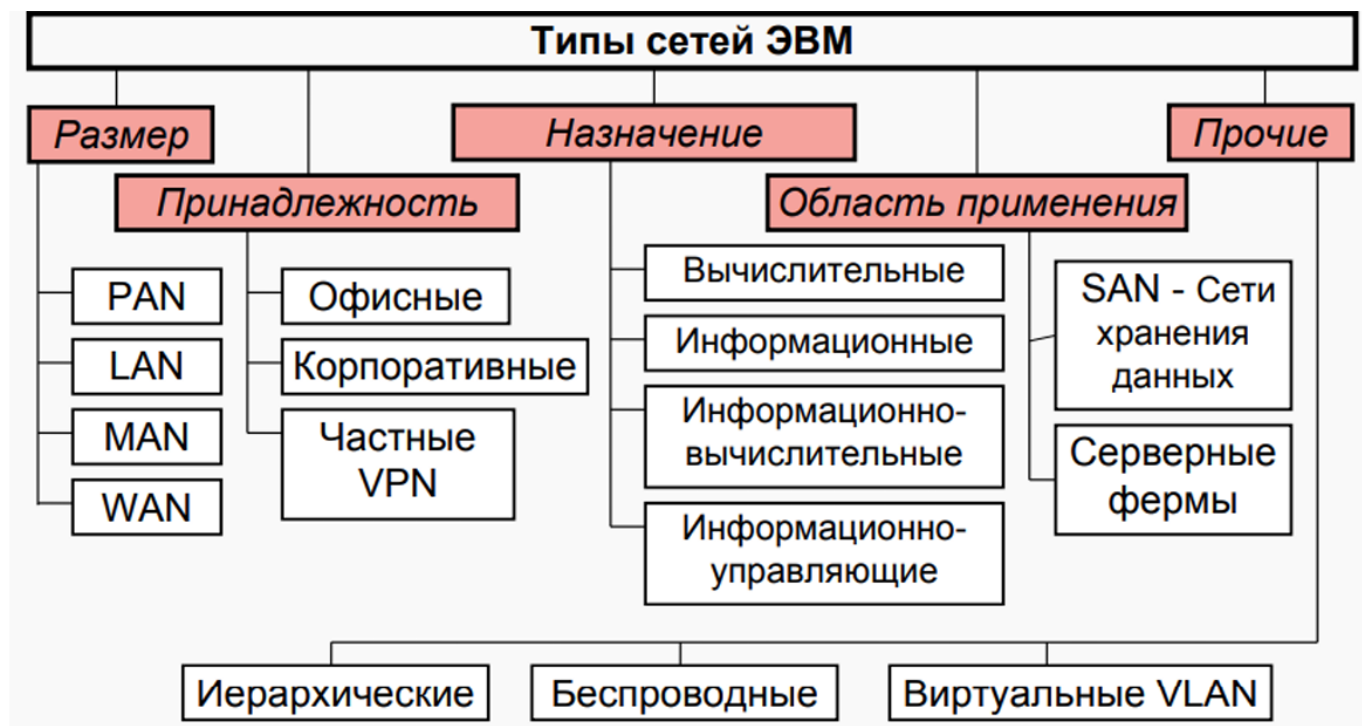
1. **Физический уровень:** Определяет физические средства передачи данных, такие как электрические сигналы или световые импульсы.
2. **Канальный уровень:** Обеспечивает адресацию в пределах локальной сети и обнаружение ошибок. Используются MAC-адреса и протокол Ethernet.
3. **Сетевой уровень:** Отвечает за маршрутизацию данных между различными сегментами сети. Используются протоколы IPv4 и IPv6.
4. **Транспортный уровень:** Обеспечивает передачу данных между узлами сети. Протоколы TCP и UDP работают на этом уровне.
5. **Сеансовый уровень:** Управляет установлением, поддержанием и завершением сеансов связи.

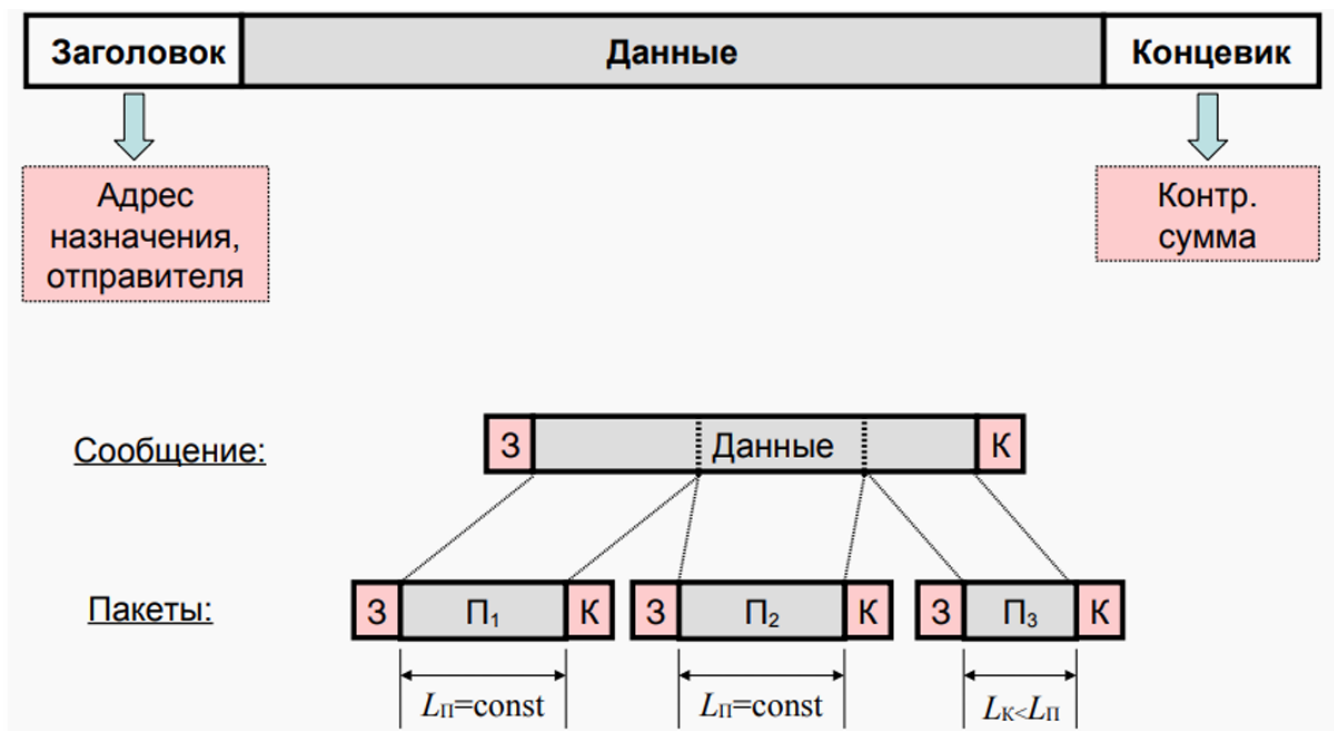
6. **Уровень представления:** Преобразует данные в формат, пригодный для прикладного уровня, такие как кодировки ASCII или форматы изображений JPEG.
7. **Прикладной уровень:** Предоставляет интерфейс для приложений, таких как браузеры и почтовые клиенты, работающие по протоколу HTTP, FTP и др.

Изображение

На изображении представлена классификация сетей ЭВМ по разным критериям:

- **Размер:** PAN, LAN, MAN, WAN.
- **Принадлежность:** Офисные, Корпоративные, Частные VPN.
- **Назначение:** Вычислительные, Информационные, Информационно-вычислительные, Информационно-управляющие.
- **Область применения:** SAN (Сети хранения данных), Серверные фермы.
- **Прочие типы:** Иерархические, Беспроводные, Виртуальные VLAN.





32) Модель TCP/IP: передающая среда, канальный и сетевой уровень.

Адресация, передача и маршрутизация пакетов.

Модель TCP/IP

Модель TCP/IP состоит из пяти уровней:

1. **Прикладной уровень (Application Layer):** Это уровень, на котором работают приложения, такие как веб-браузеры, почтовые клиенты и другие программы, которые взаимодействуют с пользователем и передают данные по сети.
2. **Транспортный уровень (Transport Layer):** Обеспечивает надежную передачу данных между узлами. Основные протоколы этого уровня:
 - **TCP (Transmission Control Protocol):** Обеспечивает надежную передачу данных с установлением соединения. Гарантирует доставку данных в правильном порядке.

- **UDP (User Datagram Protocol):** Предоставляет ненадежную передачу данных без установления соединения. Быстрее TCP, но не гарантирует доставку.
3. **Сетевой уровень (Network Layer):** Отвечает за адресацию и маршрутизацию данных. Основной протокол:
- **IP (Internet Protocol):** Отвечает за доставку пакетов данных (дейтаграмм) от источника к получателю. IP-пакеты могут идти разными маршрутами и приходить не по порядку.
4. **Канальный уровень (Data Link Layer):** Обеспечивает передачу данных между узлами в пределах одной сети. Работает с MAC-адресами (Media Access Control).
5. **Уровень передающей среды (Physical Layer):** Определяет физические средства передачи данных, такие как кабели (витая пара, коаксиальный кабель, оптическое волокно) и радиосигналы.

Уровень передающей среды

- **Коаксиальный кабель:**
 - Толстый (10Base-5): до 500 м.
 - Тонкий (10Base-2): до 50 м.
- **Витая пара:**
 - Категория 3: от 10 до 100 Мбит/с.
 - Категория 5е: 100 Мбит/с (2 пары), 1 Гбит/с (4 пары).
 - Категория 6: 10 Гбит/с (55 м).
 - Категория 7а: 40 Гбит/с (50 м), 100 Гбит/с (15 м).
- **Оптическое волокно:**
 - Multi-mode fiber: ~500 м.
 - Single Mode: ~80 км.
- **Беспроводные технологии (WiFi, WiMAX, 3G, 4G):**
 - 2.4, 5, 60 ГГц.
 - До 15 Гбит/с.

Канальный уровень (Ethernet)

Ethernet фрейм определяет формат данных, передаваемых по сети. Фрейм включает:

- **Preamble:** 8 октетов для синхронизации.
- **Destination MAC Address:** 6 октетов, адрес назначения.
- **Source MAC Address:** 6 октетов, адрес отправителя.
- **EtherType:** 2 октета, указывает тип протокола верхнего уровня.
- **Data:** от 46 до 1500 октетов данных.
- **CRC:** 4 октета для проверки ошибок.

MAC-адрес — уникальный 6-байтовый идентификатор устройства в сети.

Сетевой уровень (IP)

IP отвечает за доставку пакетов данных от источника к получателю. Пакеты могут идти разными маршрутами и приходить не по порядку.

Маршрутизация

Маршрутизация — процесс определения лучшего маршрута для передачи данных:

- **Маршрутизаторы:** Устройства, определяющие оптимальный маршрут для пакетов.
- **Настроенные компьютеры:** Компьютеры с несколькими сетевыми интерфейсами и специализированным ПО.

Таблица маршрутизации — база данных на маршрутизаторе, содержащая соответствие адресов назначения и интерфейсов для отправки данных.

Содержит:

- Адрес сети или узла.
- Маску подсети.
- Сетевой шлюз.
- Интерфейс.
- Метрику (показатель предпочтительности пути).

Разбор изображений

Фреймы и MAC-адреса

На изображении показан формат Ethernet фрейма, включающий:

- Заголовок (Preamble, MAC адреса источника и назначения, EtherType).
- Данные (Payload).
- CRC (для проверки ошибок).

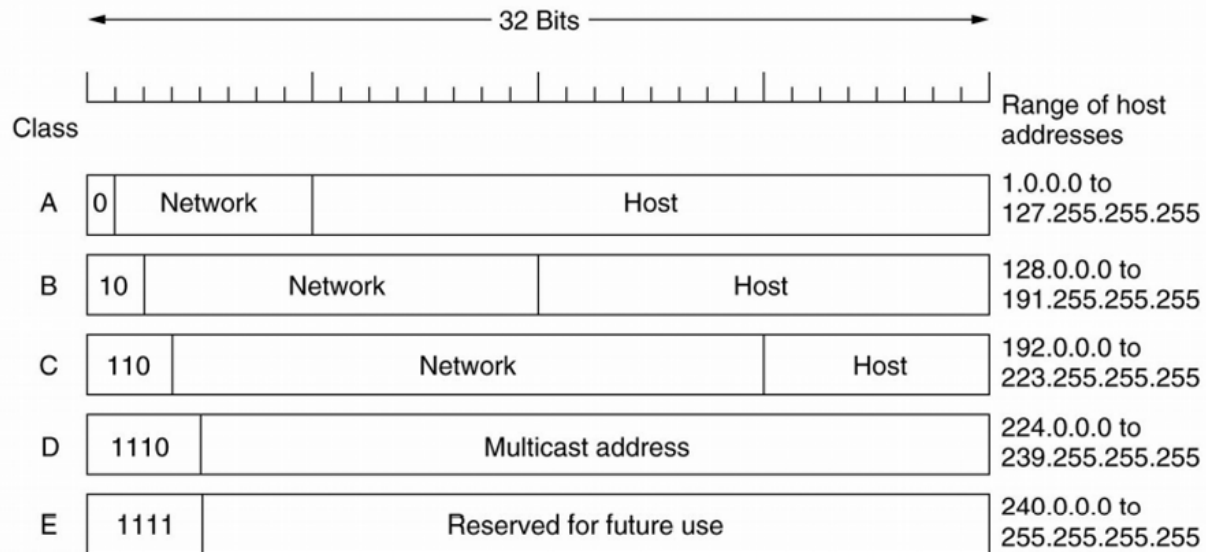
Показаны MAC-адреса и команда `arp -a`, выводящая таблицу соответствия IP-адресов и MAC-адресов в локальной сети.

Адресация в IPv4 сетях

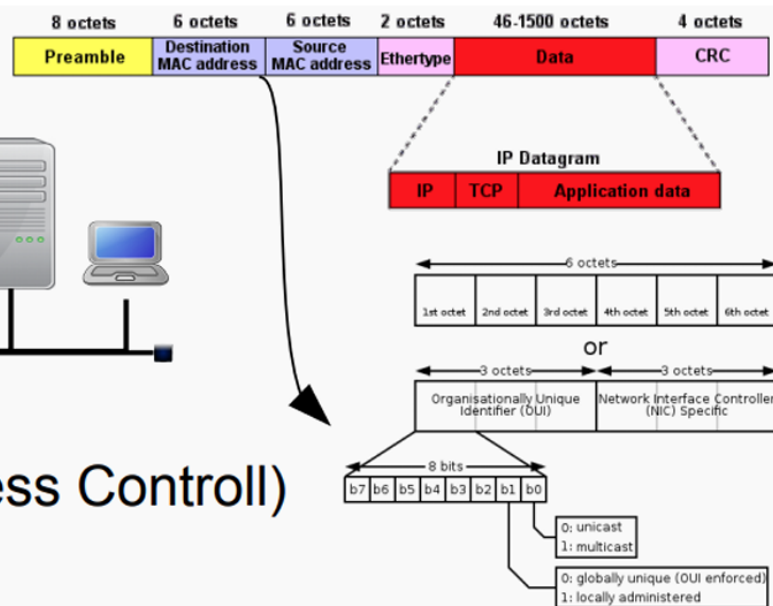
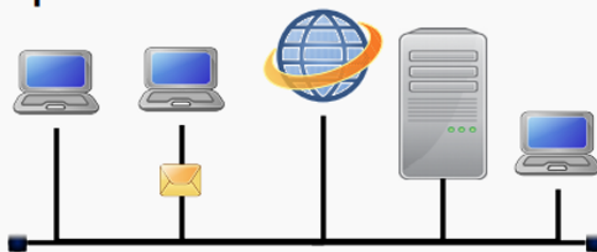
IPv4 адресация делится на классы:

- **Класс А:** Диапазон от 1.0.0.0 до 127.255.255.255.
- **Класс В:** Диапазон от 128.0.0.0 до 191.255.255.255.
- **Класс С:** Диапазон от 192.0.0.0 до 223.255.255.255.
- **Класс D:** Используется для мультикаст-адресов, диапазон от 224.0.0.0 до 239.255.255.255.
- **Класс Е:** Зарезервирован для будущего использования.

• Адресация в IPv4 сетях



• Фреймы



• MAC (Media Access Control)

```
serge@helios:/home/serge> arp -a
Net to Media Table: IPv4
```

Device	IP Address	Mask	Flags	Phys Addr
igb0	192.168.10.211	255.255.255.255	o	00:0c:29:6c:7d:93
igb0	sunrise.cs.ifmo.ru	255.255.255.255	o	00:00:5e:00:01:0a
igb0	192.168.10.214	255.255.255.255	o	00:0c:29:99:13:e3
igb0	224.101.101.101	255.255.255.255		01:00:5e:65:65:65

Команда `arp -a` показывает таблицу ARP (Address Resolution Protocol), которая отображает соответствие IP-адресов и MAC-адресов для каждого сетевого интерфейса. Эта таблица используется для преобразования IP-адресов в MAC-адреса, необходимые для передачи данных в локальной сети.

Структура вывода

1. **Интерфейс:** Указывает сетевой интерфейс, к которому относится запись (например, `192.168.31.34`).
2. **Адрес в Интернете:** IP-адрес устройства в сети.
3. **Физический адрес:** MAC-адрес, связанный с данным IP-адресом.
4. **Тип:** Тип записи в таблице (динамический или статический).

Пример разбора

- **Интерфейс: 192.168.31.34 — 0x5**
 - **192.168.31.1** (Физический адрес: `ec-41-18-d7-e7-22` , Тип: динамический)
 - Это IP-адрес устройства в сети с MAC-адресом `ec-41-18-d7-e7-22` , добавленный динамически (например, при отправке ARP-запроса).
 - **192.168.31.255** (Физический адрес: `ff-ff-ff-ff-ff-ff` , Тип: статический)
 - Это широковещательный адрес сети с соответствующим MAC-адресом. Запись статическая.
 - **224.0.0.2** (Физический адрес: `01-00-5e-00-00-02` , Тип: статический)
 - Это мультикастовый IP-адрес с соответствующим MAC-адресом.
 - **224.0.0.22** (Физический адрес: `01-00-5e-00-00-16` , Тип: статический)
 - Ещё один мультикастовый IP-адрес с соответствующим MAC-адресом.
- **Интерфейс: 192.168.56.1 — 0xd**
 - Похожие записи, но для другого интерфейса (например, виртуального интерфейса, используемого виртуальными машинами).
- **Интерфейс: 172.23.48.1 — 0x32**

- Записи для интерфейса в другой подсети, например, для другой сетевой карты или адаптера.

Детали записи

- **Динамический тип:** Запись добавляется автоматически в процессе работы сети, когда устройство отправляет или получает ARP-запросы.
- **Статический тип:** Запись добавляется вручную и сохраняется до перезагрузки или удаления.

ARP (Address Resolution Protocol)

ARP — это протокол, используемый для сопоставления IP-адресов с физическими MAC-адресами. Когда устройство в сети хочет отправить данные на другой компьютер, оно использует ARP для определения MAC-адреса, соответствующего IP-адресу назначения.

33) Модель TCP/IP: выделение адресов (DHCP), сервисы имен, транспортный и прикладной уровни.

Выделение адресов (DHCP)

DHCP (Dynamic Host Configuration Protocol) — это протокол, который автоматически назначает IP-адреса устройствам в сети. Это позволяет избежать ручной настройки IP-адресов, особенно в больших сетях, где это может быть трудозатратно и подвержено ошибкам.

Принцип работы DHCP

Получение IP-адреса через DHCP происходит в четыре шага, известные как процесс DORA:

1. **Discovery (Поиск):** Клиент, который еще не имеет IP-адреса, отправляет широковещательное сообщение DHCPDISCOVER, чтобы найти DHCP-серверы в сети.

2. **Offer (Предложение):** DHCP-серверы отвечают на запрос сообщением DHCPOFFER, предлагая IP-адрес из своего пула адресов.
3. **Request (Запрос):** Клиент принимает предложение и отправляет DHCPREQUEST серверу, подтверждая, что он хочет использовать предложенный IP-адрес.
4. **Acknowledgement (Подтверждение):** Сервер подтверждает запрос сообщением DHCPACK, завершая процесс назначения IP-адреса и указывая срок его действия.

Сервисы имен (DNS)

DNS (Domain Name System) — это система, которая преобразует удобные для человека доменные имена (например, www.example.com) в IP-адреса (например, 192.0.2.1), используемые для маршрутизации трафика в сети.

Принцип работы DNS

1. **Запрос имени:** Когда пользователь вводит доменное имя в браузере, запрос отправляется на локальный DNS-сервер.
2. **Поиск:** Если локальный DNS-сервер не знает IP-адрес, он пересылает запрос к корневым DNS-серверам.
3. **Иерархия:** Корневые серверы направляют запрос к серверу домена верхнего уровня (например, для .com), который затем направляет запрос к серверу конкретного домена (example.com).
4. **Ответ:** Когда DNS-сервер находит соответствие имени и IP-адреса, он отправляет ответ обратно пользователю.

DNS также поддерживает **обратное сопоставление**, когда по IP-адресу можно получить доменное имя. Для этого используется домен in-addr.arpa.

Транспортный уровень

Транспортный уровень обеспечивает передачу данных между приложениями на разных устройствах в сети. Основные протоколы транспортного уровня:

1. TCP (Transmission Control Protocol):

- Надежный протокол с установлением соединения.
- Гарантирует доставку данных в правильном порядке.
- Используется в приложениях, где важна надежность передачи, таких как HTTP, FTP, SSH, SMTP.

2. UDP (User Datagram Protocol):

- Ненадежный протокол без установления соединения.
- Не гарантирует доставку данных и порядок их получения.
- Используется в приложениях, где важна скорость передачи, таких как DNS, TFTP, DHCP.

Прикладной уровень

Прикладной уровень включает протоколы, используемые приложениями для связи по сети. Примеры таких протоколов:

- **HTTP (Hypertext Transfer Protocol):** Используется для передачи веб-страниц.
- **FTP (File Transfer Protocol):** Используется для передачи файлов.
- **SMTP (Simple Mail Transfer Protocol):** Используется для передачи электронной почты.
- **DNS (Domain Name System):** Используется для преобразования доменных имен в IP-адреса.
- **DHCP (Dynamic Host Configuration Protocol):** Используется для автоматического назначения IP-адресов.

Разбор изображений

DHCP

Изображение показывает процесс DORA:

1. **DHCPDISCOVER:** Клиент отправляет широковещательное сообщение для поиска DHCP-сервера.
2. **DHCPOFFER:** Сервер отвечает предложением IP-адреса.

3. **DHCPREQUEST**: Клиент принимает предложенный IP-адрес.

4. **DHCPACK**: Сервер подтверждает назначение IP-адреса.

DNS

Изображение показывает иерархическую структуру DNS:

- **Корневой домен**: Самый верхний уровень, обозначается точкой (".").
- **Домен верхнего уровня**: Например, .com, .ru.
- **Вторичный уровень**: Например, example в example.com.

Процесс запроса и ответа:

1. Пользователь вводит доменное имя в браузере.
2. Запрос отправляется на локальный DNS-сервер.
3. Если локальный сервер не знает ответа, он пересылает запрос по цепочке DNS-серверов.
4. Когда IP-адрес найден, ответ возвращается пользователю.

34) Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера.

Интерфейсы ввода-вывода

Интерфейсы определяют конкретные детали обмена данными между устройствами. Это включает:

- Частоту передачи данных.
- Набор каналов передачи.
- Способы кодирования данных.
- Команды и представление данных.

- Набор данных и последовательность их передачи.

Интерфейсы могут быть реализованы как аппаратно, так и программно. Для корректной работы необходимо, чтобы обе стороны обмена однозначно понимали детали обмена, что требует точной спецификации и стандартизации.

Уровни стандартизации интерфейсов

Стандартизация интерфейсов включает:

1. **Логическое подключение:** Определяет логическую структуру и протоколы обмена данными.
2. **Физические параметры сигналов:** Определяет характеристики физических сигналов, такие как напряжение и форма сигнала.
3. **Конструктивные особенности:** Определяет физические характеристики и разъемы, используемые для подключения устройств.

Контроллеры внешних устройств

Контроллеры внешних устройств (ВУ) обеспечивают связь между процессором и внешними устройствами. Они выполняют функции по управлению обменом данными и адаптации сигналов.

Подключение внешних устройств через контроллеры

1. **Форматы данных и режимы работы ВУ:** Контроллеры должны поддерживать форматы данных и режимы работы конкретных ВУ, что требует разнообразных контроллеров от простых до сложных.
2. **Тип системного интерфейса микроЭВМ:** Определяет способ организации схем контроллеров ВУ для связи с системными шинами и схемами распознавания адресов.

Контроллеры могут обеспечивать:

- Связь с ВУ по стандартному параллельному каналу.
- Связь с ВУ по стандартному последовательному каналу.

- Преобразование аналоговой информации в цифровую и наоборот.
- Программируемые контроллеры, которые настраиваются специальными командами или программами обмена с ВУ.

Циклы обмена и регистры контроллера

Контроллеры внешних устройств управляют обменом данными с помощью регистров, которые включают:

- **Входной регистр:** Хранит данные, полученные от ВУ.
- **Выходной регистр:** Хранит данные, отправляемые к ВУ.
- **Регистр состояния:** Содержит информацию о состоянии ВУ.
- **Регистр управления:** Используется для управления работой контроллера и ВУ.

Разбор изображений

Изображение 1

На этом изображении показаны два уровня сопряжения:

1. **Процессор ↔ Контроллеры:** Процессор связывается с контроллерами через системную шину.
2. **Контроллеры ↔ ВУ:** Контроллеры связываются с внешними устройствами через периферийные шины.

Контроллеры управляют передачей данных между процессором и внешними устройствами, используя регистры для хранения и управления данными.

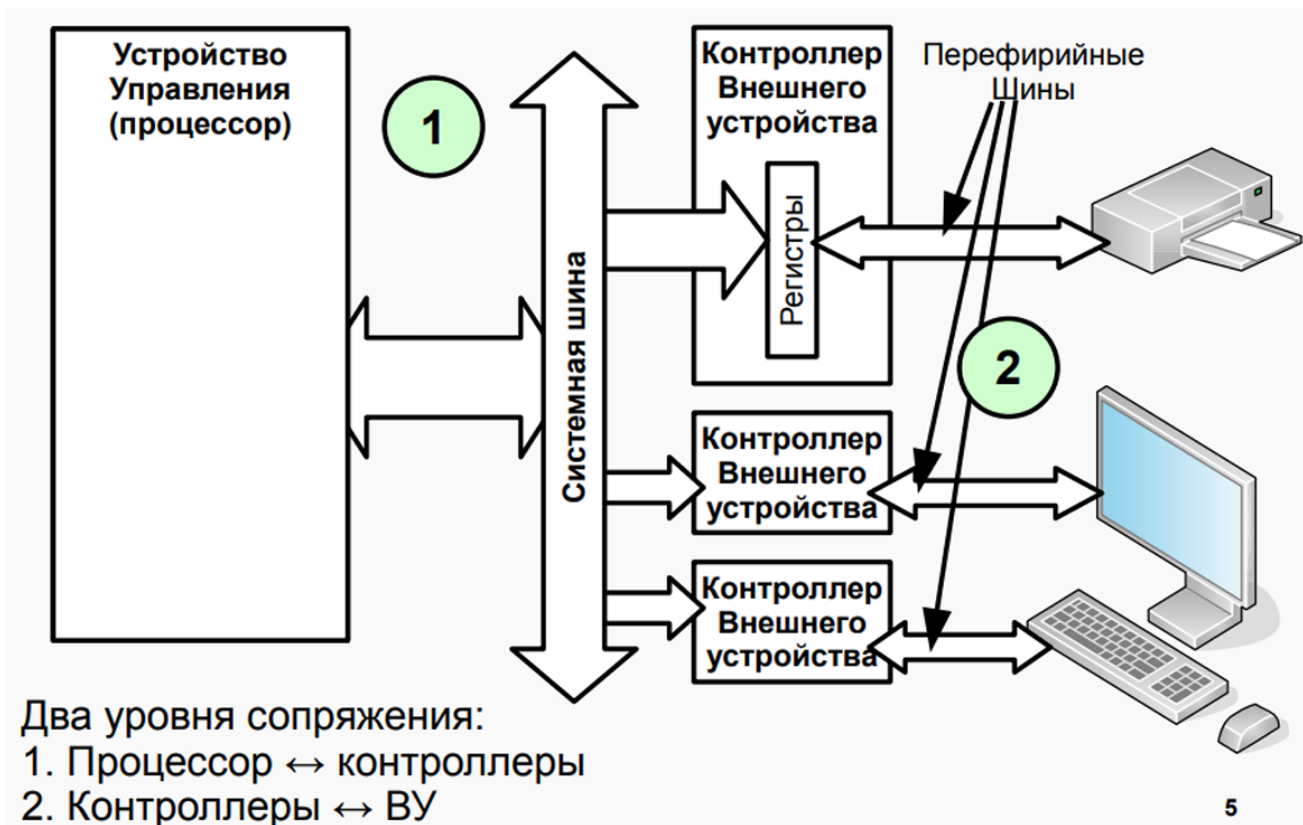
Изображение 2

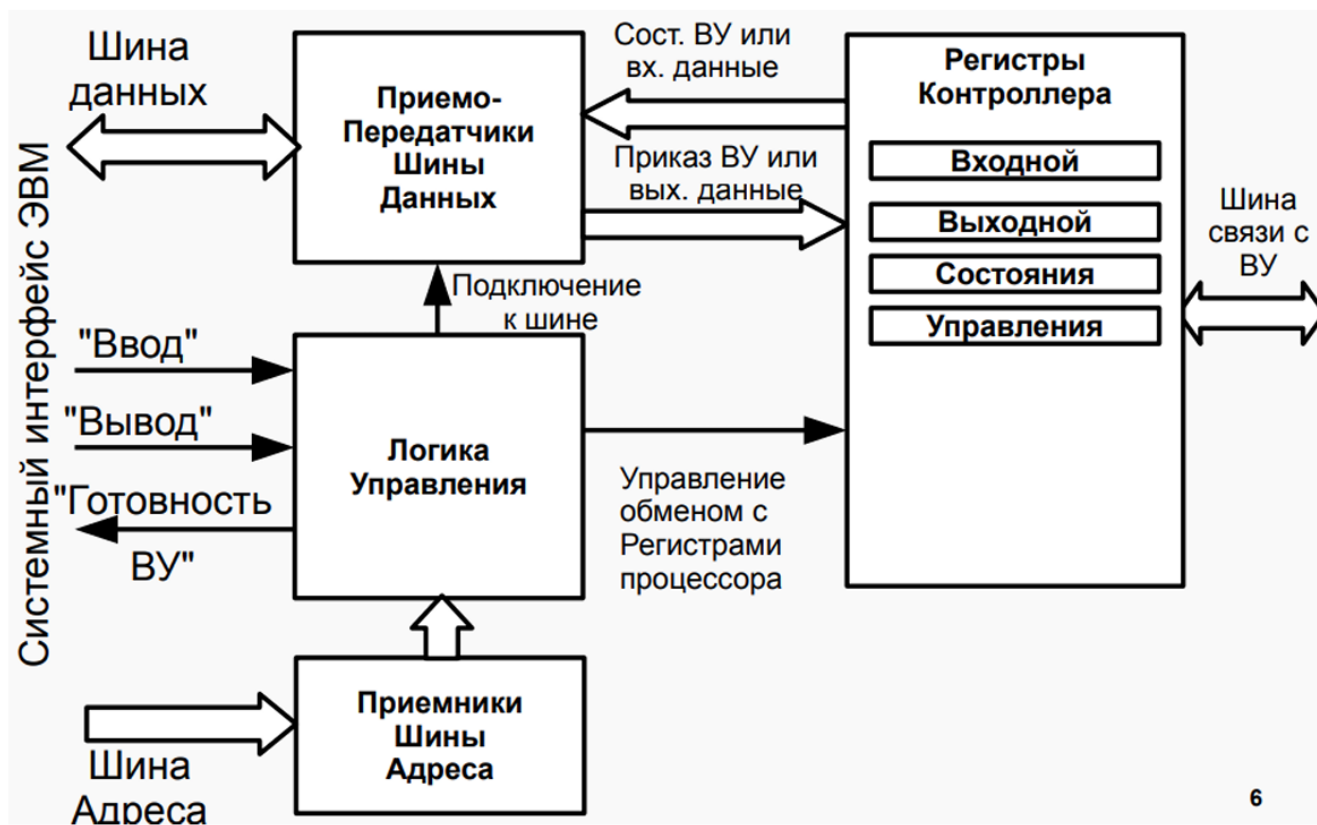
На этом изображении показан системный интерфейс ЭВМ и структура контроллера:

- **Шина данных и шина адреса:** Передают данные и адреса между процессором и контроллерами.

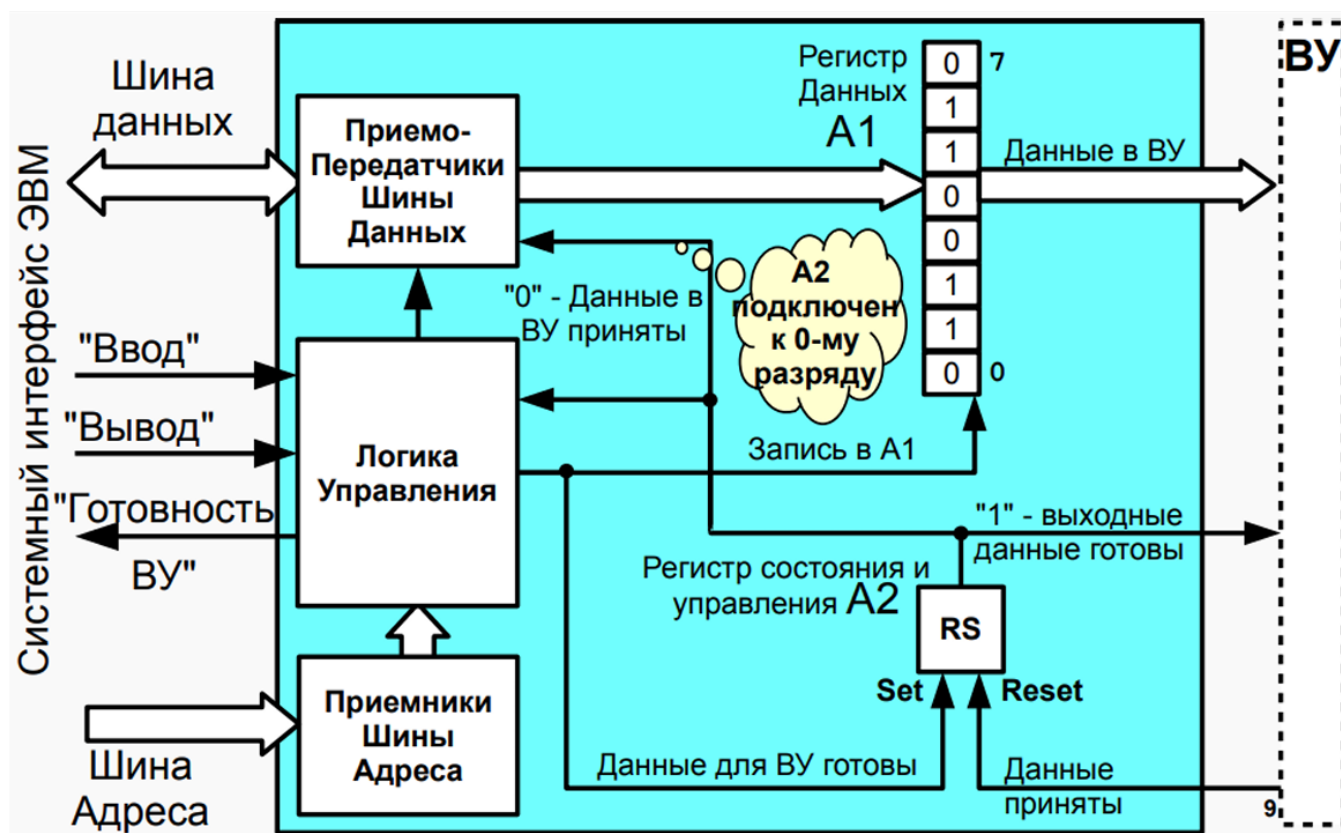
- **Приемо-передатчики шины данных:** Управляют передачей данных между шиной и регистром контроллера.
- **Логика управления:** Отвечает за управление обменом данными и взаимодействие с процессором и ВУ.
- **Регистры контроллера:** Включают входной, выходной, регистр состояния и управления.

Эти компоненты обеспечивают организацию и управление обменом данными между процессором и внешними устройствами.





35) Параллельная передача данных. Контроллеры параллельной передачи и приема.



Параллельная передача данных

Параллельная передача данных означает, что несколько бит данных передаются одновременно по отдельным линиям связи. Это позволяет достичь высокой скорости передачи данных. Однако для параллельной передачи требуется больше проводов и сложнее реализовать синхронизацию.

Контроллеры параллельной передачи и приема

Контроллеры параллельной передачи и приема управляют процессом обмена данными между процессором и внешними устройствами (ВУ). Эти контроллеры обеспечивают корректную передачу данных и обмен управляющими сигналами.

Параллельная передача данных в ВУ под управлением программы асинхронного обмена

1. Процессор проверяет готовность ВУ:

- Процессор проверяет регистр состояния и управления (A2) контроллера, чтобы узнать, готово ли ВУ к приему данных.
- Если ВУ готово (в регистре A2 логический 0), данные передаются из шины данных в регистр данных (A1) контроллера и далее в ВУ.

2. Передача данных:

- Данные передаются из шины данных системного интерфейса в регистр данных A1 контроллера и далее в ВУ.
- В регистр состояния и управления (A2) записывается логическая единица, что означает готовность выходных данных.

3. Подтверждение приема данных:

- ВУ, приняв данные, посылает управляющий сигнал "Данные приняты", который обнуляет регистр состояния и управления (A2).
- Это формирует сигнал "Готовность ВУ", который передается процессору.

Алгоритм асинхронного ввода

1. Процессор проверяет наличие данных:

- Процессор проверяет регистр состояния и управления (A2), чтобы узнать, есть ли данные в регистре данных (A1).
- Если данные готовы (в регистре A2 логическая 1), они передаются из регистра данных (A1) в шину данных и далее в регистр процессора или ячейку памяти.

2. Передача данных:

- Данные передаются из регистра данных (A1) в шину данных системного интерфейса и далее в процессор или память.
- Процессор обнуляет регистр состояния и управления (A2), что формирует сигнал "Данные приняты".

Разбор изображений

Изображение 1

На этом изображении показан процесс параллельной передачи данных из процессора в ВУ.

- **Шина данных:** Используется для передачи данных между процессором и контроллером.
- **Шина адреса:** Определяет адрес устройства, с которым идет обмен данными.
- **Логика управления:** Управляет обменом данными и состоянием контроллера.
- **Приемо-передатчики шины данных:** Управляют передачей данных между шиной и регистром данных (A1).
- **Регистр данных (A1):** Хранит передаваемые данные.
- **Регистр состояния и управления (A2):** Управляет состоянием передачи данных (0 - данные приняты, 1 - выходные данные готовы).

Процесс:

1. Процессор отправляет данные в регистр данных (A1).
2. Логика управления устанавливает бит в регистре состояния и управления (A2), сигнализируя, что данные готовы к отправке.
3. ВУ принимает данные и отправляет сигнал "Данные приняты", обнуляя регистр состояния (A2).

Изображение 2

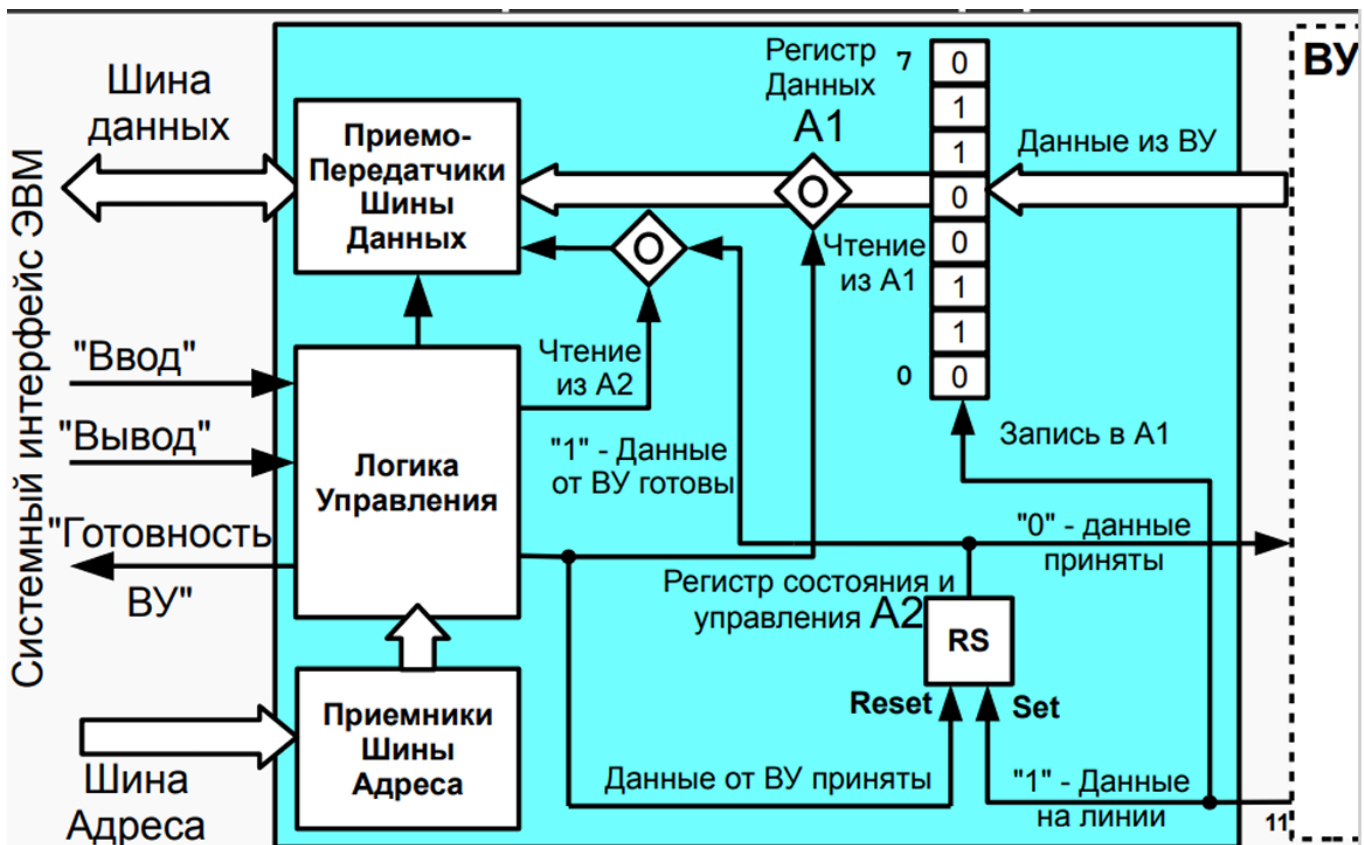
На этом изображении показан процесс параллельной передачи данных из ВУ в процессор.

- **Шина данных:** Используется для передачи данных между контроллером и процессором.
- **Шина адреса:** Определяет адрес устройства, с которым идет обмен данными.
- **Логика управления:** Управляет обменом данными и состоянием контроллера.

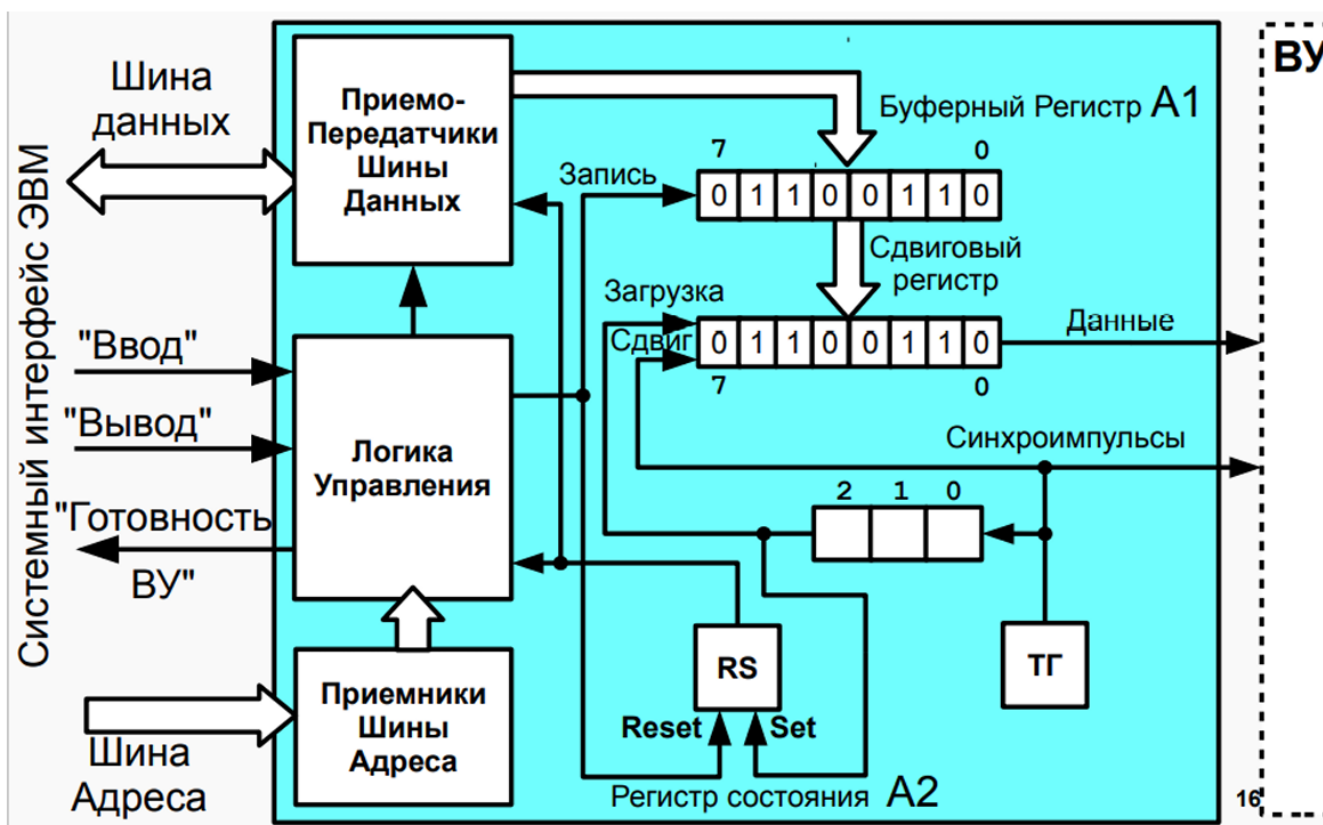
- **Приемо-передатчики шины данных:** Управляют передачей данных между шиной и регистром данных (A1).
- **Регистр данных (A1):** Хранит полученные данные.
- **Регистр состояния и управления (A2):** Управляет состоянием приема данных (0 - данные приняты, 1 - данные от ВУ готовы).

Процесс:

1. ВУ отправляет данные в регистр данных (A1) и устанавливает бит в регистре состояния (A2), сигнализируя, что данные готовы.
2. Процессор проверяет регистр состояния (A2) и читает данные из регистра данных (A1).
3. Процессор обнуляет регистр состояния (A2), сигнализируя, что данные приняты.



36) Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приема.

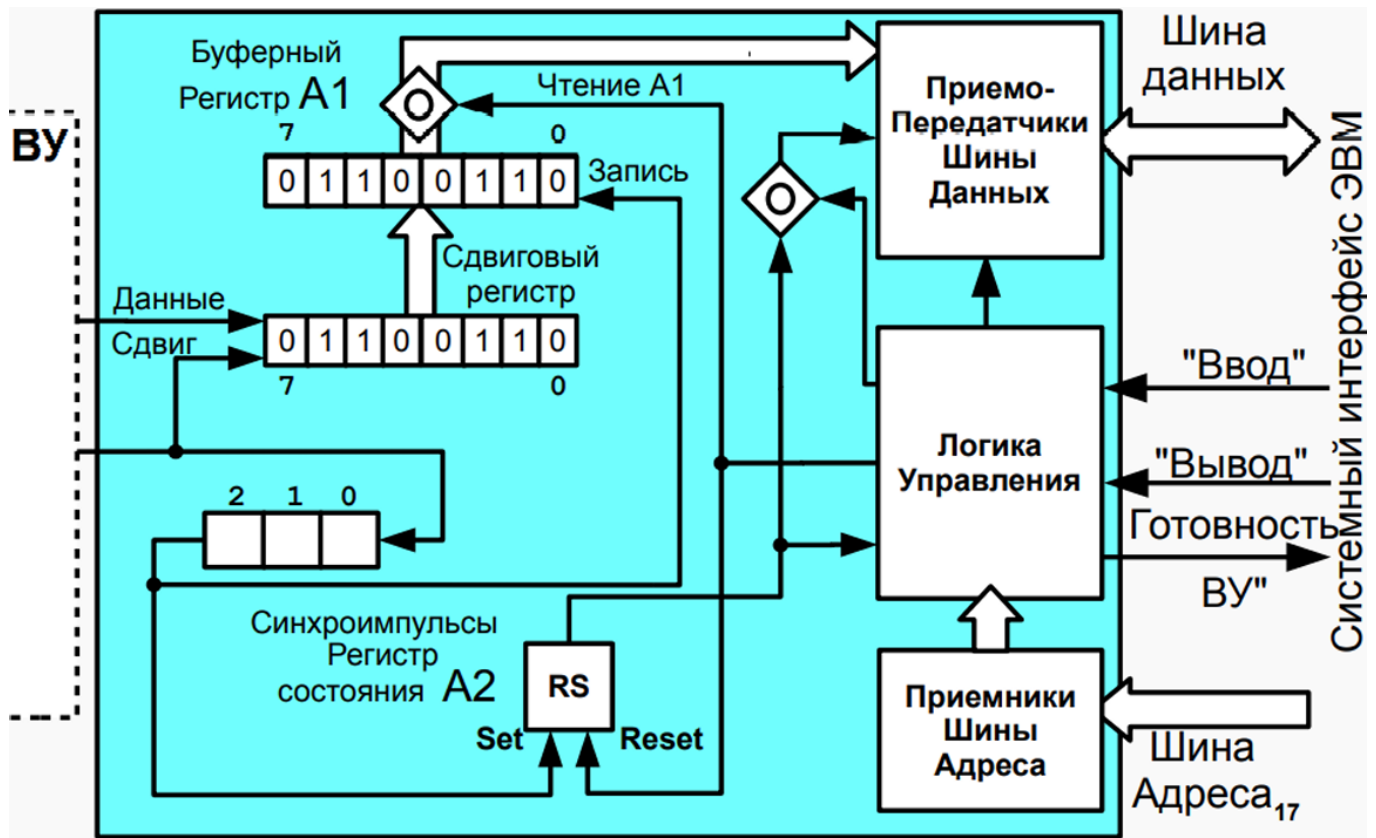


8-ми разрядный буферный регистр контроллера A1 - для временного хранения байта данных до его загрузки в сдвиговый регистр. Запись байта данных в буферный регистр происходит при наличии 1 в регистре состояния A2. Содержимое этого регистра передается в процессор по одной из линий шины данных и используется для формирования управ. сигнала «Готовность ВУ». При записи очередного байта в регистр A1 обнуляется регистр A2.

В сдвиговом регистре происходит преобразование данных из параллельного формата в последовательный и передача их в линию связи. По очередному тактовому импульсу содержимое сдвигового регистра сдвигается на 1 разряд вправо и в линию связи «Данные» выдается значение очередного разряда. Одновременно со сдвигом по линии «Синхронизация» передается тактовый импульс.

Количество переданных в линию тактовых сигналов (переданных бит) подсчитывается счетчиком тактовых импульсов. Как только его содержимое равно 7 (передано 8 бит информации) формируется управляющий сигнал «Загрузка» и происходит запись в сдвиговый регистр очередного байта из

буферного. Устанавливается в 1 регистр состояния. Следующим тактовым импульсом счетчик будет сброшен в 0 и начнется очередной цикл выдачи 8 бит из сдвигового регистра в линию связи.



Буферный регистр контроллера A1 - для временного хранения байта , поступившего из сдвигового регистра. Чтение байта данных из буферного регистра происходит при наличии 1 в регистре состояния A2.

Данные, поступающие из линии связи в последовательном коде преобразуются в параллельный с помощью сдвигового регистра. Линия «Данные» подключается в контроллере к последовательному входу сдвигового регистра, а линия «Синхронизация» - на управ. вход «Сдвиг» и на вход счетчика тактовых импульсов. По тактовому импульсу по линии «Синхронизация» производится сдвиг содержимого сдвигового регистра на 1 разряд влево и запись очередного бита данных из линии «Данные» в младший разряд этого регистра. Одновременно увеличивается на 1 счетчик тактовых импульсов. Как только он становится равным 7 формируется управ. сигнал «Запись» и происходит запись в буферный регистр байта из сдвигового. Устанавливается в 1 регистр состояния.

При передаче байта данных из буферного регистра в шину данных регистр состояния обнуляется (т.е. в сдвиговый регистр принимается очередной байт информации).

37. Асинхронный обмен. Принципы деления частоты, формат кадра.

Асинхронный обмен

При реализации асинхронного обмена интервал между командами передачи данных задается самим внешним устройством. Контроллеры этих устройств снабжаются регистром состояния, который информирует ЭВМ о готовности устройства к обмену информацией.

Обмен происходит по готовности ВУ, из этого вытекают следующие:

Преимущества:

1. Не нужно знать время выполнения операции на ВУ;
2. ВУ всегда успеет выполнить обработку данных перед началом следующей операции обмена.

Недостаток:

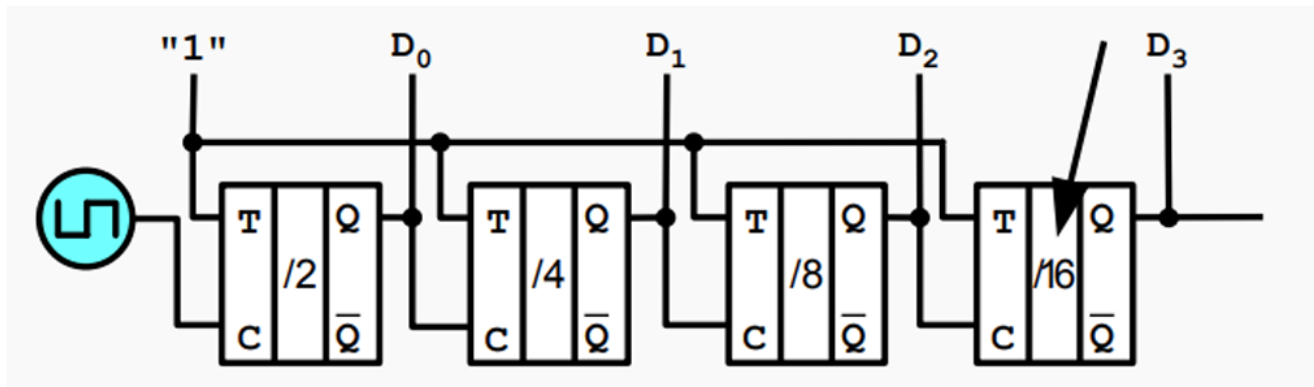
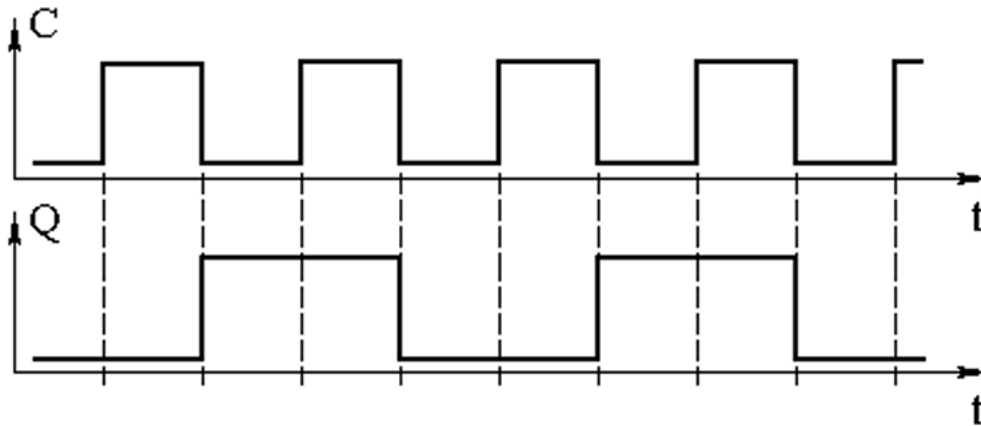
1. ЭВМ не выполняет никаких полезных действий во время ожидания момента готовности ВУ.

Принципы деления частоты

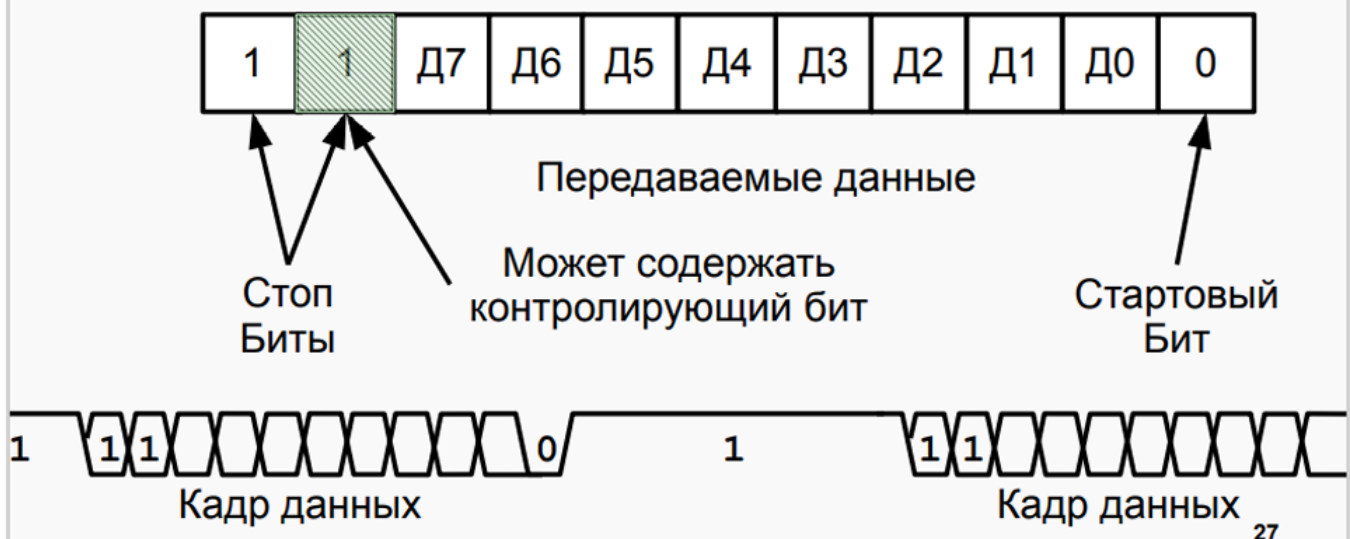
При асинхронной передаче данных, со временем может происходить рассинхронизация генераторов тактов передатчика и приёмника, в результате чего данные могут быть переданы с искажениями, или не быть переданы вовсе.

Одним из способов решения этой проблемы является деление тактовой частоты генераторов.

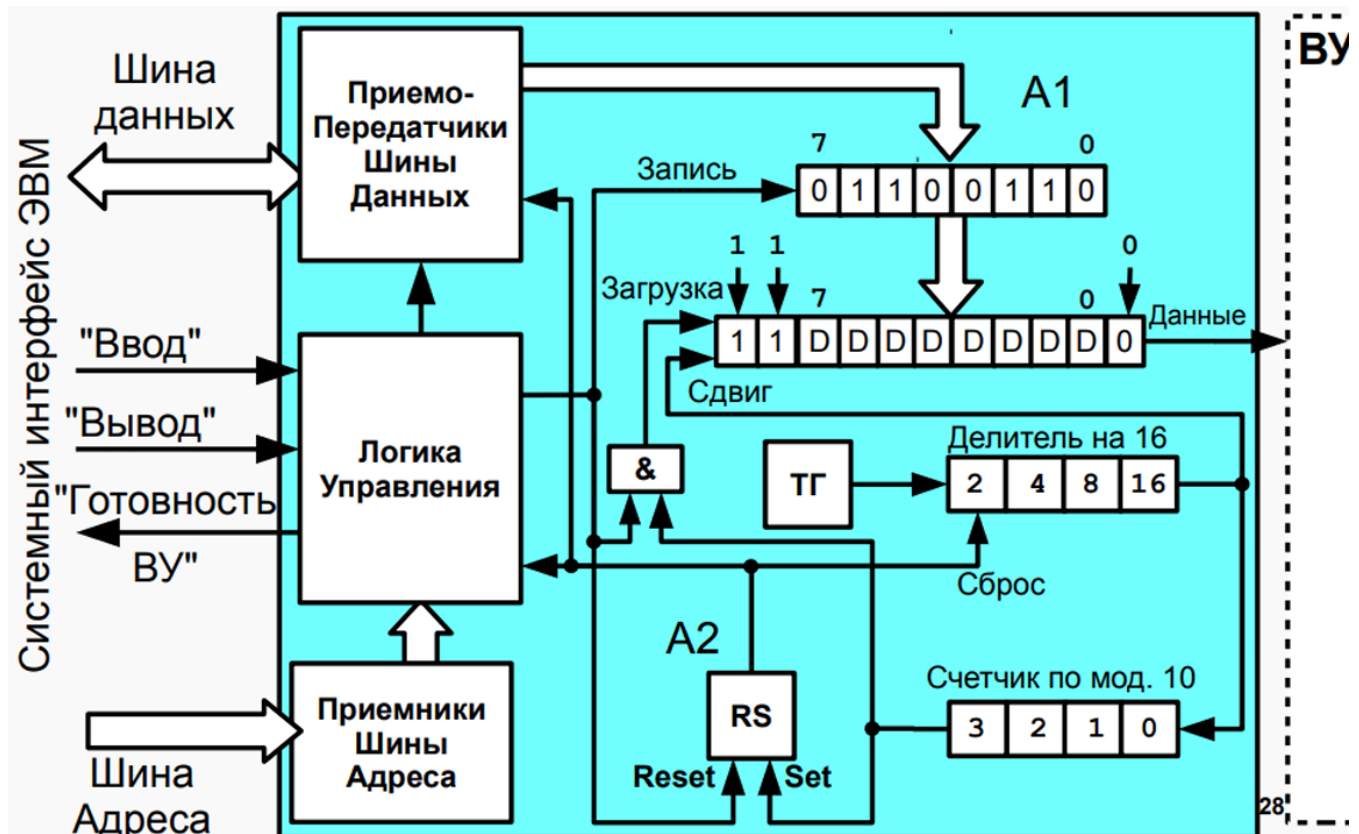
Принцип его заключается в том, что исходная тактовая частота делится с некоторым коэффициентом, чаще всего кратным степени двойки. Таким образом увеличивается область совпадения фаз и время передачи данных, что увеличивает точность передачи.



- Пример параметров настройки последовательного асинхронного порта 9600,8,n,1



38) Контроллер передачи асинхронного последовательного интерфейса.

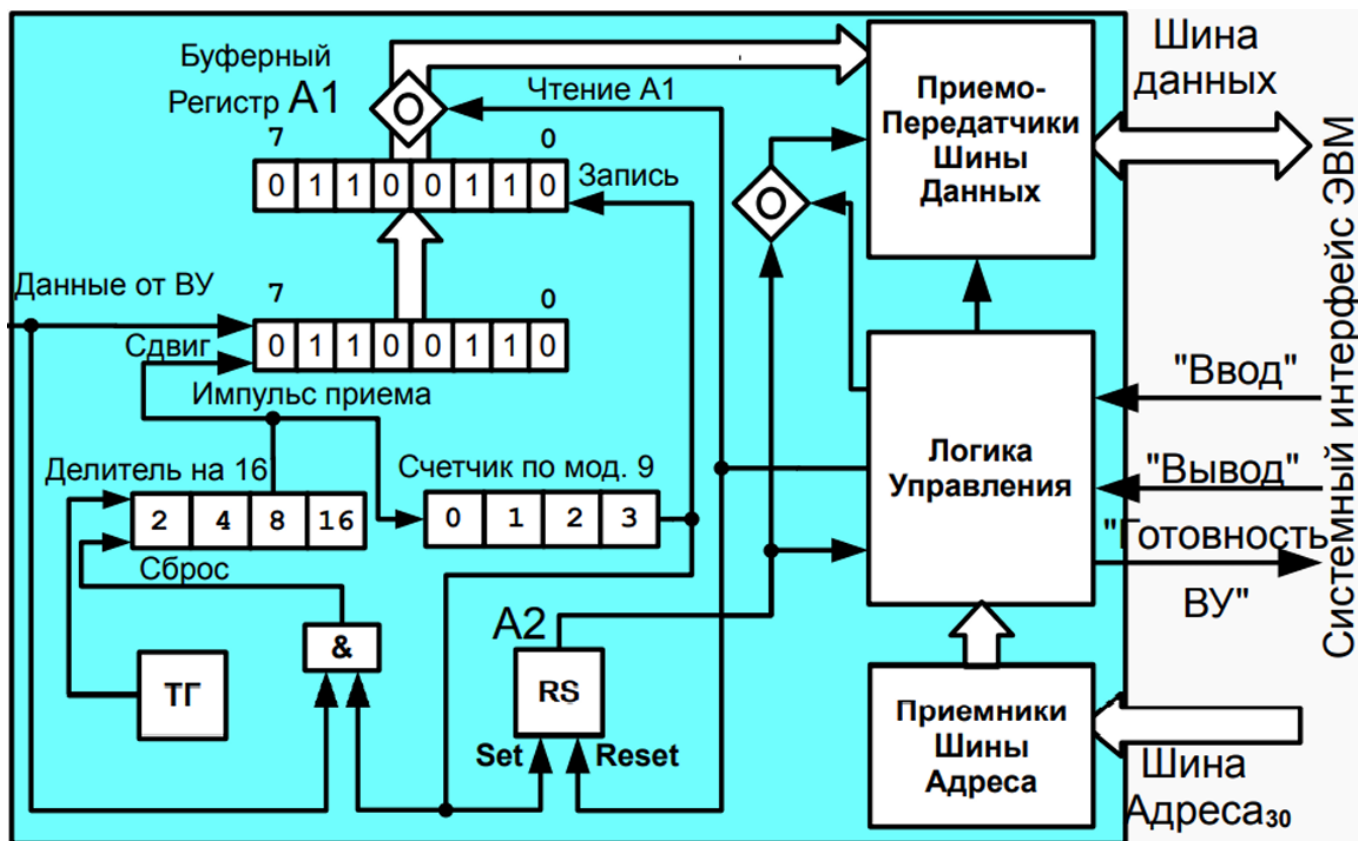


Процесс передачи

1. После передачи предыдущих байтов данных в Регистр Состояния A2 записывается 1, что информирует процессор о готовности контроллера к приему следующего байта данных и передаче его по линии связи в ВУ. Он же запрещает формирование импульсов со схемы выработки импульсов сдвига – делителя частоты тактового генератора на 16 (счетчик по mod 16). Счетчик импульсов сдвига (счетчик по mod 10) находится в нулевом состоянии, и его единичный выходной сигнал поступает на вентиль И, подготавливая цепь выработки сигнала загрузки сдвигового регистра.
2. Процессор, выполняя команду «Вывод», выставляет передаваемый байт на шине данных и формирует управляющий сигнал системного интерфейса «Вывод».

3. По сигналу «Вывод» в контроллере происходит запись передаваемого байта в буферный регистр A1, сброс регистра состояния A2 и формирование на вентиле И сигнала «Загрузка».
4. Передаваемый бит переписывается в разряды 1..8 сдвигового регистра, в 0 разряд записывается ноль – стартовый бит, а в разряды 9 и 10 единицы – стоповые биты.
5. Снимается сигнал «Сброс» с делителя частоты, он начинает накапливать импульсы генератора тактовой частоты и в момент приема шестнадцатого тактового импульса срабатывает импульс сдвига (так реализовано деление частоты).
6. На шине «Данные» поддерживается 0 (значение стартового бита) до тех пор, пока не будет выработан первый импульс сдвига (время передачи 1 бита). Импульс сдвига изменит состояние счетчика импульсов сдвига и перепишет в нулевой разряд сдвигового регистра первый информационный бит передаваемого байта данных. Значение этого бита будет поддерживаться на линии «Данные» до следующего импульса сдвига.
7. Аналогично передаются остальные информационные биты, первый стоповый бит, и, наконец второй стоповый бит, при передаче которого счетчик импульсов сдвига снова установится в нулевое состояние. Это приведет к записи 1 в регистр состояния A2. Единичный сигнал с выхода регистра A2 запретит формирование импульсов сдвига, и информирует о готовности к приему нового байта данных.
8. После завершения передачи очередного кадра (стартового бита, информационного бита и 2х- стоповых битов), на линии передачи данных поддерживается значение второго стопового бита – единицы

39) Контроллер приема асинхронного последовательные интерфейса.



Процесс передачи

1. На линии «Данные» находится единица, что запрещает работу делителя частоты генератора тактовых импульсов.
2. При обнаружении нулевого сигнала на линии «Данные» (смена стопового бита на стартовый), снимается сигнал «Сброс» с делителя частоты, он начинает накапливать импульсы генератора тактовой частоты.
3. Когда на счетчике накопится значение 8 (половина времени передачи бита), он выдаст сигнал, поступающий на входы сдвигового регистра и счетчика импульсов сдвига. (Таким образом уменьшается вероятность искажения данных.)
4. Последующие сдвиги происходят после прохождения 16-ти тактовых импульсов.
5. При приеме в сдвиговый регистр 9-го бита кадра (8-го инф. Бита), из него выдвинется стартовый бит, и, следовательно в сдвиговом регистра будет размещен информационный байт. В этот момент счетчик импульсов сдвига придет в нулевое состояние и на его выходе

будет выработан единичный сигнал, по которому:

- a. Содержимое сдвигового регистра будет переписано в БР
- b. В РС A2 запишется 1 и он будет информировать процессор об окончании приема очередного байта
- c. Вентиль И подготовится к выработке сигнала «Сброс»(он сформируется после прихода первого стопового бита).
- d. Получив сигнал готовности (1 в РС A2), процессор выполнит команду «Ввод». При этом вырабатывается сигнал системного интерфейса «Ввод», по которому производится пересылка принятого байта данных из БР в процессор (сигнал «Чтение») и сброс РС A2

40) Организация прямого доступа к памяти. Контроллер ПДП

Прямой доступ к памяти (ПДП) — это способ обмена данными между оперативной памятью и внешними устройствами, при котором контроллер ПДП берет на себя управление передачей данных, освобождая процессор для выполнения других задач. Контроллер ПДП подключается к шинам данных и адреса системного интерфейса ЭВМ, что не позволяет процессору и контроллеру ПДП одновременно использовать эти шины.

Чтобы решить эту проблему, используются два метода:

1. **Захват цикла:**

- **Простой захват цикла:** передача данных происходит в те машинные циклы, когда процессор не обращается к памяти. Это может быть сделано с помощью специального указывающего цикла или селектирующих схем.
- **Захват цикла с отключением процессора:** контроллер ПДП отправляет сигнал "Требование ПДП" (ТПДП), процессор приостанавливает выполнение команды и выдает сигнал "Предоставление ПДП" (ППДП), освобождая шины для контроллера ПДП.

2. **Блокировка процессора:** процессор передает управление шинами системного интерфейса контроллеру ПДП на все время обмена

блоком данных.

Контроллер ПДП для ввода данных из внешнего устройства (ВУ) в режиме "Захват цикла" работает следующим образом:

1. Процессор загружает в контроллер количество байтов для приема и начальный адрес в памяти.
2. Данные из ВУ поступают в регистр данных (РД) контроллера, сопровождаясь сигналом "Ввод данных".
3. При наличии данных контроллер формирует сигнал ТПДП.
4. Получив сигнал ППДП, контроллер передает данные в память.
5. После записи байта данных в память контроллер декрементирует счетчик байтов и обновляет регистр адреса. Когда счетчик становится равным нулю, контроллер прекращает формирование сигналов ТПДП.

Эти методы позволяют эффективно управлять передачей данных и повышать производительность системы за счет разгрузки процессора.

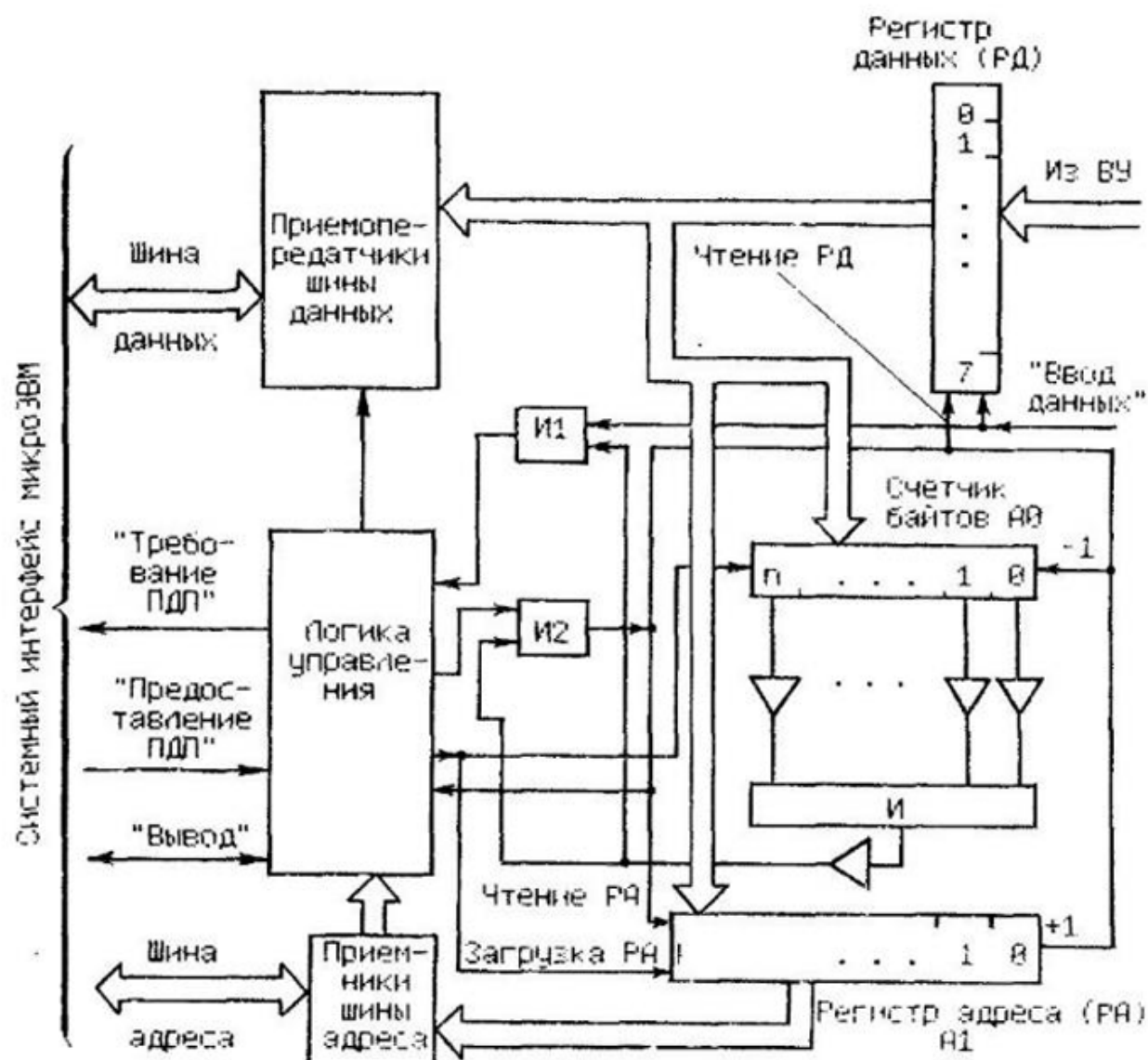


Рис. 8. 16. Контроллер ПДП для ввода данных из ВУ в режиме «Захват цикла»